



WRITING BOOKS IN THE TERMINAL

The Inkhaven Manual

The Complete Guide to Writing a Book in the Terminal

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

Contents

What Inkhaven is	1
How this manual is organised	2
This manual, and the companions	2
A word on the pictures	3
Part I — Getting Started	4
1 — Installing Inkhaven	5
What you are installing	6
Prerequisites	6
Supported platforms	8
Installing with Cargo	10
Building from source	12
The first run	14
Verifying the install	15
Troubleshooting the install	16
The CLI and the TUI	18
2 — Your First Project	21
The one command that starts everything	22
What init writes to disk	24
The anatomy of the manuscript tree	26
The node model at a glance	27
The system books	29
Opening the project and making your first book	31
Two ways to shape the tree first	33
3 — The Four Panes	36
One window, four regions	37
The right-hand region — three panes in one place	39
Moving your attention — focus and navigation	40
The chord system — the heart of the interface	42
The status bar and the title bars	44
Finding a chord — the palette and the quick reference	45
Fullscreen and focus modes	47
A note on the mouse	48
Part II — Writing	50
4 — The Editor	51
Opening a paragraph, and reading its state	52

Moving the cursor	53
Selecting text	54
The clipboard — cut, copy, and paste	55
Deleting by the chunk	56
Undo and redo	57
Every change since the last save	57
Finding and replacing	57
Saving, and the round trip	59
Split-edit — two versions at once	60
Grammar check — F7 and the g-apply	61
The reading-pace preview	63
5 — The Tree and Its Files	65
What a tree row says	66
Moving through the tree	68
Building the tree	70
Reshaping the tree	72
Deleting, and getting it back	73
The leaf types	75
The morph cycle — turning one leaf into another	80
6 — Snapshots and History	83
What a snapshot is	84
When snapshots are taken	85
Taking a snapshot — F5 and the annotation prompt	86
The per-paragraph picker — F6	87
The project-wide snapshot browser — Ctrl+F6	90
Deleted-paragraph history — the kill-ring	91
The crash mirror and recovery	93
The configuration knobs	95
7 — Search	98
Two kinds of search	99
The classic case — the lantern that never appears	100
The Search bar — asking the whole book a question	101
What makes a good query — and what does not	104
Finding the paragraph like this one — similar-paragraph mode	105
Under the hood — how meaning becomes a search	106
Searching within a scope — Facts search	109
Keeping the index healthy — reindex	110
Search from the shell — the CLI	111

Search from a script — the Bund word	112
8 — The Style Overlays	115
The filter-word overlay — Ctrl+B Shift+F	116
The echo overlay — Ctrl+B Shift+K	120
Show, don't tell — the overlay and the AI scan	122
The terminology overlay — Ctrl+V z	123
The POV chip — Ctrl+B Shift+P	125
The sentence-rhythm gauge — Ctrl+B Shift+H	125
The reader-pace preview — Ctrl+B Shift+E	127
Master switches and session overrides	128
Part III — The AI Assistant	132
9 — The AI Assistant	133
Opening and focusing the pane	134
The title bar — the pane's state at a glance	135
Streaming inference	136
Scope — telling the AI what to look at	137
Mode — local-only RAG versus full knowledge	140
Destination — where an answer lands	142
Chat history — the conversation is continuous	143
Prompt templates and the Help! prefix	144
Providers — six bundled, and any the router reaches	144
Prompt-language resolution	146
10 — Chat With Your Book	148
The scope dial — F9 and what an answer stands on	149
Book scope — retrieval-augmented conversation	150
What's in the pool — the retrieval scope	154
Tuning retrieval — the book_rag config block	154
Inspecting retrieval from the terminal	156
Facts scope — chat against the world's invariants	157
Graph scope — chatting with how your book connects	157
The reader conversations — Socrates and Editor	159
Living with the chat — cost and habits	159
Scripting retrieval — the ink.book_rag.* words	160
11 — Reusable Prompts	163
Why keep a prompt around	164
Two libraries: the file and the book	164
Opening the picker	165
Direct invocation: slash a name	167

The substitutions	167
System prompts in prompts.hjson	169
Book prompts in the Prompts book	170
Localized prompts	171
Named flows and the resolution ladder	172
From the command line	173
Scripting: setting the system prompt	174
How prompts sit beside scope and mode	174
12 — Watching the Cost	176
Cost informs, it never blocks	177
Almost everything is free	177
Where the money goes	178
The cost dashboard	179
The daily caps	181
How usage is tracked	183
A field guide to free versus paid	185
Part IV — The World & Its Facts	187
13 — Places and Characters	188
Two books, seeded for you	189
Recording a place, recording a person	190
The highlight overlays	192
Asking the AI: Ctrl+B P and Ctrl+B C	194
Character arcs — a first tour (CHAR-1)	196
How the roster feeds the rest of Inkhaven	200
14 — The World	202
The two halves	203
Declaring the physics	204
The compiler and what it derives	207
The World hub — Ctrl+B W	208
Maps — the plakat cartographer	209
The interactive worldbuilder — at a glance	210
Coherence for imagined societies — utopia	212
The live fact-checker	213
Facts, and grounding the AI	215
The ink.world Bund surface	216
15 — The Knowledge Graph	218
The edge — a typed line between two nodes	219
Populating the graph	222

The graph command	224
Chatting with your graph	226
From a script — the ink.graph words	229
Multilingual by construction	229
Storage and crash-safety	230
16 — The Timeline	232
Turning the timeline on	233
What an event is	234
Recording an event	235
A calendar of your own devising	237
Seeing the timeline	241
The timeline critique	242
How the timeline feeds the other intelligences	244
The CLI and the Bund word	245
Part V — The Intelligences	248
17 — Continuity and Knowledge	249
18 — The Read-Through and the Voices	250
19 — Revision and History	251
20 — The Inner Family	252
Part VI — Language, Verse & Scholarship	253
21 — Constructed Languages	254
22 — Poetry	255
23 — Research and Scholarship	256
Part VII — Producing the Book	257
24 — Assembly and PDF	258
25 — EPUB, Web and More	259
26 — Technical Documentation	260
Part VIII — Scripting with Bund	261
27 — Scripting with Bund	262
Part IX — Keeping It Healthy	263
28 — Keeping It Healthy	264
29 — Configuration	265
Appendix A — The Keybinding Map	266
Appendix B — The Command Line	267
Appendix C — The Configuration Reference	268
Appendix D — The Feature Index	269
About the Author	270
Why Inkhaven exists	271

A work of love 271

A note on cooperation 272

Where to find more 272

About This Manual

Most writing tools ask you to leave the page. You open a browser to research, a second app to track your characters, a third to check continuity, a spreadsheet to count words, a design program to typeset. Inkhaven's premise is the opposite: **the whole of writing a book lives in one terminal window** — the prose, the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF — and you never leave the keyboard to reach any of it.

This manual is the complete tour of that window. It assumes **no prior knowledge** — not of Inkhaven, not of the terminal beyond opening one — and it runs in reading order from installing the binary to publishing a book. Read it front to back once for the shape of the thing; return to any chapter when you reach the workflow it covers.

What Inkhaven is

Inkhaven is a single command-line program for writing books and long-form documentation. It pairs a full-screen editor for Typst — a modern typesetting language — with a local semantic index, an AI writing assistant you fully control, versioned snapshots, a backup pipeline, and an unusually deep set of **reading** intelligences that watch a manuscript for the things a long book gets wrong. Your prose lives as plain `.typ` files on disk; a local database tracks the hierarchy and the search index, but the words are always text you can read, diff, and version-control yourself.

TERM The manuscript is plain files

A paragraph is a `.typ` file on disk, inside a tree of folders for books, chapters, and subchapters. Inkhaven manages that tree and indexes it, but it never locks your words inside a proprietary format. You can open any paragraph in any editor, at any time, and Inkhaven will notice.

How this manual is organised

The book is in nine parts and a set of reference appendices:

● ● ● *The shape of the manual*

I	Getting Started	install, first project, the panes
II	Writing	the editor, the tree, snapshots, search
III	The AI Assistant	scopes, chat-with-your-book, prompts, cost
IV	The World & Facts	places, characters, the world, graph, timeline
V	The Intelligences	continuity, knowledge, read-through, voices,
		revision, the inner readers
VI	Language & Verse	conlang, poetry, research, scholarship
VII	Producing the Book ...	PDF, EPUB, web, technical docs
VIII	Scripting	the embedded Bund language
IX	Keeping It Healthy ...	backup, doctor, configuration
A–D	Reference	keys, commands, config, the feature index

Parts I through III are the ground floor — everything you need to write and get help. Parts IV and V are Inkhaven’s distinctive depth: the machinery for a book with a world, a cast, and secrets that must stay kept. Parts VI through IX are the specialist tracks and the plumbing.

This manual, and the companions

Inkhaven ships a small library of **companion books**, each a deep dive on one domain — **Know Your Book** on the reading intelligences, **Building the World** on worldbuilding, **Poetry with Inkhaven**, **Grounding Your Book in Fact** on research, and others. This manual is deliberately **breadth-first**: it tells you what

each feature is, how to reach it, and how to run it, then points you to the companion when you want the full treatment.

FICTION

If you write **fiction**, the heart of the book is Parts IV and V — a world and cast Inkhaven can hold in mind, and the readers that watch continuity, knowledge, voice, and pacing across a whole manuscript.

NON-FICTION

If you write **non-fiction**, your centre of gravity is the facts and the graph (Part IV), research and sources (Part VI), and the technical-doc and index tools (Part VII) — the machinery of claims that must be right.

A word on the pictures

Inkhaven is a terminal application, so this manual illustrates it with **terminal frames** rather than photographs — a faithful rendering of what you will see on screen, set in a monospace type. A frame is truer than a screenshot for a text interface, and it keeps the book self-contained. When you see one, read it as “this is what the screen shows here.”

WHAT YOU LEARNED

- Inkhaven is one terminal program for the **whole** of writing a book — prose, world, facts, an AI you steer, and the finished document.
- Your manuscript is always **plain .typ files** on disk; nothing is locked in a proprietary format.
- This manual is the **breadth-first tour**, install to publish; the companion books hold the deep dives.

PART I

Getting Started

CHAPTER 1

1

Installing Inkhaven

Inkhaven is a single program. When it is installed you have one command, `inkhaven`, and everything else in this book happens inside it or through it — the editor, the world, the facts, the assistant, the finished PDF. There is no server to run, no account to create, no companion app to keep open. This chapter gets that one command onto your machine, explains what happens the first time you run it, and tells you how to check that all is well before you write a word.

It assumes you can open a terminal and type into it, and nothing more. If you have never compiled a Rust program, that is fine — you will not have to understand Rust, only to install its toolchain and let it work for a few minutes. Read this chapter

once, top to bottom; you will not need it again until you set Inkhaven up on a second machine.

What you are installing

Inkhaven ships as one self-contained binary. It carries its own database engine, its own semantic-search index, and — after the first run — its own local embedding model. It does **not** depend on any external program being present on your system: no Node, no Python, no Docker, no database server, no paid AI account. The one genuinely optional companion is the Typst compiler, and only if you want to render your manuscript to PDF from inside the editor; everything else, including the plain-text and single-`.typ` exports, works with Inkhaven alone.

TERM Binary

A single executable file — here, one called `inkhaven`. “Installing” it means getting that file onto your machine and onto your `PATH` (the list of directories your shell searches for commands), so that typing `inkhaven` anywhere runs it. Everything Inkhaven does lives in that one file plus the per-user model cache it downloads once.

There are two ways to get the binary: let Cargo **compile** it for you (from `crates.io` or from source), or download a **prebuilt** one. Compiling is the path this chapter treats as canonical, because it always produces a binary matched to your exact machine and needs nothing but a Rust toolchain. The prebuilt paths are faster and are covered too.

Prerequisites

You need three things: a Rust toolchain, a terminal, and a little disk space. Only the first requires any action.

A Rust toolchain

Inkhaven is written in Rust, edition 2024, and requires a compiler of at least **version 1.85**. That minimum is not arbitrary — the 2024 edition and several language features Inkhaven relies on landed in that release. Anything newer is fine; Rust is strongly backward-compatible.

The official way to install Rust is

TERM **rustup**

the standard Rust toolchain installer — a small script that places **rustc** (the compiler) and **cargo** (Rust’s build tool and package manager) under your home directory and adds them to your shell’s **PATH**. It needs no administrator rights and installs entirely inside your own account.

. On macOS and Linux one line does it:

● ● ● *Installing the Rust toolchain via rustup*

```
$ curl --proto '=https' --tlsv1.2 -sSf \
  https://sh.rustup.rs | sh
```

Accept the default when it prompts (option 1 is correct). When it finishes, either open a fresh terminal or reload your shell so that **cargo** is on the **PATH**:

● ● ● *Making cargo available in the current shell*

```
$ source "$HOME/.cargo/env"
```

Then confirm both tools are present and new enough:

● ● ● *Verifying the toolchain*

```
$ rustc --version
rustc 1.85.0 (a028ae42f 2026-01-09)
$ cargo --version
cargo 1.85.0 (d73d2caf9 2026-01-06)
```

If either prints “command not found”, your shell has not yet picked up **\$HOME/.cargo/bin** — close and reopen the terminal, or add that directory to your **PATH** by hand.

NOTE

The Rust toolchain is a one-time, whole-system install of roughly 600 MB. It is not part of Inkhaven and is reused by every Rust program you ever build. You install it once and forget it exists.

A terminal

Inkhaven is a terminal application. It runs in whatever terminal emulator you already have — the built-in Terminal on macOS, or any of the common Linux ones (GNOME Terminal, Konsole, Alacritty, kitty, WezTerm, xterm). It also runs perfectly over SSH, inside `tmux`, and on a tiling window manager, because it is text the whole way down and never reaches for a browser or a graphical window. Any terminal from the last decade with 256-colour support will do; the richer ones (kitty, iTerm2, WezTerm) additionally give you inline image previews, but those are a luxury, not a requirement.

Disk space

A fresh, complete install occupies a few hundred megabytes, most of it the one-time Rust toolchain. The parts that are actually Inkhaven are modest:

● ● ● *Disk footprint of a fresh install*

Rust toolchain	~600 MB	one-time, shared by all Rust
Inkhaven binary	~90 MB	the compiled `inkhaven`
Embedding model	~120 MB	downloaded once, per-user
cache		
A new empty project .	~5 MB	database + index scaffolding

The embedding model is a single shared download, not a per-project cost: a hundred projects reuse the same cached model. A project's own footprint grows only with your prose and its snapshots.

Supported platforms

Inkhaven is released and supported on two platforms:

● ● ● *Supported release targets*

Linux	x86_64-unknown-linux-gnu	
macOS	aarch64-apple-darwin	(Apple Silicon)

These are the targets the release pipeline builds, tests, and ships prebuilt binaries for. Compiling from source with `cargo` works on any host Rust and the dependencies support — an Intel Mac, for instance, compiles cleanly even though there is no prebuilt asset for it — but the two above are the tested, first-class combination.

Why not Windows

Windows is **deferred**: there is no released Windows binary, and you should not expect one yet. The reason is specific and worth stating plainly, because it is not a matter of effort or interest.

Inkhaven computes its embeddings locally through `fastembed`, which in turn runs models through the ONNX Runtime by way of the `ort` crate. `ort` ships prebuilt runtime binaries for the Windows **MSVC** toolchain but not for the Windows **GNU** toolchain, and Inkhaven's build has historically targeted the GNU toolchain (alongside the DuckDB and font-handling C++ that MSYS2 builds cleanly). The build therefore fails at the ONNX Runtime bindings — upstream, in a dependency, not in Inkhaven's own code.

WHY IT IS DEFERRED, NOT ABANDONED

The fix is real and known; it is simply not on the critical path. Any one of three routes unblocks Windows: `ort` shipping a Windows-GNU prebuilt (the CI job is kept ready and would simply go green), adopting the Windows-MSVC target so that `ort`'s existing prebuilts apply, or bundling a build-time-verified runtime library in the release archive. The one route deliberately **rejected** is fetching and loading a runtime library at startup — that would run unsandboxed native code pulled over the network, a far larger attack surface than the model data Inkhaven does download, and against the project's no-external-binary principle. Until one of the sound routes is taken, Windows users should build under WSL (a Linux environment) and treat it as the Linux target.

Installing with Cargo

If you have the Rust toolchain, the shortest path to a working `inkhaven` is to let Cargo compile the published crate.

cargo install inkhaven — compile from crates.io

Inkhaven is published on crates.io; every release tag pushes a new version. This one command downloads the crate and its dependencies and compiles the lot:

● ● ● *Installing the published crate*

```
$ cargo install inkhaven
```

Be patient the first time. The build pulls roughly a hundred crates and then compiles the heavy ones — DuckDB, `fastembed`, and the ONNX Runtime bindings are large C and C++ projects — so a first build takes on the order of **ten minutes** on a modern laptop. This is a one-off: Cargo caches the compiled dependencies, and you will not pay that cost again.

When it finishes, the binary lands in Cargo’s binary directory, which is already on your `PATH` if rustup set your shell up:

● ● ● *Where cargo install puts the binary*

```
~/.cargo/bin/inkhaven
```

That is the whole install. Skip ahead to “The first run” once `inkhaven --version` prints a version.

cargo binstall — the prebuilt fast path

If you would rather not wait for a compile and you have

TERM cargo-bininstall

a small Cargo extension that downloads a **prebuilt** binary from a project's GitHub releases instead of compiling it. Install it once with `cargo install cargo-bininstall`; thereafter `cargo bininstall <crate>` fetches the right prebuilt asset for your platform.

installed, one command fetches the prebuilt binary and drops it into `~/.cargo/bin`:

● ● ● *Installing a prebuilt binary*

```
$ cargo bininstall inkhaven
```

`cargo-bininstall` reads the packaging metadata from Inkhaven's `Cargo.toml`, picks the asset matching your platform off the GitHub releases, and installs it without compiling anything. This is the quickest route on the two supported platforms.

GitHub Releases — a direct download

You can also bypass Cargo entirely. Each release publishes a tarball per platform on the project's Releases page. Download the one for your platform, unpack it, and place the `inkhaven` binary anywhere on your `PATH`:

● ● ● *Installing from a release tarball*

```
$ tar xzf inkhaven-<version>-<platform>.tar.gz
$ sudo install inkhaven /usr/local/bin/inkhaven
```

The releases live at github.com/vulogov/blackInkhaven/releases the asset names encode the version and the target triple so you can pick the right one at a glance.

cargo install --git — a specific tag or branch

When you want a particular tagged version, a pre-release branch, or your own fork, install straight from the git repository and name the tag:

● ● ● Installing a specific tagged version

```
$ cargo install \  
  --git https://github.com/vulogov/blackInkhaven \  
  --tag v3.0.0
```

This compiles exactly the named revision. Drop `--tag` to build the default branch's current tip. Check the Releases page for the tag you actually want; the examples here name `v3.0.0`, the stable edition this manual documents.

Building from source

Compiling from a clone is the path to choose if you want to read or modify the code, track a development branch, or simply keep the whole thing under your own eye. It needs only the Rust toolchain and `git`.

First, clone the repository and enter it:

● ● ● Cloning the source

```
$ git clone \  
  https://github.com/vulogov/blackInkhaven.git  
$ cd blackInkhaven
```

If you have no `git`, the GitHub page offers the same source as a zip; unpack it and `cd` into the folder. Then build an optimised binary:

● ● ● Building the release binary

```
$ cargo build --release
```

The first build downloads about a hundred crates and compiles them; expect several minutes — commonly three to eight on a modern laptop, longer on older hardware because of the same heavy C/C++ dependencies noted above. Later builds are

incremental and reuse the cache, so they finish in seconds. When it is done, the binary is here:

● ● ● *Where the source build puts the binary*

```
./target/release/inkhaven
```

You can run it straight from that path. To type just `inkhaven` from anywhere, copy it onto your `PATH`:

● ● ● *Putting inkhaven on your PATH*

```
# system-wide (asks for your password)
$ sudo install target/release/inkhaven \
    /usr/local/bin/inkhaven

# or, no sudo, if ~/.local/bin is on PATH
$ cp target/release/inkhaven ~/.local/bin/
```

WHY THE `--release` FLAG MATTERS

Without `--release`, Cargo builds an unoptimised debug binary. It runs, but it is several times slower and a few hundred megabytes larger — embedding and search in particular feel sluggish. Always build `--release` for real writing; reserve the debug build for hacking on Inkhaven’s own code.

Optional: the Typst compiler

Inkhaven writes and manages Typst, but it does not bundle the Typst compiler. If you want `inkhaven export pdf` (and the in-editor “build to PDF” chords) to actually produce a PDF, install Typst separately — it, too, is a single static binary on every platform, available from its own project page. Everything else, including the `export typst` path that emits one combined `.typ` file, works without it.

The first run

The install is inert until the first time Inkhaven opens a project. That first open does two one-time things worth understanding, so that a slightly longer startup does not alarm you.

The embedding model download

Inkhaven’s semantic search — finding “the moment the lighthouse fails” even in a paragraph that never says **lighthouse** — is powered by a local embedding model. That model is not shipped inside the binary; it is downloaded the first time you initialise or open a project. The default is

TERM **MultilingualE5Small**

the default embedding model — a compact, multilingual model (English, Russian, German, French, Spanish, and more) about 120 MB in size. It is chosen as a sensible balance of quality against footprint; the configuration lets you switch to the Base or Large variant, at the cost of a larger download and slower embedding.

, roughly 120 MB, fetched once into a per-user cache and reused by every project thereafter.

The cache location depends on your platform:

● ● ● *Where the embedding model is cached*

```
macOS ~/Library/Caches/
      dev.inkhaven.inkhaven/embeddings/
Linux  $XDG_CACHE_HOME/inkhaven/embeddings/
      (defaults to ~/.cache/inkhaven/)
```

On a normal connection the download completes in well under two minutes, behind a splash screen that shows the elapsed time. It happens once per model: switching the configured model later downloads the new one on the next open and leaves the old one on disk until you remove the cache directory yourself. If you

ever change `embeddings.model` in a project's configuration, Inkhaven re-embeds every paragraph against the new model the next time it opens.

Model init and the project lock

After the model is present, Inkhaven initialises it into memory and takes an **advisory** lock on the project so that two sessions do not write the same database at once. The lock is a small file, `.inkhaven.lock`, in the project root, held for as long as the session runs and released automatically by the operating system when the process exits — even on a crash or a `kill -9`, so there is never a stale lock to clean up by hand.

THE LOCK INFORMS, IT NEVER BLOCKS

In keeping with Inkhaven's permissive character, opening a project that another session already holds does not fail. The launcher tells you who holds it — a process id, host, and time — and lets you open it anyway. Only genuine data-safety is at stake, and the choice stays yours. This is the same principle you will meet throughout Inkhaven: it warns, it does not forbid.

Once the model is cached and warm, subsequent launches are quick — there is no download and the lock is instantaneous.

Verifying the install

Two commands confirm a healthy install without touching any project. First, the version:

● ● ● *Confirming the binary runs*

```
$ inkhaven --version
inkhaven 3.0.0
```

A version number means the binary is on your `PATH` and runs. If instead you see “command not found”, the binary is not on your `PATH` — invoke it by its full path (`./target/release/inkhaven` from a source build, or `~/cargo/bin/inkhaven` from a Cargo install), or copy it to a directory that is on your `PATH`.

Second, the help surface, which lists every top-level subcommand and the global flags:

● ● ● *Listing the command surface*

```
$ inkhaven --help
TUI literary work editor for Typst books

Usage: inkhaven [OPTIONS] [COMMAND]

Commands:
  init      Initialize a new project
  add       Add a node to the hierarchy
  list      Print the hierarchy as a tree
  search    Run a semantic search
  export    Export the book(s)
  backup    Zip the whole project
  ...

Options:
  -p, --project <PROJECT>  Path to a project root
  -h, --help                Print help
  -V, --version             Print version
```

Any subcommand takes `--help` of its own — `inkhaven export --help`, `inkhaven init --help` — which prints that command’s full set of flags. This is the authoritative reference for the exact surface of the version you have installed; when this book and the binary ever disagree, the binary’s `--help` is right.

Troubleshooting the install

Most installs are uneventful. The handful of things that can go wrong have short answers.

command not found: inkhaven

Your shell cannot find the binary on its `PATH`. From a source build it lives at `./target/release/inkhaven`; from `cargo install` at `~/.cargo/bin/inkhaven`. Either call it by that full path, copy it into a directory that is on your `PATH` (`/usr/local/bin` or `~/.local/bin`), or alias it. Confirm

your `PATH` contains `$HOME/.cargo/bin` if you expected Cargo’s install to be found automatically.

The first-run download is slow or stalls

The model download is the one part of startup that reaches the network. If the splash screen’s elapsed counter climbs past a couple of minutes on a connection you know is working, something upstream is slow rather than broken. You can:

● ● ● *Recovering from a stalled first-run download*

- Check the machine actually has connectivity.
- Watch the splash's elapsed timer – under ~2 min is normal for MultilingualE5Small.
- Press Ctrl+Q to abort startup, then retry later.

Aborting is safe: nothing is half-written into the cache in a way that a later retry cannot replace.

Offline, air-gapped, or behind a proxy

Inkhaven never needs the network **except** for that one first-run model download (and, separately, for any external AI provider you choose to configure — which is entirely optional). Two consequences follow.

On a **proxy**, make sure the standard `HTTPS_PROXY` environment variable is set before the first run so the download can reach out. On a genuinely **air-gapped** machine, you cannot download at all — so seed the cache from a networked machine instead: install and run Inkhaven once on a connected computer, then copy the populated cache directory (the platform paths listed under “The first run”) onto the air-gapped machine before its first launch. With the model already present, Inkhaven starts fully offline and stays offline for the whole of writing, worldbuilding, search, and snapshots — every subsystem but the external providers is on-device.

A configuration error on open

If opening a project reports a missing configuration field, you are opening a project whose `inkhaven.hjson` was written by an older release than the binary you just installed. Either add the missing field by hand, or regenerate a fresh configuration from the current template with `inkhaven init --force` (back the old file up first). New projects created by the version you installed never hit this.

The CLI and the TUI

One binary, two shapes. Understanding the split now will save confusion in every later chapter, because this manual moves freely between them.

Run `inkhaven` with **no subcommand** — inside a project directory, or with a `--project` path — and it launches the full-screen editor: the

TERM TUI

the Text User Interface — Inkhaven’s full-screen, keyboard-driven editor, with its tree, editor, search, AI, and output panes. It takes over the terminal, and you leave it with `Ctrl+Q`. This is where you actually write.

, the interactive program that fills the terminal and is the subject of most of this book:

● ● ● Opening the editor

```
$ cd ~/Books/my-novel
$ inkhaven                # opens the TUI here
# ...or from anywhere:
$ inkhaven --project ~/Books/my-novel
```

Run `inkhaven` **with** a subcommand and it does one job and exits, printing to the terminal without ever entering full-screen mode. These headless commands are what you script, pipe, and run in continuous integration:

● ● ● Headless, one-shot commands

```
$ inkhaven init ~/Books/my-novel
$ inkhaven --project ~/Books/my-novel list
$ inkhaven --project ~/Books/my-novel \
    search "the lighthouse fails"
$ inkhaven --project ~/Books/my-novel \
    export pdf --status ready
```

The global `--project` flag (also `-p`, or the long alias `--project-directory`) tells any command which project to act on, and defaults to the current directory — so `cd` into a project first and you can drop it entirely. The top-level command families you will meet across the book group roughly like this:

init · add · mv	Create and shape the hierarchy
list · outline	Inspect the tree from the shell
search · book-rag	Semantic search and retrieval
export · index	Produce PDF, Typst, EPUB, indexes
backup · restore	Project-level disaster recovery
reindex · doctor	Reconcile store with disk; health
ai	One-shot inference from the shell

A few subcommands — the standalone configuration editor, the research and worldbuilder and linguistic workspaces — are themselves full-screen TUIs rather than one-shot commands; the book flags each where it introduces it. Everything else you type after `inkhaven` runs, prints, and returns you to your prompt.

FICTION

If you write **fiction**, your usual entry is the bare `inkhaven` — you live in the editor, with the tree, the world, and the readers a keystroke away. The headless commands matter mainly at the edges: `init` to begin, `backup` to be safe, `export` to ship.

NON-FICTION

If you write **non-fiction** or documentation, you will lean on the headless side far more — `search`, `export`, `index`, and the check commands slot naturally into build scripts and continuous integration, where an editor cannot go.

With `inkhaven` installed, verified, and its two shapes clear, you are ready to create a project — which is where the next chapter begins.

WHAT YOU LEARNED

- Inkhaven is **one self-contained binary**; the only prerequisite you must install yourself is a **Rust toolchain of version 1.85 or newer** via rustup.
- Install it by compiling — `cargo install inkhaven` (a 10-minute first build) or `cargo build --release` from a clone — or grab a prebuilt binary with `cargo binstall` or from GitHub Releases.
- The supported targets are **Linux x86_64** and **macOS Apple Silicon**; Windows is deferred because the ONNX Runtime bindings `fastembed` needs have no Windows-GNU prebuilt.
- The **first run** downloads a 120 MB embedding model once into a per-user cache, then takes an advisory project lock that **informs but never blocks**.
- Verify with `inkhaven --version` and `inkhaven --help`; running `inkhaven` bare opens the **TUI**, while `inkhaven <subcommand>` runs **headless** and exits.

CHAPTER 2

2

Your First Project

A book, to Inkhaven, is a **directory**. Not a file with an opaque extension, not a row in some cloud database you cannot see — a plain folder on your own disk, with your prose in readable text files and a small local database beside them that remembers the shape of the thing. This chapter creates one, opens it up, and names every part, so that when a later chapter says “the Facts book” or “the `.inkhaven/` sidecars” or “a structural leaf” you already know exactly where on disk it lives and what it is for.

We will run one command, read what it laid down, meet the twenty **system books** Inkhaven seeds into every project, and then make a book, a chapter, and a

paragraph of your own. By the end you will have a project you can open, close, back up, and put under version control — and a mental map of every file in it.

The one command that starts everything

Everything begins with `inkhaven init`. It takes a single positional argument — the directory the project should live in — and creates it if it does not yet exist. The convention is one project per book (or per tightly-related set), kept somewhere like `~/Books/`.

● ● ● *Creating a project*

```
$ inkhaven init ~/Books/my-first-project
Initialized inkhaven project at /home/you/Books/my-first-
project
config:    .../my-first-project/inkhaven.hjson
prompts:   .../my-first-project/prompts.hjson
store db:  .../my-first-project/metadata.db
vecstore:  .../my-first-project/vectors
books:     .../my-first-project/books
```

The directory need not exist beforehand — Inkhaven makes it. If it **does** exist, Inkhaven refuses to touch it silently, because `init` starts from a clean database and re-initialising means **wiping** what is there. It asks first:

● ● ● *The overwrite guard*

```
$ inkhaven init ~/Books/my-first-project
Directory `/home/you/Books/my-first-project` already exists.
Remove it and re-initialise? [y/N]
```

Only `y` or `yes` proceeds; a bare Enter, an `n`, or end-of-input all abort with your files left untouched. There is one further safety interlock: Inkhaven will **refuse** to wipe a directory that your current shell is sitting inside, because deleting the ground beneath your own feet leaves the terminal in a broken state. `cd` out first.

The two flags

`init` has exactly two options beyond the path.

- force** Skip the [y/N] prompt and overwrite an existing directory unattended. For scripts and fresh-checkout automation — never needed interactively.
- template** Scaffold a starting tree instead of an empty one. Defaults to empty.
- <name>**

The templates are pre-built manuscript skeletons applied **after** the standard `init`, so a template never changes what `init` itself lays down — it only adds books and chapters on top. Run `inkhaven template list` to see them all; the built-in set is:

```
● ● ● inkhaven init --template <name>
```

empty	the default — system books only, no manuscript
novel	three-act manuscript + Characters stubs
nonfiction	intro / parts / conclusion + Research method
rpg-sourcebook	Setting/Rules/Adventures + Places/Artefacts/
Threads	
technical	Overview / Reference / Tutorials / Index
nanowrimo	like novel, with a 50 000-word goal + pacing

If you are unsure, take `empty`. You can build the same tree by hand in the TUI in a minute, and this chapter shows you how. A template is a convenience, never a commitment.

FIRST RUN DOWNLOADS A MODEL

The very first `init` on a machine fetches a multilingual embedding model (about 120 MB) into a **per-user** cache — not into the project. On macOS that is `~/Library/Caches/dev.inkhaven.inkhaven/embeddings/`; on Linux `~/.cache/inkhaven/embeddings/`; on Windows under `%LOCALAPPDATA%`. Every later `init` reuses it, so only the first project pays the download. This is the model that powers semantic search and the reading intelligences — all of it runs on your own machine, offline.

What init writes to disk

`cd` into the new project and list it. Ignoring the model cache (which lives elsewhere), a fresh project is small and entirely legible:

● ● ● *A fresh project, top level*

```
my-first-project/
├─ inkhaven.hjson      the configuration file
├─ prompts.hjson       your prompt library
├─ metadata.db         hierarchy + node metadata (DuckDB)
├─ blobs.db            paragraph bodies, snapshots, blobs
├─ frequency.db        the full-text search index
├─ vectors/            the HNSW semantic-search index
└─ books/              your prose – one folder per book
```

Two of these you will edit by hand; the rest Inkhaven owns. The two yours are `inkhaven.hjson` (theme, language, AI provider, autosave cadence – Chapter and appendix on configuration cover every field) and `prompts.hjson` (your library of reusable AI prompts). The four database entries – `metadata.db`, `blobs.db`, `frequency.db`, and the `vectors/` directory – are the local store, created and maintained by the embedded database engine. You never open them with a text editor.

TERM The store is three databases and an index

Inkhaven’s metadata lives in DuckDB files beside your prose. `metadata.db` holds the **hierarchy** – every book, chapter, and paragraph as a node with a stable UUID, its title, order, and file path. `blobs.db` holds paragraph bodies and versioned snapshots. `frequency.db` is the full-text index behind keyword search. `vectors/` is the HNSW index that makes **semantic** search and the reading intelligences possible. Your words themselves, though, are never trapped in there – they are the `.typ` files under `books/`, and the database merely points at them.

This split is the heart of Inkhaven’s data model, and worth dwelling on for a moment. The **prose is the filesystem**. The **database is an index over the**

filesystem. If the two ever drift apart — you edited a paragraph in another editor, or a crash left them out of step — `inkhaven reindex` re-reads the `.typ` tree and rebuilds the database from it. The files are the source of truth; the database is derived, and therefore always rebuildable. That is why your manuscript is safe even if you never trust the database at all: it is ordinary text, in ordinary folders, that you can read, `grep`, diff, and commit to Git without Inkhaven’s help.

Where session, backups, and caches live

Beyond the store, Inkhaven keeps a handful of **dotfiles** and one **dot-directory** in the project root. None of these exist right after `init` — they appear as you use the project — but knowing them now saves confusion later.

● ● ● *The runtime sidecars (appear with use)*

```
my-first-project/
├─ .session.json      cursor + which paragraph was open
├─ .inkhaven-backup.json  timestamp of the last backup
├─ .inkhaven.log       the runtime log (append-only)
├─ .inkhaven/          advisory sidecars + local caches
│   ├─ drift.json      continuity-drift findings
│   ├─ facts_scan.json fact-check results
│   ├─ tensions.json   pacing / tension curve
│   ├─ continuity.json the continuity bible cache
│   ├─ submissions.json submission-package tracker
│   ├─ ai_usage.json    rolling AI cost ledger
│   ├─ prose.duckdb     per-chapter prose metrics
│   ├─ voices/          character-voice profiles
│   └─ digest-<slug>.json per-book cached digests
```

The three top-level dotfiles are small and transient. `.session.json` remembers which paragraph you had open and where the cursor sat, so re-opening the project puts you back exactly where you left off. `.inkhaven-backup.json` is a single timestamp the auto-backup logic checks against your configured `backup.max_age`. `.inkhaven.log` is the append-only log — the first place to look when something misbehaves.

The `.inkhaven/` directory is a growing collection of **advisory sidecars**: the cached output of scans and the reading intelligences. Every file in it is disposable. Each records what some analysis last found — drift, fact-checks, tension curves, voice profiles — so that opening a view is instant instead of re-computing from

scratch. Delete any of them and Inkhaven simply recomputes on next use. Crucially, **nothing here is your prose**: these are derived opinions about your prose, kept to one side. Chapters throughout Parts IV and V describe each sidecar where its feature is covered; for now, treat `.inkhaven/` as “the scratchpad the intelligences share.”

FOR VERSION CONTROL

Commit `inkhaven.hjson`, `prompts.hjson`, and `books/` — the prose and its configuration. You may commit the `.db` files too (they are your index and snapshots), but they are large and rebuildable, so many authors add `*.db`, `vectors/`, `.inkhaven/`, `.session.json`, and `.inkhaven.log` to `.gitignore` and let `inkhaven` `reindex` rebuild the store on a fresh clone.

The anatomy of the manuscript tree

Open `books/` and you find the real structure. Inkhaven’s hierarchy — **Book** → **Chapter** → **Subchapter** → **Paragraph** — is mirrored one-to-one onto the filesystem: branches are folders, leaves are files, and every entry carries a zero-padded numeric prefix so a plain directory listing sorts in reading order.

● ● ● *One book on disk*

```
books/
├─ my-first-book/
│   ├─ 01-chapter-one/
│   │   ├─ 01-the-storm.typ
│   │   ├─ 02-landfall.typ
│   │   └─ 02-a-quiet-town/      ← a subchapter
│   │       ├─ 01-the-inn.typ
│   │       └─ 02-the-innkeeper.typ
│   └─ 02-chapter-two/
│       └─ 01-morning.typ
```

Read that tree carefully, because its rules are the whole model:

- A **book** is a folder named by its bare slug (`my-first-book`) — no number, because books are ordered among themselves in the database, and their folders sit at the top level of `books/`.
- A **chapter** and a **subchapter** are numbered folders (`01-chapter-one`, `02-a-quiet-town`). A subchapter is simply a chapter-level folder that lives inside a chapter — the same kind of container, one level down.
- A **paragraph** is a numbered `.typ` file (`01-the-storm.typ`). This is the leaf: the actual unit of prose you edit. Despite the name, a “paragraph” can hold as much text as you like — think of it as a titled **section** of the manuscript, the smallest thing Inkhaven versions, embeds, and re-orders.

The numeric prefixes are Inkhaven’s bookkeeping, not yours to hand-edit. When you reorder rows in the tree, Inkhaven rennumbers the files on disk to match; the slug (the human-readable part after the number) comes from the title, and a paragraph with an empty title gets its slug — and title — from the first sentence you type on first save.

TERM The manuscript is a Typst project in disguise

Each paragraph file is a fragment of Typst, the typesetting language. A new paragraph opens with a single `=` heading line — Typst’s level-one heading marker — so the paragraph renders as its own section in the final book. When you export, Inkhaven walks the tree in order, assembles the fragments into one Typst document, and hands it to the typesetter. Nothing about the on-disk format is proprietary: it is Typst, all the way down.

The node model at a glance

Every row in the tree is a **node**, and every node has a **kind**. Four kinds are the structural spine you have already met — Book, Chapter, Subchapter, Paragraph. Alongside the prose paragraph, though, Inkhaven allows several **non-prose leaves**: siblings in the same tree that carry something other than Typst prose. Each has its own on-disk extension and its own chapter later in this manual; here they are only named, so you recognise them in a tree.

● ● ● Leaf kinds at a glance

KIND	ON DISK	WHAT IT HOLDS
paragraph	NN-slug.typ	prose (Typst) — the default leaf
image	NN-slug.png	a picture: png / jpg / webp / svg
hjson ¶ source...)	NN-slug.hjson	structured data (a place, a
jinja ? export	NN-slug.jinja	a template rendered to Typst on
bund ⚙	NN-slug.bund	an embedded Bund script

A few notes to fix the picture:

- An **image** is a first-class node, not a paragraph. Drop a `.png` / `.jpg` / `.webp` / `.svg` into the tree and Inkhaven ships the bytes into the assembled book with the right image call; a caption and alt-text ride along in the metadata.
- An **hjson** leaf is a paragraph whose content type is data rather than prose. This is how the world books store structure — a place, a character, a bibliography entry — as an HJSON record instead of paragraphs of text.
- A **jinja** leaf is a template: it holds Jinja markup that Inkhaven renders down to Typst at assembly time, for repeated or generated structure.
- A **bund** leaf is a script in Inkhaven’s embedded Bund language, evaluated into the scripting engine when the project opens — the home of custom hooks and rules. Its default home is the Scripts system book, but it can live anywhere.

There is one more flavour that is **not** a separate file type but a **subtype** of an ordinary Typst paragraph — the **structural paragraph**. In the editor, the structural-subtype picker turns a paragraph into a piece of recognised scaffolding — code block, admonition, mathematics, procedure, or table — each with its own glyph in the tree (▢ ▴ ∫ ≡ ▣). It is still a `.typ` file; the subtype only tells Inkhaven how to treat and render it. A single leaf can be cycled through its types — `typst` → `hjson` → `jinja` → `bund` — as your needs change. Every one of these gets a full treatment in a later chapter; for now, the point is simply that **the tree holds more than prose**, and each non-prose leaf is still a plain, readable file.

The system books

Here is the part that surprises people opening a fresh project: it is not empty. Before you have written a word, `inkhaven list` shows a stack of books already there. These are the **system books** — reference and machinery books Inkhaven seeds into **every** project, on every open, whether you use them or not. They carry stable internal tags, they are marked **protected** (you cannot delete or rename them), and anything you create lands **above** them, at the top of the tree.

HOW MANY, REALLY

Older documentation counted “six” and then “nine” system books; the project has grown since. The authoritative list is the `SYSTEM_BOOKS` table in the source, and as of this edition it seeds **twenty**. If a future release adds one, it appears automatically the next time you open any existing project — the seeder is idempotent and back-fills.

They divide cleanly into three groups: reference books you write into, world books that hold your fiction’s ground truth, and machinery books Inkhaven and its tools own. First, the reference and prose books:

● ● ● *System books · reference & prose*

Notes	free-form scratch prose, kept out of the manuscript
Research	research notes and gathered source material
Sources	bibliography entries (HJSON) → compiled to a <code>.bib</code>
Glossary	canonical terms + banned synonyms (terminology)
Snippets	reusable Typst blocks, <code>#include'd</code> elsewhere
Prompts	your AI prompt library (seeded with examples)
Help	Inkhaven's own manual — F1 answers by RAG here

Then the **world** books — the ground truth a work of fiction must stay consistent with. These are where Parts IV and V of this manual do their work:

● ● ● *System books · the world*

Facts	the world's invariants – the fact-check reference
Places	your gazetteer – locations, named and highlighted
Characters	the cast – names light up in the editor
Artefacts	objects of significance – items, relics, devices
World	simulation output (the realworld compiler writes here)
Threads	narrative plot threads and their arc shapes
Planning	story structure – beats and acts of a framework
Mythology	declared symbols, motifs, and archetypes
Intent	your declared authorial choices (silences findings)

And finally the **machinery** books – homes for the tooling that surrounds the prose:

● ● ● *System books · machinery*

Typst	reusable Typst templates and skeletons
Scripts	.bund scripts auto-loaded into the engine at open
Language	invented-language books (one child book per conlang)
Submissions	the submission package – query letter, synopsis...

A handful of these deserve a sentence of **why** now, because you will meet them again and again:

- **Notes** and **Research** are the free-form catch-alls – prose that supports the book without being in it. The AI can be scoped to read them.
- **Facts** is the reference the continuity and fact-checking tools ground against: the climate, the geography, the chronology your chapters must not contradict. **Grounding Your Book in Fact** is the companion book for it.
- **Places**, **Characters**, and **Artefacts** are **lexicon** books – Inkhaven highlights their names where they appear in your prose, and you can ask the AI about any of them by name.
- **Prompts** ships pre-seeded with `.example` prompts for the built-in AI actions; rename one to drop the `.example` suffix and it overrides the default. **Help** is

Inkhaven’s own documentation, which is why pressing **F1** can answer questions about the program itself.

- **Intent** is where you record “I meant to do that” — a declared choice that tells the reading intelligences to stop flagging something they would otherwise report.

You do not need to touch most of these on day one. They are there so that when a later feature needs a home — a bibliography, a cast list, a plot thread — it already has one, in a known place, tagged the same in every project you will ever open.

Opening the project and making your first book

You have a project on disk. Now open it. Running **inkhaven** with no arguments opens the project in the **current** directory; **--project <path>** opens one elsewhere.

● ● ● *Opening the editor*

```
$ cd ~/Books/my-first-project
$ inkhaven
# — or, from anywhere —
$ inkhaven --project ~/Books/my-first-project
```

The full-screen editor comes up in a multi-pane layout — a search bar across the top, the **Tree** on the left, the **Editor** in the middle, an **AI** pane on the right, and a status line along the bottom. Chapter three walks the panes in detail; here we only need the Tree, which has focus on startup, and where you see the system books stacked and waiting.

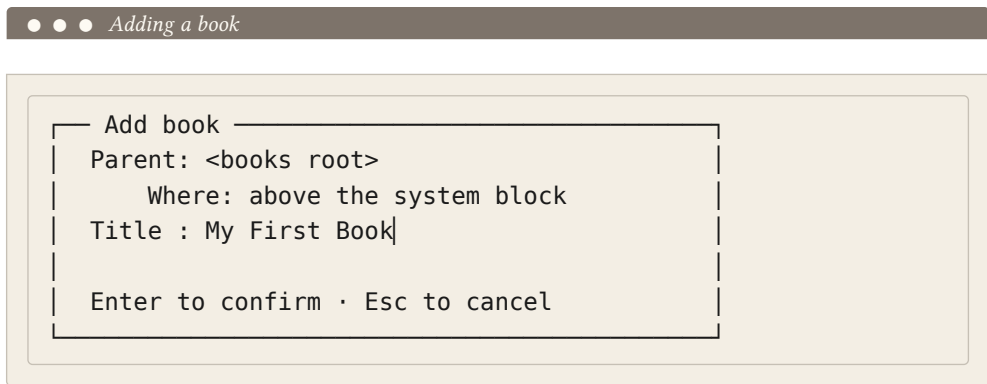
The Tree is driven by single letters. To build the spine of a book you need just three of them — one for each level — plus Enter to open a leaf for editing.

B	add a book at the root (above the system books)
C	append a chapter under the selected book
A	append a subchapter under the selected chapter
+	append a paragraph under the selected branch
Enter	open the selected paragraph in the Editor

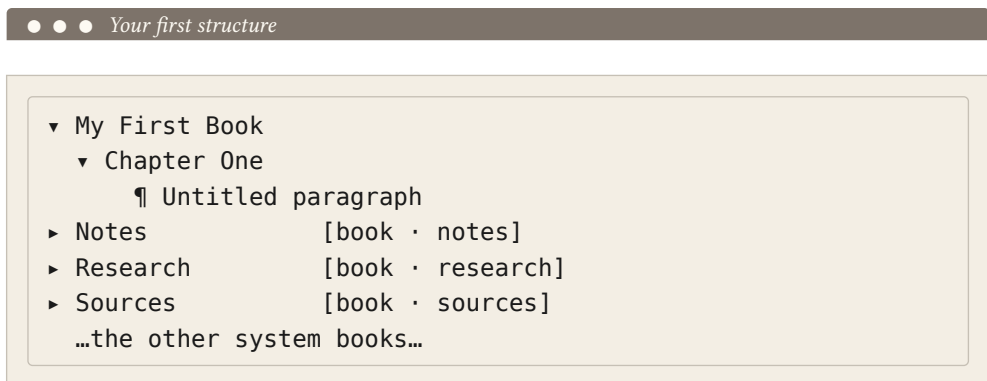
F2 rename the selected node

Each of **C**, **A**, and **+** has an **insert-after** twin — **V**, **S**, and **P** respectively — that drops the new node immediately below the current row rather than at the end of its parent. You will reach for those when a manuscript grows and you need to slot something in the middle; for a first pass, append is all you want.

Press **B**. A small dialog asks for the title; type one and press Enter, and your book appears at the very top of the tree, above **Notes** :



With the cursor on your new book, press **C** to add a chapter beneath it; with the cursor on the chapter, press **+** to add a paragraph. You may leave the paragraph's title blank — Inkhaven will name it from your first sentence when you save. The tree now shows your work sitting above the seeded books:



Put the cursor on the paragraph row and press Enter. Focus moves to the Editor, which shows the paragraph's starting template — a single Typst heading line:

 *A new paragraph in the Editor*

= Untitled paragraph

|

Press **End**, then Enter twice to leave the heading behind, and write. The border around the Editor turns **yellow** the moment the buffer is dirty. Press **Ctrl+S** to save: the **.typ** file is written under **books/**, the metadata is updated, and a fresh embedding is computed for search — and the border returns to **green**. Because you left the title empty, the Tree row now reads back your opening sentence, and the file on disk has been renamed to match its new slug.

That is a complete round trip: a book, a chapter, a paragraph, saved to disk as plain Typst, indexed for search, and ready to reopen exactly where you left it. To leave, press **Ctrl+Q** — Inkhaven autosaves anything dirty and records the session so your next launch reopens this very paragraph.

Two ways to shape the tree first

Before you pour in words, it is worth a minute's thought about **shape**, because the tree you build is the outline you will write against — and a novelist and a non-fiction author reach for it differently.

FICTION

Start with the **spine of the story**, not the prose. Make one book, then a chapter per act or major movement, and let paragraphs be **scenes**. Lean on the world books early: drop your cast into **Characters** and your locations into **Places** as you invent them, so their names highlight in the prose and the continuity readers have something to watch. The novel template lays exactly this down if you would rather not start empty.

NON-FICTION

Start from the **table of contents**. Make a book, a chapter per part or major section, and subchapters for the sections beneath — the tree **is** your outline, and it will become the document's structure verbatim. Put your gathered material and citations into **Research** and **Sources** from the first day, so claims have a home and a bibliography assembles itself. The nonfiction and technical templates scaffold this shape.

Neither choice is binding. The tree reorders freely — rows move up and down, paragraphs move between chapters, whole branches relocate — and every move rennumbers the files on disk for you. Build the shape you can see today; reshape it the moment the book tells you to.

WHAT YOU LEARNED

- `inkhaven init <dir>` creates a project directory; `--force` skips the overwrite prompt and `--template` scaffolds a starting tree, defaulting to **empty**. The first ever init downloads a shared embedding model to a per-user cache, not into the project.
- A project is **plain files**: `inkhaven.hjson` and `prompts.hjson` are yours to edit; `metadata.db`, `blobs.db`, `frequency.db`, and `vectors/` are the local store; `books/` is your prose. The files are the source of truth and the store is a rebuildable index — `inkhaven reindex` restores it from the tree.
- The hierarchy **Book** → **Chapter** → **Subchapter** → **Paragraph** is mirrored onto disk as folders and numbered `.typ` files; alongside prose paragraphs the tree can hold **image**, **hjson**, **jinja**, and **bund** leaves, plus structural-paragraph subtypes.
- Every project is seeded with **twenty protected system books** — reference (Notes, Research, Sources, Glossary, Snippets, Prompts, Help), world (Facts, Places, Characters, Artefacts, World, Threads, Planning, Mythology, Intent), and machinery (Typst, Scripts, Language, Submissions) — that you cannot delete and that your own books sit above.
- In the TUI the Tree is driven by single letters — B book, C chapter, A subchapter, + paragraph, Enter to edit, Ctrl+S to save — and the `.inkhaven/` directory plus the `.session.json` / `.inkhaven-backup.json` / `.inkhaven.log` dotfiles hold disposable session, backup, and cache state.

CHAPTER 3

3

The Four Panes

Everything in the last two chapters happened somewhere on one screen. Now we name the places. Inkhaven fills your terminal edge to edge with a single full-screen window, and the whole of writing a book takes place inside it — you never open a second app, never reach for a mouse you do not need, never lose your place. This chapter is the map of that window: the four regions it is divided into, how you move your attention between them, and the two-key **chord** language that reaches every command the tool has. Learn this chapter and the rest of the manual becomes a matter of looking things up; skip it and every later chapter will feel like a room you entered through the wrong door.

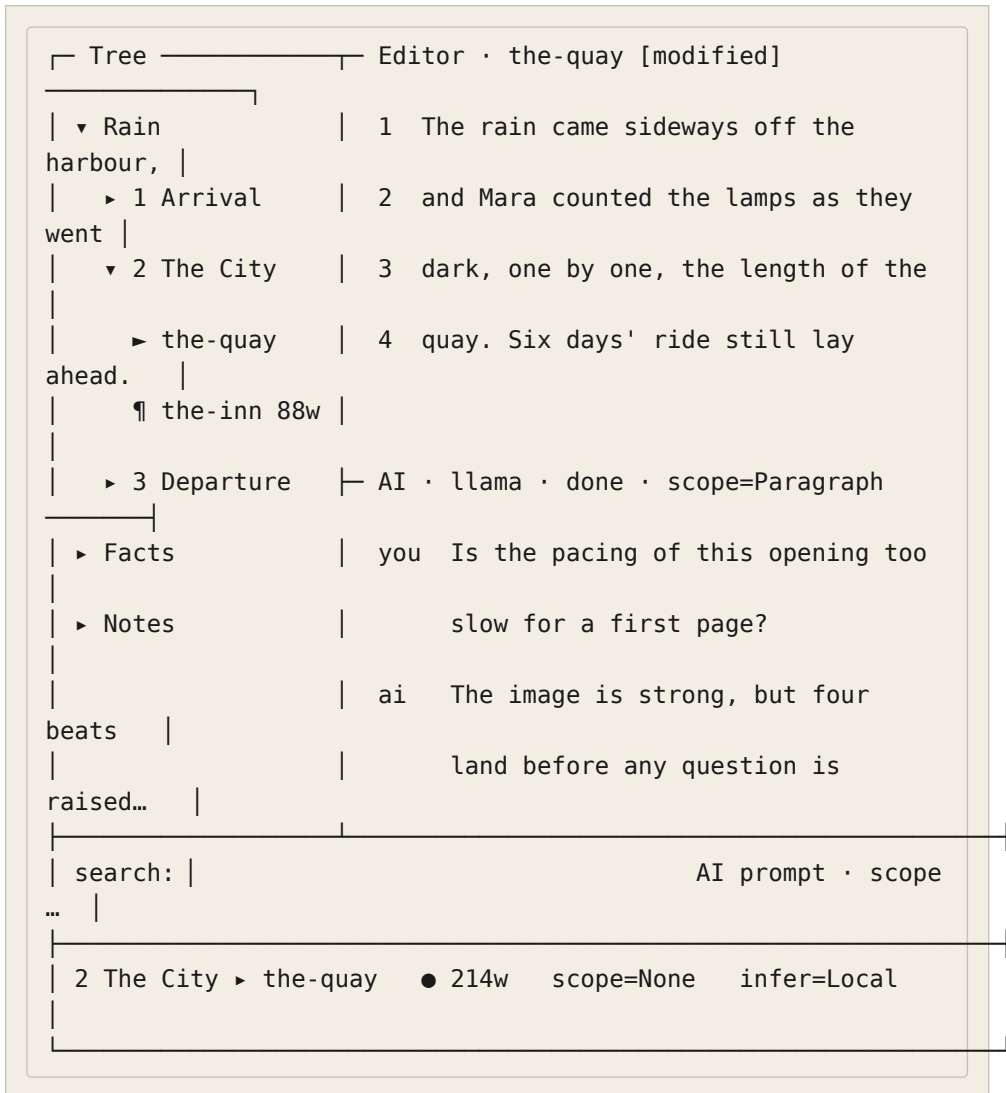
One window, four regions

Open a project and the terminal divides into four working areas. On the left, a narrow **Tree** holds the shape of your book — its parts, chapters, and paragraphs, folded and unfolded like an outline. Filling the centre is the **Editor**, where the open paragraph's words live. To its right sits a region that shows one of three things at a time — most often the **AI** pane, a conversation with your assistant. Across the bottom run two thin input bars — a Search bar and the AI prompt — and beneath them a single **status bar** summing up the moment. That is the whole instrument.

TERM Pane

A **pane** is one of the window's working regions — the Tree, the Editor, the AI pane, and the others. Exactly one pane has **focus** at any moment: the pane your keystrokes are talking to. Everything you type is read as input to the focused pane (or, when a chord is in flight, as a command).

The book you are reading calls this “the four panes” for the shape you meet first — Tree, Editor, AI, and the region beside the Editor — but the window is really a small family of surfaces, and the right-hand region alone can show three different panes. Hold the four in mind as the frame; the rest hang off it.



Read it top to bottom. The Tree names the book; the green ► marks the paragraph currently loaded in the Editor, wherever your tree cursor happens to be. The Editor holds that paragraph's lines with a number gutter down the left. The right region — here showing the AI pane — carries the conversation. The two bottom bars wait for a search query and an AI prompt. The status bar, last of all, tells you where you stand: the breadcrumb to the open paragraph, its word count, the red ● that means unsaved edits, and the current AI scope and inference mode.

TERMINAL-FIRST

Because it is only text, this window runs anywhere a terminal does — over SSH, inside `tmux`, on a bare tiling window manager, on a laptop on a train with the lid half-closed. There is no separate GUI, no browser tab, nothing to install beyond the one binary. The frames in this book are faithful to what you will actually see.

The right-hand region — three panes in one place

The region to the right of the Editor is not a single pane but a slot that holds **one of three** at a time. This is the part of the layout newcomers find surprising, so it is worth naming plainly.

TERM The right region

The right-hand slot cycles between three panes: the **AI pane** (a conversation you drive), the **Output pane** (a notice board subsystems post to), and the **Thoughts pane** (a quiet home for long reflective text). Only one is visible at a time; switching between them is a single chord.

The three are three different **kinds** of surface, which is why they are three panes and not one. The AI pane is a **dialogue** — you ask, the model answers, the history scrolls. The Output pane is a **notice board** — structured, one-way messages that the fact-checker, the continuity watch, the translation engine, a finished background job, or a Bund script post for you to read, act on, and dismiss; each carries a severity glyph (● info, ▲ warning, ⊗ contradiction, ♻ still-running) and expires on its own so the board never becomes a junk drawer. The Thoughts pane is a **reading** surface — a scrollable, read-only place where the longer, essayistic output lands, such as the Inner Theologian's questions or a developmental brief.

● ● ● *The right region showing the Output pane — a notice board*

```

┌─ Output · 3/4 · fact-check ───┐
│ ⊗ fact_conflict                │
│ │ "they reached Rillmark by nightfall" – the bible │
│ │ puts Rillmark six days' ride away.              │
│ ● review_complete              │
│   3 checks clean · 1 contradiction · 0 warnings    │
│ ▲ uncovered_words              │
│   2 word(s) uncovered: sword, sun                  │
└────────────────────────────────┘
└─ ↑↓ select  o expand  Enter jump  a ask-AI  d dismiss ─┘

```

Which pane the slot shows when you launch is remembered: on a project's very first start it opens on the Output pane, and after that Inkhaven restores whichever of the three you last left active, exactly as it restores which paragraph was open. When fresh content arrives for a pane it will quietly surface that pane for you — unless you are actively working in a right pane at the time, in which case it holds still rather than steal your place mid-read.

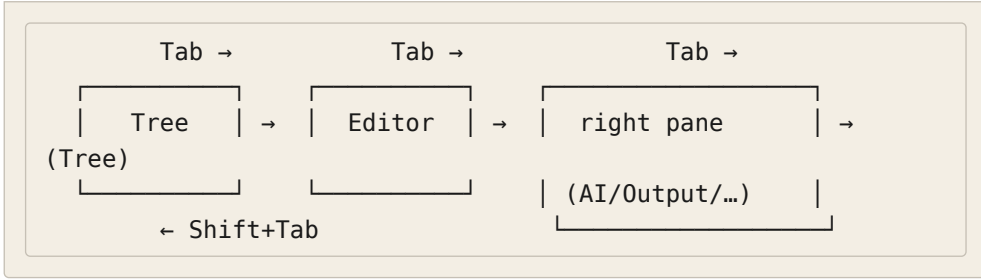
Moving your attention — focus and navigation

Only one pane listens to your keys at a time. Moving that attention around is the most common thing you will do, so Inkhaven gives you both a cycle and a set of direct jumps.

The plain cycle — Tab

The everyday key is **Tab**. It walks focus through a short ring — the Tree, the Editor, and **whichever right pane is currently shown** — then back to the Tree. It is a three-stop loop, not a tour of every surface: the right region counts as one stop, so if the slot is on the Output pane, **Tab** visits the Output pane; if it is on the AI pane, **Tab** visits the AI pane. **Shift+Tab** walks the same ring in reverse. You will reach for **Tab** far more than any other navigation key, and because Inkhaven intercepts it before the text editor sees it, pressing **Tab** never inserts a literal tab into your prose.

● ● ● *Tab cycles a three-stop ring*



Cycling the right region — Ctrl+B Tab

To change **which** pane the right slot shows, you cycle the region itself with **Ctrl+B Tab**. Each press advances the slot through the three panes in order — Output, then AI, then Thoughts — and moves focus onto it; **Ctrl+B Shift+Tab** steps backward. This is a different motion from plain **Tab**: **Tab** moves your focus **to** the region, **Ctrl+B Tab** changes **what the region is**. The chord works from anywhere — even mid-edit — because the **Ctrl+B** prefix (which you will meet properly in a moment) swallows the keystroke before the editor could read **Tab** as a tab.

Jumping straight to a pane

When you know exactly where you want to be, skip the cycle and jump. Each of the main surfaces has a direct key, so no pane is ever more than one chord away.

Tab	Cycle focus: Tree → Editor → current right pane → Tree.
Shift+Tab	Cycle focus in reverse.
Ctrl+B Tab	Cycle the right region Output → AI → Thoughts, and focus it.
Ctrl+B Shift+Tab	Cycle the right region backward.
Ctrl+T	Focus the Tree pane.
Ctrl+1	Focus the Editor pane.
Ctrl+3	Focus the AI pane.
Ctrl+/	Focus the Search bar (top). Ctrl+4 does the same.
Ctrl+I	Focus the AI prompt bar (bottom). Ctrl+5 does the same.

How you know which pane has focus

Focus is never a guess — every pane shows it. The Editor’s border carries the signal most vividly: **green** when the pane is focused and saved, **yellow** when focused with unsaved edits, and plain **white** when it is not focused at all. In the Tree, the open paragraph keeps its green ► marker whether or not the Tree has focus, so you can always find your place. And when you have started a chord but not finished it, a yellow **META** chip lights up in the status bar to tell you the tool is waiting for your next key. Between the border, the marker, and the chip, the window always tells you what it is listening to.

FOCUS-LOSS AUTOSAVE

Whenever focus leaves the Editor — by Tab, by a direct jump, by Esc from another input — the open paragraph is saved for you if it was dirty. You can shift your attention mid-sentence without a thought for losing work; the words are on disk before the next pane has your keystrokes.

The chord system — the heart of the interface

Inkhaven has more commands than a keyboard has keys, and it refuses to bury the useful ones behind menus. Its answer is the **chord**: a short, deliberate two-key sequence, led by a prefix that tells the tool “a command is coming.” Once the idea clicks, the whole interface opens up, because every serious action in the program is one chord away from wherever you are.

TERM Chord

A **chord** is a two-key command: a **prefix** key, then an **action** key. You press the prefix, release it, then press the second key — they are struck in sequence, not held down together. Between the two presses the tool is in a brief **pending** state, waiting for the action key and showing you it is waiting.

There are two prefixes, and the difference between them is a genuine distinction of **kind**, not an accident of which keys were free.

Ctrl+B — the meta prefix

Ctrl+B is the **meta** prefix: it introduces commands that **act on your book** — add a chapter, take a snapshot, run a check, tag a paragraph, translate a line. Press Ctrl+B and release it, and the tool enters **meta mode**: the status bar lights its yellow **META** chip and prints the actions available for the pane you are in. The next key you press selects one of them. Esc backs out of a pending chord without doing anything, and any key the table does not recognise cancels with a hint naming which pane's table it consulted.

TERM Meta prefix

Ctrl+B — the key that opens **meta mode**. The single most important thing to learn about it is that the action table is **pane-specific**: Ctrl+B S means **save the paragraph** in the Editor, **add a subchapter** in the Tree, and **clear the inference** is one of the AI pane's few actions. The same second key, read against a different pane, is a different command.

● ● ● *Meta mode pending — the status bar prompts for the second key*

```

┌ Editor · the-quay [modified] ───────────┐
│ 1 The rain came sideways off the harbour, and... │
├──────────────────────────────────────────┤
│ META S save · N snapshot · R history · L load · │
│      T retitle · P place-RAG · C char-RAG · H help │
└──────────────────────────────────────────┘

```

That the table is pane-specific is not a complication to memorise but a simplification to lean on. You do not learn one enormous list of chords; you learn the handful that belong to the pane you are working in, and the same letters mean the natural thing in each place. In the Tree, Ctrl+B B, C, S, P add a **book**, a **chapter**, a **subchapter**, and a **paragraph** — the second key is the word's first letter. In the Editor, Ctrl+B N takes a **new** snapshot and Ctrl+B R opens its **history**. A last tier of meta chords — the ones that run whole-book intelligences, like Ctrl+B Shift+C to run every fast checker at once — work from **any** pane, because they belong to the book rather than to a place in it.

Ctrl+V — the view prefix

Ctrl+V is the **view** prefix, and it introduces a different kind of command: things that **show you a view** or **reach a tool** rather than change the manuscript's structure — export this paragraph to markdown, open the fuzzy paragraph picker, float a rendered preview, jump a bookmark, run the Editorial Pass, open the command palette. Where **Ctrl+B** is about **acting on the book**, **Ctrl+V** is about **looking through it** and reaching the pickers and panels that help you look. It reads the same way — press and release **Ctrl+V**, then the action key — and like the meta layer it is fully rebindable.

The mnemonic is worth stating out loud, because it is what lets you **guess** a chord correctly instead of looking it up: **Ctrl+B** for the **book** (things you do to it), **Ctrl+V** for the **view** (things you do to see it). Nearly every chord you meet in this manual sits under one of these two prefixes, and the prefix tells you which half of your mind the command lives in.

Overlays — the modal stack on top of it all

Some chords do not run and return; they **open something** — a picker, a confirmation, a full-screen editor for your config, a floating preview. These are **overlays**, and while one is up it sits on top of the panes and takes your keys for itself. The pane beneath keeps its visual focus but does not see input until the overlay closes, and **ESC** is the near-universal way to close the top overlay and drop back to what was underneath. Overlays stack: a picker can open a confirm on top of itself, and closing the confirm returns you to the picker. This is why **ESC** is the key you can always press when you feel lost — it peels one layer off the top and never does anything destructive.

The status bar and the title bars

The bottom line and the little labels along the tops of panes are not decoration; they are a continuous, quiet readout of state. Learning to glance at them is most of what “knowing where you are” amounts to.

The status bar

The status bar is the window's single most information-dense line. In calm moments it shows the breadcrumb path to your cursor, the open paragraph's word count, a red ● when there are unsaved edits, and the current AI scope and inference mode. In busy moments it is also where transient messages land — a search reporting **match 1 / N**, a diagnostic jump reading **diag 2/5 line 40:12**, a **marked N**

count while you multi-select in the Tree, the yellow **META** prompt while a chord is pending, or a notice that a file changed on disk and was reloaded. It is the first place to look when you wonder what just happened.

● ● ● *The status bar — a running readout of the moment*

```

2 The City ▶ the-quay ● 214w scope=None infer=Local
└─ breadcrumb ─┐ dirty ┘ | | ┘
Local / Full
word count ┘ | AI scope (F9)

```

Optional chips ride the same line when you turn them on — a POV chip naming the paragraph’s viewpoint character, a `lang=` chip for the prompt language, a live syllable readout while a verse paragraph is open. They are quiet by default and each has its own toggle; the point is that the status bar is the one place Inkhaven speaks to you without being asked.

The title bars

Every pane wears a thin title along its top, and each title earns its space. The Editor’s names the open paragraph and appends `[modified]` when it is dirty. The Tree’s is simply the pane’s name. The right region’s title tells you which of the three panes you are looking at and its live state — the AI pane’s reads like `AI · llama · done · infer=Full · scope=Paragraph`, folding in the provider, the stream state, the inference mode, and the scope; the Output pane’s shows a `shown/total · filter` count; the AI prompt bar’s title echoes the scope as `AI prompt · scope: Paragraph`. You rarely read them word for word, but they are the reason you are never guessing which mode a pane is in.

Finding a chord — the palette and the quick reference

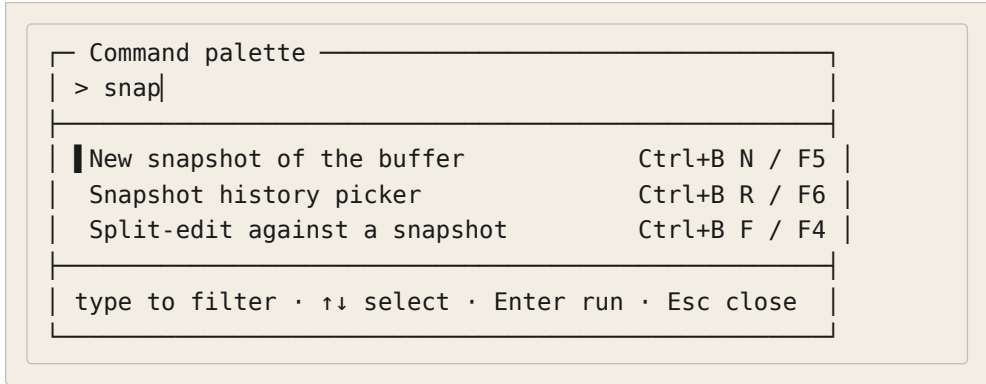
Two prefixes and a pane-specific table is a system you can **reason** about, but nobody memorises every chord, and Inkhaven does not ask you to. Three surfaces exist precisely so you can find a command without remembering its keys.

The command palette — Ctrl+V Space

`Ctrl+V Space` opens the **command palette**: a fuzzy search over the entire keybind registry. Start typing any part of a command’s name, its chord, or its description, and the list narrows to match; `↑↓` moves the selection, `Enter` runs it,

Esc closes. This is the fast way to reach **any** command without knowing its chord — type “snapshot”, or “fact”, or “outline”, and run what comes up. It is also how you **learn** the chords: the palette shows each command’s key beside its name, so the thing you searched for by word today you will strike by chord tomorrow.

● ● ● *Ctrl+V Space — fuzzy-find any command in the registry*



The quick reference — **Ctrl+B H**, and **?** from the Tree

Where the palette is for **running** a command you can half-name, the **quick reference** is for **browsing** what is available from where you stand. **Ctrl+B H** opens it from any pane — a floating, scrollable panel of the chords that apply, pane-aware so it shows you the ones that matter here first. Scroll with the arrows or **PgUp** / **PgDn**; **Esc** closes it. When the Tree has focus you can also open it with a bare **?**, since the Tree is the one pane where **?** is not a character you would type; in the Editor and the input bars **?** stays a literal question mark, and **Ctrl+B H** is the way in.

Ctrl+V Space	Command palette — fuzzy-find and run any command by name, chord, or description.
Ctrl+B H	Quick reference — the pane-aware chord panel, from any pane.
?	Quick reference, but only when the Tree pane has focus.
Ctrl+B V	Version / credits pane — the running version, author, licence, and dependencies.

Between them the two surfaces cover both ways a person forgets a chord. When you know **what you want** but not the keys, reach for the palette. When you want

to **see what is possible** from where you are, open the quick reference. Neither asks you to leave the window or read this book.

Fullscreen and focus modes

The four-pane layout is the working default, but sometimes you want less on the screen — the prose alone while you draft, or one right pane blown up while you read a long result. Inkhaven has a mode for each, and each is a toggle: press to enter, press again to return.

Focus mode — **Ctrl+B Shift+W**

Ctrl+B Shift+W enters **focus mode** — the distraction-free layout. It hides the Tree, the AI pane, the Search bar, and the AI prompt, and gives the whole window to the Editor, forcing focus there as it opens. It is the mode for the moment you have decided to stop tending the book and simply write it: nothing on screen but your paragraph. Press **Ctrl+B Shift+W** again to restore the four panes. (The setting is called “typewriter mode” in a few older strings and in the config file, for backward compatibility — same mode, the friendlier name won.)

● ● ● *Focus mode — the Editor alone, everything else hidden*

```

┌ Editor · the-quay ───────────────────────────────────────────────────────────┐
│ 1 The rain came sideways off the harbour, and                               │
│ 2 Mara counted the lamps as they went dark, one                             │
│ 3 by one, the length of the quay. Six days' ride                             │
│ 4 still lay ahead, and the city behind her had                             │
│ 5 already forgotten her name.                                                 │
└────────────────────────────────────────────────────────────────────────────────┘

```

Fullscreen panes — **Ctrl+Z f** and **Ctrl+B K**

When it is a **result** you want to sink into rather than the prose, fullscreen the right pane instead. **Ctrl+Z f** blows up the currently shown right pane — the Output pane or the Thoughts pane — to fill the window, for reading a long notice board or a developmental brief without the Editor crowding it. The AI pane has its own dedicated fullscreen, **Ctrl+B K**, which lays out the AI pane, its scrollable chat

history, and the prompt across the whole window for a long conversation. Focus mode and AI-fullscreen are mutually exclusive — you are either writing alone or talking at length, not both — and each toggle returns you to the four-pane layout.

Ctrl+B Shift+W Focus mode — hide everything but the Editor. Press again to restore.

Ctrl+Z f Fullscreen the current right pane (Output / Thoughts).

Ctrl+B K Fullscreen the AI pane with its chat history and prompt.

A note on the mouse

Inkhaven is a keyboard instrument, but it does not ignore the mouse. It captures mouse input on start: a left-click moves focus to the pane you click, and inside a pane the click lands where you expect — the row cursor in the Tree, the character cursor in the Editor (clicks in the number gutter are ignored). The scroll wheel scrolls whatever it is over — three rows per tick in the Tree, three lines in the Editor, the chat history in the AI pane, the message list in the Output pane. It is enough to point and scroll when a hand is already on the mouse, and it is entirely optional — nothing in this book requires it.

SELECTING TEXT THROUGH THE TERMINAL

Because Inkhaven captures the mouse, dragging to select does not reach your terminal's own copy by default. Hold **Shift** (or **Option**, depending on the terminal) while you drag to bypass the capture and select through the terminal, or toggle capture off entirely with **Ctrl+Shift+M** when you want the terminal's native clipboard for a while.

The window you have just mapped does not change from here. Every later chapter — the editor's depths, the AI's scopes, the world and its facts, the reading intelligences — plays out on these same four panes, reached by these same chords, read off this same status bar. You will spend the rest of the book learning what to **do** in this window; you now know what the window **is**.

WHAT YOU LEARNED

- The window is **four regions**: the **Tree** (left), the **Editor** (centre), and a **right slot** that shows one of the **AI**, **Output**, or **Thoughts** panes, over a Search bar, an AI prompt, and a status bar.
- Plain **Tab** cycles focus **Tree** → **Editor** → **current right pane**; **Ctrl+B Tab** cycles **which** pane the right slot shows (**Output** → **AI** → **Thoughts**); **Ctrl+T**, **Ctrl+1**, and **Ctrl+3** jump straight to a pane.
- A **chord** is a two-key command: **Ctrl+B** (the **meta** prefix — **act on the book**) or **Ctrl+V** (the **view** prefix — **reach a view or tool**), then a **pane-specific** action key. **Esc** cancels a pending chord or closes the top overlay.
- The **status bar** and **title bars** are a running readout — breadcrumb, word count, the red ● dirty chip, AI scope and inference mode, and the yellow **META** prompt while a chord waits.
- You never have to memorise a chord: **Ctrl+V Space** fuzzy-finds and runs any command, **Ctrl+B H** (or **?** in the Tree) opens the pane-aware quick reference.
- **Ctrl+B Shift+W** gives the window to the Editor alone; **Ctrl+Z f** fullscreens the current right pane and **Ctrl+B K** the AI pane — each a toggle back to the four panes.

PART II

Writing

CHAPTER 4

4

The Editor

The Editor is the pane you will live in. The last chapter mapped the whole window; this one goes to the centre of it and stays there, because the great majority of a writing session is one paragraph open in front of you and your hands on the keys. Everything the Editor does — move, select, cut, find, replace, save, split, check — is reachable without the mouse and without a menu, and almost all of it follows the plain conventions your fingers already know. The handful of places where Inkhaven departs from those conventions it departs for a reason, and this chapter names each one so nothing surprises you mid-sentence. Read it once for the shape, then let the muscle memory take over.

ONE PARAGRAPH AT A TIME

The Editor holds a **single paragraph** — one `.typ` file — not a whole chapter scrolled end to end. This is deliberate: a paragraph is Inkhaven’s unit of prose, of snapshot, of search, and of every reading intelligence in the later chapters. You move between paragraphs in the Tree (Chapter 5); you write **within** one here. Opening another paragraph saves this one first, so the boundary costs you nothing.

Opening a paragraph, and reading its state

You open a paragraph from the Tree: move the row cursor to it and press `Enter`. Focus jumps to the Editor, the file’s text loads into the buffer with a line-number gutter down the left, and from that moment the pane is a running readout of where the words stand. You never have to wonder whether your work is safe — the Editor tells you, in three places at once.

The most vivid signal is the **border colour**, and it speaks only while the Editor has focus. A **green** border means the buffer matches what is on disk — you are saved, nothing is pending. A **yellow** border means you have unsaved edits. When the pane is **unfocused** the border goes plain white and hands the dirty signal off to two always-on indicators: the `[modified]` suffix in the title, and the red ● chip in the status bar. Between them you can tell the state of your paragraph from anywhere in the window.

TERM Dirty

A buffer is **dirty** when it holds edits not yet written to disk. Inkhaven shows dirtiness three ways — a yellow border (focused), a `[modified]` title suffix, and the red ● in the status bar — and it clears all three the instant a save completes. “Clean” is the opposite: buffer and file agree.

The Editor’s title carries the paragraph’s name, the `[modified]` chip when dirty, and a live cursor read-out in the form `L<row> C<col>` — the line and column your cursor sits on, updated as you move. It is the quiet coordinate you glance at when a message says “jump to line 40” or when you want to know how far down a long paragraph you have wandered.

● ● ● *The Editor title — name, dirty chip, live cursor*

```

┌ Editor · Opening Scene [modified] · L4 C18 ───────────┐
│ 1 The rain came sideways off the harbour, and        │
│ 2 Mara counted the lamps as they went dark, one      │
│ 3 by one, the length of the quay. Six days'          │
│ 4 ride still lay ahead of her, and the city          │
│ 5 behind had already begun to forget her name.       │
└────────────────────────────────────────────────────────┘

```

One kind of paragraph opens in a state you cannot type into. If the paragraph lives inside the **Help** system book, the Editor goes read-only: the border turns teal, the title gains a `[read-only]` mark, and every key that would change the text is intercepted with a `Help is read-only` status message. Movement, copy, search, and scrolling all still work — you can read and lift from Help freely — but `Ctrl+S` is a no-op and nothing you press alters the file. Help is reserved for the material that answers `F1` lookups; your own prose simply never lives under it.

Moving the cursor

Movement is entirely conventional, which is the point — your existing habits transfer whole. The arrow keys move a cell or a line at a time. `Ctrl+←` and `Ctrl+→` jump by **word**, landing on each word boundary, for crossing a line faster than a character at a time. `Home` and `End` snap to the start and end of the current line; `Ctrl+Home` and `Ctrl+End` leap to the very top and bottom of the paragraph. `PageUp` and `PageDown` move by a viewport for the longer paragraphs. There is nothing here to unlearn.

<code>← → ↑ ↓</code>	Move one character or one line.
<code>Ctrl+←</code>	Jump to the previous word boundary.
<code>Ctrl+→</code>	Jump to the next word boundary.
<code>Home / End</code>	Start / end of the current line.
<code>Ctrl+Home</code>	Top of the paragraph.
<code>Ctrl+End</code>	Bottom of the paragraph.

PageUp / **Page-** Move one viewport up / down.
Down

Jumping between scene breaks

A long paragraph of fiction is often a run of scenes separated by a break line — `* * *`, `***`, `---`, `___`, `###`, `~~~`, or a lone `§`. Inkhaven recognises all of these, and gives you a pair of chords to leap between them without hunting: **Ctrl+B >** jumps the cursor forward to the next scene break, **Ctrl+B <** back to the previous one. They are the fast way to walk the beats of a scene-heavy passage. (The mnemonic borrows vim’s angle brackets; the **<** avoids a collision with the tag-search chord that wanted the same key.)

Ctrl+B > Jump to the next scene break in the paragraph.

Ctrl+B < Jump to the previous scene break.

Selecting text

Inkhaven has two kinds of selection: the ordinary **linear** run you know from every editor, and a **rectangular** block for column work. They are separate models, and only one is ever active at a time.

Linear selection

Hold **Shift** and move: **Shift+←** and **Shift+→** extend the selection a character at a time, **Shift+↑** and **Shift+↓** a line at a time. Combine the two — a few **Shift+→** to reach across a phrase, **Shift+↓** to take whole lines — and you build up exactly the run you want, drawn in reversed video so it stands out against the syntax colours. **Ctrl+A** selects the entire paragraph at a stroke. The cut, copy, and paste chords in the next section all operate on whatever linear selection is live; with none, they act on the cursor position instead.

Vertical block selection

Sometimes what you want is not a run of text but a **rectangle** of it — a column of leading numbers, a stack of names, the aligned left edge of a verse stanza. For that, hold **Alt** and move. The first **Alt+arrow** drops an anchor at the cursor and enters block-select mode; every further **Alt+arrow** grows a rectangle from that anchor to the cursor, redrawn each frame in reversed video. When the rectangle covers what you want, **Alt+C** copies it to the system clipboard as a multi-line string — one line per row of the block. **Esc** cancels the block without copying, and any ordinary key ends block mode and falls back to normal editing.

● ● ● *A block selection — a rectangle, not a run*

```

Editor · Roster
1  █14█ Mara    quay-watch
2  █09█ Tomas  ropewalk
3  █22█ Elin   the counting-house
4  █07█ Bran   harbour gate

Alt+↑↓↔ grow · Alt+C copy rectangle · Esc cancel

```

BLOCK MODE IS COPY-ONLY

Rectangular **copy** is supported; rectangular **cut** and **paste** are not, in this release — the multi-line character surgery they need is more than the underlying text widget exposes cleanly. Copy-only still covers the everyday cases: pulling out a column of figures, a list of names, or the aligned start of a stanza to reuse elsewhere.

The clipboard — cut, copy, and paste

Here is the one place the Editor departs from the keys your fingers expect, and it departs for a concrete reason: the conventional **Ctrl+V** collides with the terminal's own paste on many setups, and **Ctrl+X** and **Ctrl+Z** are already spoken for elsewhere in Inkhaven. So the clipboard chords are chosen to sit on distinct, unambiguous keys. **Ctrl+C** copies, **Ctrl+K** cuts, and **Ctrl+P** pastes; **Ctrl+A** selects everything first. Learn this one triad and the rest of the editor is convention.

- Ctrl+C** Copy the selection to the system clipboard.
- Ctrl+K** Cut the selection to the clipboard (marks dirty).
- Ctrl+P** Paste at the cursor, replacing any selection.
- Ctrl+A** Select the entire paragraph.

The clipboard is the **real, system** clipboard — Inkhaven talks to it through the **arboard** library, so what you copy here you can paste into another application

and vice versa. On a headless or restricted session where no system clipboard is reachable — a bare SSH login, some Wayland setups — the chords do not fail. They fall back to an internal yank buffer, so cut, copy, and paste keep working **within** the editor session; the only thing you lose is crossing the process boundary to other apps.

WHY CTRL+P FOR PASTE

If your muscle memory reaches for `Ctrl+V`, retrain it here to `Ctrl+P`. `Ctrl+V` is Inkhaven’s **view** prefix (the pickers and panels of Chapter 3), and on many terminals it is also the native paste — leaving it as an editor chord would mean two pastes fighting over one key. `Ctrl+P` is the explicit, unambiguous Inkhaven paste, and it behaves identically everywhere.

Deleting by the chunk

Beyond `Backspace` and `Delete`, four chords remove whole **pieces** of a line at once — the fast erasers for revision. `Ctrl+Backspace` deletes the word before the cursor. `Ctrl+D` removes the entire current line and closes the gap. `Ctrl+E` clears from the cursor to the end of the line; `Ctrl+W` clears from the cursor back to the start. None of them touch your clipboard — each saves and restores the yank buffer around itself — so a big delete never costs you the last thing you copied.

Ctrl+Backspace	Delete the word before the cursor.
Ctrl+D	Delete the whole current line.
Ctrl+E	Delete from the cursor to the end of the line.
Ctrl+W	Delete from the cursor back to the start of line.

IF YOUR TERMINAL EATS CTRL+W

Some shells and multiplexers (bash, tmux) grab `Ctrl+W` for their own “delete previous word” before Inkhaven ever sees it. If `Ctrl+W` seems inert, that is your terminal layer intercepting it, not a bug — the fix is to rebind it in `inkhaven.hjson` or reach the same delete another way. Chapter 27 on configuration covers the rebinding.

Undo and redo

Undo is `Ctrl+U`; redo is `Ctrl+Y`. The history is **per paragraph** — each open buffer keeps its own stack — so undoing in one paragraph never reaches back into another. If you wonder why undo is not the usual `Ctrl+Z`, the answer is the same distinctness principle as the clipboard: `Ctrl+Z` is Inkhaven’s **Bund** scripting prefix (Part VIII), and on many terminals the bare `Ctrl+Z` also suspends the process to the shell’s job control. Putting undo on `Ctrl+U` sidesteps both. Reach for `Ctrl+U` to step back, `Ctrl+Y` to step forward again.

Ctrl+U Undo the last edit (per-paragraph history).

Ctrl+Y Redo.

Every change since the last save

While you write, Inkhaven quietly renders **bold** every character you have added since the paragraph was last saved. It is a running visual diff against the on-disk version: at a glance you can see exactly what this editing pass has touched, which is invaluable when you have been picking at a paragraph and want to know what actually changed. The bolding is computed with a fast per-line diff, so it keeps up with typing at literary scale, and it **clears the moment you save** — bold means “new since disk”, and a save makes the buffer and the disk agree, so there is nothing left to mark. Watching the bold appear and then dissolve on `Ctrl+S` is the simplest confirmation that your work has landed.

Finding and replacing

Search inside a paragraph is regex-powered and lives on two chords: `Ctrl+F` to find, `Ctrl+R` to find-and-replace. Both take the full Rust regular-expression syntax, so a plain word matches literally while `\bword\b` pins a whole-word match, `(?i)` makes it case-insensitive, and `(?s)` lets `.` cross a line.

Press `Ctrl+F` and a magenta-bordered Find modal opens. Type a pattern, press `Enter`, and every match in the buffer lights up in red while the cursor jumps to the first; the status bar reports `match 1 / N` so you know how many you have. From there, `Ctrl+X` is “repeat” — it advances to the next match and wraps at the

end — and the match your cursor currently sits on is drawn a brighter red-and-bold so it stands out among its siblings. **Esc** clears the search and drops the highlights, returning you to plain editing.

- ● ● *Ctrl+F* — the regex Find modal

A screenshot of a 'Find (regex)' dialog box. The title bar says 'Find (regex)'. Inside the dialog, there is a text input field labeled 'Search:' containing the text 'the\s+thunder|'. Below the input field, there is a line of text that reads 'Enter find · Ctrl+X next · Esc cancel'. The dialog box has a dashed border and a light gray background.

For replacement, press **Ctrl+R** instead. The modal grows a second field; **Tab** switches between Search and Replace. The **first** press of **Enter** applies one replacement — the current match — and keeps you in replace mode so you can walk the rest one at a time with **Ctrl+X**. A **second** press of **Ctrl+R** while replace mode is live does the wholesale thing: replace every remaining match and exit. Re-opening either modal after a search pre-fills your last pattern and replacement, so refining a search is a matter of editing what is already there.

- ● ● *Ctrl+R — Find & Replace, two fields*

```
Find & Replace (regex)
```

```
Search:  the\s+thunder
Replace: the storm
```

```
Enter one · Ctrl+R all · Tab field · Esc cancel
```

There is one more reach in the replace modal. **Ctrl+B** toggles the **scope** of the replacement between this paragraph and the **whole book**. In book scope, **Enter** scans every paragraph in your user books and opens a review modal: matches shown in context, **↑↓** to move, **Space** to skip the one under the cursor, **a** to keep all, **n** to skip none, **Enter** to apply, **Esc** to cancel. Book-wide replace starts in the safe whole-word literal mode, and **w**, **i**, **x** toggle whole-word, ignore-case,

and full-regex in place. Every paragraph it changes is snapshotted first — annotated with the substitution — so **F6** is always your undo. The same operation is available

headless as `inkhaven replace`, covered in the reference.

SEARCH MATCHES A LINE AT A TIME

In-buffer find works line by line, so a pattern that must span a line break will not match. In practice the literary tasks — swapping a word, changing a character’s name, fixing a repeated phrase — all sit within a line, so this rarely bites; when you truly need a cross-line change, the whole-book replace and the CLI are the wider tools.

Saving, and the round trip

Saving is `Ctrl+S`. It writes the paragraph’s `.typ` file to disk, updates the metadata, re-embeds the text into the semantic index so search stays current, clears the dirty flag, and reloads the Tree so word counts refresh. The status bar confirms it with the path and a word count. Because your prose is **plain files**, this is a genuine round trip: the words Inkhaven holds and the words on disk are the same bytes, and you are free to open, diff, or version-control that file with any tool you like — Inkhaven will notice a change made from outside and reload it.

TERM The round trip

Inkhaven never locks your prose inside a private format. A paragraph **is** a `.typ` file; a save is an ordinary write of that file. Edit it in another program while Inkhaven is open and, if your buffer here is clean, Inkhaven silently reloads the newer version; if your buffer is **dirty**, it warns rather than clobber your edits and leaves the choice — overwrite with `Ctrl+S`, or reconcile — to you.

You will reach for `Ctrl+S` less than you might expect, because Inkhaven saves for you at three natural moments. It saves on **idle** — after a few seconds of not typing (the interval is `editor.autosave_seconds`, default five). It saves on **paragraph switch** — opening another paragraph writes this one first. And it saves on **focus loss** — the moment your attention leaves the Editor by `Tab`, a direct jump, or

Esc from another input, a dirty paragraph is written before the next pane sees a keystroke. Between the three, moving around the window costs you no work; the deliberate **Ctrl+S** is there for the moments you want the confirmation.

THE CRASH MIRROR

Against the rare case of a crash between saves, Inkhaven keeps a **mirror** of your unsaved buffer. Every couple of seconds while a paragraph is dirty (the window is `editor.crash_mirror_seconds`, default two) it pushes the current text and cursor position into the crash-report context, so if the program ever panics its handler can flush that buffer to a rescue file on the way down. The worst you can lose is one debounce window of typing — and a split or side-by-side second buffer is mirrored just the same.

Split-edit — two versions at once

Some revisions are easier when you can see where you came from. **F4** toggles **split-edit**: the Editor area divides in half horizontally. The **upper** pane is your live, read-write editor, exactly as full-featured as ever. The **lower** pane is a frozen, read-only snapshot of the buffer as it stood the instant you pressed **F4**, drawn in dim grey. You rewrite above while the earlier version stays visible below.

Because a long earlier passage may not line up with where you are editing, the lower pane scrolls on its own: **Ctrl+H** nudges it up a line, **Ctrl+J** down. These two chords are routed to the snapshot pane **only while split is active**, so they shadow nothing in ordinary use. When you are done, **F4** again closes the split and drops the snapshot — or **Ctrl+F4** **accepts** it, replacing your live buffer with the frozen copy. Acceptance is the clean way to roll back a change you decided against: split, rewrite, and if the rewrite is worse, take the old version wholesale.

● ● ● *F4 split-edit — live above, frozen snapshot below*

```

┌ Editor · Opening Scene [modified] ───────────
│ 1 The rain came hard off the harbour, and Mara
│ 2 counted the lamps as they failed, one by one.│
└ snapshot · line 1/4 · Ctrl+H/J scroll ─────────
│ 1 The rain came sideways off the harbour, and
│ 2 Mara counted the lamps as they went dark, one

```

The lower half is one particular **snapshot**, and snapshots are Inkhaven's point-in-time bookmarks of a paragraph: **F5** takes a fresh one, **F6** opens the picker of every snapshot for this paragraph to load or compare. They are the subject of their own chapter; here it is enough to know that split-edit builds on the same idea — a captured earlier version you can hold beside the present one and, if you wish, restore.

F4	Toggle split-edit; capture the snapshot on enter.
Ctrl+F4	Accept the snapshot — replace the live buffer.
Ctrl+H	Scroll the lower snapshot pane up (split only).
Ctrl+J	Scroll the lower snapshot pane down (split only).
F5	Take a fresh snapshot of the buffer.
F6	Open the snapshot picker (load / diff / delete).

Grammar check — F7 and the g-apply

Grammar checking is the Editor's most-used piece of AI, and it is built to be safe: it never touches your prose until you say so. Press **F7** and Inkhaven sends the open paragraph to your configured model with a grammar-and-punctuation prompt keyed to the project's language, explicitly told to preserve every bit of Typst markup. The review streams into the AI pane and focus moves there, so you watch it arrive in real time. Nothing has changed in your buffer yet — the check is a proposal, not an edit.

To **take** the correction, press **g** in the AI pane. This is the “grammar-check apply”: it lifts only the corrected paragraph out of the response — the model returns it between markers so Inkhaven knows exactly which part is the clean text — and overwrites the editor buffer with it. Because the grammar prompt preserves Typst markup verbatim, the apply skips the usual markdown conversion and drops the corrected prose in as-is. Every character that changed is then rendered in red, so you can read the correction as a diff against what you wrote.

● ● ● F7 review in the AI pane, ready for the g-apply

```
AI · llama · done · scope=Paragraph —
ai <<<CORRECTED>>>
  The rain came sideways off the harbour, and
  Mara counted the lamps as they went dark,
  one by one, the length of the quay.
  <<<END>>>

g apply · c copy · r replace · Esc back to prompt
```

Those red change-highlights are deliberately stubborn: unlike the “added since save” bolding, they **survive a save**, so the record of what the grammar pass altered stays visible while you read back over it. When you are satisfied, dismiss them explicitly — **Ctrl+B** then **C** clears the highlight, and so does simply moving to another paragraph. If the response has no correction Inkhaven can lift, **g** refuses with a status message rather than guessing.

THE AI NEVER WRITES WITHOUT YOUR KEY

F7 only **shows** you a correction; **g** is the separate, deliberate keystroke that accepts it. This two-step — propose in the AI pane, apply on demand — is the same contract every AI-to-prose path in Inkhaven honours: the assistant advises, and no word of your manuscript changes until you press the key that says so. The change-highlights then let you audit exactly what it did.

The reading-pace preview

One editor-scoped tool is worth meeting here because it changes how a paragraph **reads** rather than how it is edited. **Ctrl+B Shift+E** opens the **reading-pace preview** — a teleprompter that walks a highlight through the open paragraph one word at a time, at a reader’s speed rather than an editor’s glance (the rate is `editor.reading_wpm`, default two hundred words a minute). Words already passed dim, the current word is reverse-highlighted, the words ahead sit normal, and a footer tracks your position and the time left. Experiencing your own prose at the pace a reader will meet it surfaces problems the editing eye skims straight over — a sentence that drags, a beat that lands too abruptly. **Space** pauses and resumes, **←** and **→** step the highlight a word at a time, **r** restarts from the top, and **Esc** closes it. It reads the same clean prose the audiobook export uses, with the structural markup stripped away.

- | | |
|-----------------------|--|
| F7 | Grammar-check the open paragraph (streams to the AI pane). |
| g | In the AI pane: apply the correction to the buffer. |
| Ctrl+B C | Dismiss the grammar change-highlights. |
| Ctrl+B Shift+E | Reading-pace preview (teleprompter over the paragraph). |

WHAT YOU LEARNED

- The Editor holds **one paragraph** — a plain `.typ` file — and shows its save state three ways: a **green** (saved) or **yellow** (dirty) focused border, the `[modified]` title chip, and the red ● in the status bar.
- Movement is conventional — arrows, `Ctrl+←/→` by word, `Home/End`, `Ctrl+Home/Ctrl+End`, `PageUp/PageDown` — with `Ctrl+B > / Ctrl+B <` to hop between scene breaks.
- Selection is either **linear** (`Shift` +arrows, `Ctrl+A` for all) or a **rectangular block** (`Alt` +arrows, `Alt+C` to copy — copy-only this release).
- The clipboard breaks convention on purpose: **copy** `Ctrl+C`, **cut** `Ctrl+K`, **paste** `Ctrl+P` — the real system clipboard, with an in-editor fallback when none is reachable.
- Chunk-deletes (`Ctrl+Backspace` word, `Ctrl+D` line, `Ctrl+E` to end, `Ctrl+W` to start) never touch the clipboard; undo is `Ctrl+U`, redo `Ctrl+Y`, per paragraph.
- `Ctrl+F` finds by regex (`Ctrl+X` next), `Ctrl+R` replaces (again for replace-all, `Ctrl+B` to widen to whole-book with a review modal).
- `Ctrl+S` saves, but Inkhaven also autosaves on idle, on paragraph switch, and on focus loss, and mirrors your dirty buffer against a crash.
- `F4` splits the pane to edit against a frozen snapshot (`Ctrl+F4` accepts it, `Ctrl+H/Ctrl+J` scroll the lower half); `F7` grammar-checks and `g` applies the correction, with red change-highlights you dismiss via `Ctrl+B C`.

CHAPTER 5

5

The Tree and Its Files

The last chapter mapped the window; this one goes into the pane on its left and never really leaves it. The Tree is where a book **has a shape** — where its parts and chapters and paragraphs stand in order, where you add a scene and move it three chapters earlier, where you fold a finished act out of the way and open the one you are still fighting. It is also, quietly, the pane where Inkhaven stops being only a prose editor: the same outline that holds your chapters holds data paragraphs, templates, scripts, images, and a family of **structural** paragraphs for the code, tables, and admonitions a technical book needs. This chapter teaches the Tree as a builder's tool first — how to grow and reshape the outline, how to delete without fear — and then walks every kind of leaf that can hang from it, one at a time, in real depth.

Everything here happens with the Tree pane focused. Reach it with **Ctrl+T**, or **Tab** around to it; it is the pane that has focus when a project first opens. The plain-letter shortcuts in this chapter — **B**, **C**, **+**, **D**, **t**, **i**, **e**, and the rest — fire **only** while the Tree has focus, and they reject **Ctrl** and **Alt**, so **Ctrl+A** will never quietly add a subchapter. Every one of them also has a **Ctrl+B** chord equivalent that works from any pane; the bare letters exist because terminals and multiplexers so often eat the meta prefix.

What a tree row says

The Tree draws your project as an indented outline, one node per row, each row carrying more information than its width suggests. Read a row left to right and it tells you, in order: how deep it sits (by indentation), whether it is marked for a batch operation, what **kind** of node it is (a glyph), how far along its draft is (a status letter), its title, and then a train of small **pips** — a progress gauge, tag chips, and any findings a reader has posted against it. Paragraphs also carry a dim **Nw** word count at the end.

● ● ● *A tree, read top to bottom — glyphs, badges, and pips*

```

Tree
├─ Rain Over Quaymouth
│   └─ 1 Arrival
│       ├── the-quay      R ● #opening    214w
│       │   └─ the-inn    F ●             88w
│       │       └─ install-listing
│       └─ 2 The City      ▲2
│           └─ 3 Departure
│               ├── ledger-of-ships    hjson
│               │   └─ crew-sidebar    jinja
│               │       └─ the-parting  ⊗1
│           ├── Facts
│           ├── Notes
│           └─ Scripts
│               └─ λ on-save-guard      bund

```

TERM Node

A **node** is one entry in the tree — a book, a chapter, a subchapter, a paragraph, an image, or a script. Branches (book, chapter, subchapter) can hold children; leaves (paragraph, image, script) cannot. The whole book is a tree of nodes, and almost everything you do in this pane is adding a node, moving a node, changing a node's kind, or deleting one.

The kind glyph

The glyph in front of a title is the fastest read on the row — it tells you what the node **is** without opening it. Branches show a fold arrow: ▾ when expanded, ▸ when collapsed. Leaves show their nature. Prose is the plain ¶; the other leaves each earn a glyph of their own, and this chapter's second half is a tour of exactly what stands behind each one.

● ● ● *Every glyph you will meet in the Tree*

```

  ▾ / ▸   branch, expanded / collapsed
(book·chapter·subchapter)
  ▸       the paragraph currently open in the Editor (green,
bold)
  ¶       prose paragraph — the default leaf (.typ)
  {       HJSON data paragraph (.hjson)
  [?]     Jinja template paragraph (.jinja)
  λ       Bund script (.bund)
  ■       image (.png · .jpg · .webp · .svg)
  ◆       timeline-event paragraph
  ≡ △ ∫ ≡ ▤ structural: code·admonition·math·procedure·table
  || ∩ = ∴ ≠ verse:
line·stanza·couplet·tercet·quatrain·translation

```

The open paragraph is special: whichever leaf is loaded in the Editor is drawn with a green, bold ▸ in place of its usual glyph, and it keeps that marker whether or not the Tree has focus. This is your anchor — however far your tree cursor wanders, the ▸ always shows you which paragraph the Editor is holding. If the cursor happens to land on that same row, the reversed cursor highlight wins the foreground, but the green underneath still marks it.

The badges and pips

After the glyph comes a run of optional marks. None of them is noise; each is a readout you can act on.

- A **status letter** rides just before the title on paragraphs — the draft-status ladder as a single initial (N apkin, F irst, S econd, T hird, F inal, R eady). Cycle it up a rung with **o** in the Tree, or from the Editor. A paragraph with no status shows nothing here.
- A **target gauge** appears when a paragraph carries a word-count goal — a single filling pip (○ ◐ ◑ ◒ ◓) that greens as the draft approaches its target and turns bold green at 100%. Set or clear the target with **Ctrl+V t**.
- **Tag chips** show up to two of a paragraph's tags as dim **#tag** labels, with a **+N** when there are more. Manage tags with **g** in the Tree or **Ctrl+B]** in the Editor.
- A **report-card badge** appears when a reader has posted findings against a node — a worst-severity glyph and a count (⊗2 a contradiction, △3 a warning, ●1 an informational note). On a branch the badge is **aggregated** up from the paragraphs beneath it, so a collapsed chapter still tells you something inside it needs attention. These come from the review pass (**Ctrl+B Shift+C**) and the intelligences in Part V.
- A dim **Nw** word count closes every paragraph row.

Moving through the tree

Navigation is arrow-driven and quiet. **↑** and **↓** walk the cursor one visible row at a time; **Home** and **End** jump to the first and last rows; **PageUp** and **PageDown** move ten rows at once. **Enter** **opens** the cursor's node — a paragraph loads into the Editor and pulls focus there (autosaving whatever was open if it was dirty), an image pops a preview, and a branch simply prints a status hint and stays put, since a chapter is a container, not a document.

Folding — the four keys

A long book is unreadable in the Tree unless you can fold the parts you are not working in. Two motions expand and collapse a single branch; two more fold in bulk.

- **→** **expands** the branch under the cursor, revealing its children. It is a no-op on a paragraph or an already-open branch.

- **←** **collapses** an expanded branch. If the branch is already collapsed — or the cursor is on a leaf — **←** instead walks the cursor **up to the parent**, so pressing it repeatedly climbs you out of a deep subtree toward the root.
- **Z** folds the cursor's **enclosing subchapter** (or the cursor's node itself if it already is one) and lands the cursor on the folded row — the fast way to tuck away the section you just finished.
- **X** **collapses everything** — folds every expanded branch in the whole tree, leaving you the top-level outline to navigate from. Paragraphs and empty branches are left untouched.

THE CURSOR IS NOT THE OPEN PARAGRAPH

Two different things move in the Tree: the **cursor** (the reversed highlight you drive with the arrows) and the **open paragraph** (the green ►). Moving the cursor changes nothing on disk and opens nothing — it only chooses where the next action lands. You have to press **Enter** to actually open a node. This separation is what lets you reorder, tag, or delete a paragraph without first loading it into the Editor.

↑ / ↓	Move the cursor one visible row up / down.
→	Expand the cursor's branch.
←	Collapse the branch, or step up to the parent.
Home / End	Jump to the first / last row.
PageUp / Page-Down	Move the cursor ten rows.
Enter	Open the node — load a paragraph, preview an image.
Z	Collapse the cursor's enclosing subchapter.
X	Collapse every expanded branch in the tree.
Space	Mark / unmark the row for a batch operation.
Esc	Cycle focus onward to the Search bar.

Building the tree

A new project is a nearly empty tree — a book and the system books Inkhaven maintains for you. You grow it by adding nodes, and the Tree gives you a small, learnable vocabulary for doing so. Every add opens the same green **Add modal**: a floating box that names the parent it will slot into and takes a title. Type the title, press **Enter**, and Inkhaven derives a slug, writes the file to disk, inserts the record, reloads the tree, and moves the cursor onto the new node. **Esc** cancels; an empty title keeps the modal open with a hint (except for paragraphs, where an empty title is allowed — the first sentence of the body becomes the title on the next save).

● ● ● *The Add modal — every kind is created through this box*

```
└─ Add chapter ───────────┘  
| Parent: rain-over-quaymouth  
| Title : The Long Road South|  
  
| Enter to confirm · Esc to cancel
```

The four building blocks — and where they land

The everyday structure keys come in two flavours, and the difference is **where the new node lands among its siblings**. This is the single most useful thing to understand about building a tree.

- **Append at end.** **B** (book), **C** (chapter), **A** (subchapter), and **+** (paragraph) add the new node at the **end** of its parent's children. Inkhaven chooses the parent by walking up from your cursor to the nearest node that can legally host the kind you asked for — press **+** deep inside a subchapter and the paragraph appends to that subchapter; press **C** anywhere in a book and the chapter appends to the book.
- **Insert after current.** **V** (chapter), **S** (subchapter), and **P** (paragraph) add the new node **immediately after** the cursor's same-kind ancestor, shoving every later sibling down by one — the way you add a scene **between** two scenes rather than at the end of the act. If there is no same-kind ancestor to insert after, these fall back to append-at-end so the key still does something.

The mnemonic is the shape of the keyboard row: the append keys sit under your fingers as whole words (B ook, C hapter, A for subchapter, + for one more), and

the insert-after keys are the ones you reach for when order matters. Each has a **Ctrl+B** equivalent — **Ctrl+B B**, **Ctrl+B C**, **Ctrl+B S**, **Ctrl+B P** — that does the same thing from any pane.

● ● ● *Append versus insert-after, on the same chapter*

cursor on 'the-inn', press +
press P

```

┌ 1 Arrival ───────────┐
│ ¶ the-quay           │
│ ¶ the-inn   ←here    │
│ ¶ the-market         │
│ ¶ NEW      ←end      │
└────────────────────────┘

```

cursor on 'the-inn',

```

┌ 1 Arrival ───────────┐
│ ¶ the-quay           │
│ ¶ the-inn   ←here    │
│ ¶ NEW      ←after    │
│ ¶ the-market         │
└────────────────────────┘

```

BOOKS SLOT IN ABOVE THE SYSTEM BLOCK

A new user book (B) is inserted **above** the built-in system books — Notes, Research, Prompts, Places, Characters, Help, and the rest — which shift down to make room. Those system books are **protected**: Inkhaven will not let you rename or delete them, because features across the tool find them by tag. One context-sensitive exception is worth knowing: pressing **b** on the Language system book scaffolds a whole conlang sub-book under it, five chapters deep — a special case covered in the language chapters.

Renaming and the file picker

F2 opens the **Rename** modal, pre-filled with the current node's title. It changes only the displayed title and re-embeds the node for search; the slug and the on-disk filename stay put, so a rename never breaks a cross-reference or a snippet include. **F3** opens a **file picker** rooted at your shell's working directory: **Enter** on a file creates a paragraph after the cursor with that file's contents, and **Enter** on a **directory** recursively imports the whole tree — subfolders become branches, files become paragraphs, flattened to fit the hierarchy's depth. (An image file picked this way is routed to the image import path described later, not turned into prose.)

Reshaping the tree

Structure is never right the first time. The Tree gives you three ways to move a node that already exists: reorder it among its siblings, change its nesting level, and relocate it to a different parent entirely. All three reuse the same filesystem-aware store primitives, so the `.typ` files on disk are renamed and renumbered to match whatever you do — the tree and the folder never drift apart.

Reordering among siblings

U moves the cursor's node **up** — swapping it with its previous sibling — and **J** moves it **down**. These are the plain-letter forms of `Ctrl+B ↑` and `Ctrl+B ↓`; use the letters when your terminal swallows Ctrl with the arrow keys. Each swap renumbers the two nodes' filesystem entries, so the order you see is the order the book assembles in. The CLI mirror is `inkhaven mv ... up`.

Promote and demote — changing nesting level

Reordering keeps a node at its level; **promote** and **demote** change the level. These live in the full-screen Outline pane (`Ctrl+2`), the structural sibling of the side Tree, where `<` **promotes** a childless node one level out (appending it under its grand-parent) and `>` **demotes** it one level in (nesting it into the preceding sibling). Any move that would break a placement rule — a paragraph where a chapter belongs — leaves the manuscript untouched and tells you why. The Outline pane is also where you get a wide, foldable view of the whole book with a detail panel; it shares the Tree's clipboard, so the two stay in lock-step.

Copy and move across parents

To relocate a paragraph to an entirely different chapter, use the cross-pane clipboard. **y** **copies** the cursor paragraph onto it; **m** **moves** (cuts) it; then navigate to the destination and press **f** to **affix** the clipboard paragraph as the last child of the cursor's effective parent — into a branch when the cursor is a branch, alongside it when the cursor is a paragraph. A copy duplicates with a fresh UUID and keeps the clipboard loaded; a move relocates the original and clears the clipboard. The same `y / m / f` work in the Outline pane, and the CLI equivalent is `inkhaven paragraph copy|move <src> <dest>`.

- U · Ctrl+B ↑** Move the node up one place among its siblings.
- J · Ctrl+B ↓** Move the node down one place among its siblings.
- < / > (Outline)** Promote / demote a childless node one nesting level.

Deletion is the one action that can lose work, so Inkhaven wraps it in three layers of safety — a confirmation that tells you the cost, an immediate undo, and a durable snapshot for the large deletes. Understanding the layers is what lets you delete freely.

Delete is kind-specific on purpose. `-` deletes the cursor's node **only if it is a paragraph**; press it on a branch and you get a hint to use `D` instead. `D` deletes **only a branch** (book, chapter, subchapter); on a paragraph it points you back to `-`. The split is a guard rail: `-` will not nuke a whole chapter because your cursor drifted onto it, and `D` will not kill a paragraph you meant to keep. If you genuinely want a kind-blind delete, the global `Ctrl+B D` does it. Either way a red **Confirm delete** modal opens first.

The confirm names the kind, the title, the descendant count, and – the part that matters – **the word count you are about to lose**:

```

Confirm delete
Delete chapter `Act II` and 12 descendants
(15,342 words)?

Removes files from disk AND records from the
store.  y / Enter confirm · n / Esc cancel

```


A single paragraph reads `Delete paragraph the-quay (342 words)?`. Zero-word deletes — an empty paragraph, an HJSON or Jinja leaf — omit the count, since there is no prose to lose. `y`, `Y`, or `Enter` confirms; `n`, `N`, or `Esc` backs out. If the paragraph you had open was inside the deleted subtree, the Editor closes with it.

The kill-ring — immediate undo

Every deleted paragraph is stashed on a **kill-ring**, and a branch delete stashes **every paragraph leaf** under it, not just the one you were pointing at. `Ctrl+B U` restores the front of the ring — the most recent deletion — instantly; the status line reports the word count that came back. For a branch delete, repeated `Ctrl+B U` restores the paragraphs one at a time, in their original order.

TERM Kill-ring

A bounded stack (up to ten entries, capped by `editor.deleted_paragraph_history`) of recently deleted paragraphs. `Ctrl+B U` pops the front; `Ctrl+V Shift+U` opens a **picker** over the whole ring so you can choose which deletion to bring back. Restored paragraphs return at their original tree position but get a **fresh UUID**, so any cross-reference that pointed at the old node stays broken — Inkhaven flags this in the status rather than silently mis-linking.

● ● ● *Ctrl+V Shift+U — the kill-ring restore picker*

```

┌ Restore deleted paragraph ────────────┐
│ █ the-parting          Act III · 512w · 2m ago │
│   the-market          Act I   · 88w   · 14m ago │
│   ledger-of-ships     Act III · hjson · 20m ago │
├────────────────────────────────────────┤
│ ↑↓ select · Enter restore in place · Esc cancel │
└────────────────────────────────────────┘

```

Pre-delete snapshots — the long-term net

A very large branch can overflow a ten-slot ring, so before a **branch** delete Inkhaven takes an annotated snapshot of every paragraph leaf, labelled `pre-delete: <title> · <date>`. These live in the ordinary snapshot system, so you can find and restore any of them from the `F6` snapshot picker long after the kill-ring has cycled — even in a later session. The snapshots are taken **before**

the delete fires, so they survive even a delete that partially fails. Together the three layers mean a branch delete is fully recoverable: the confirmation tells you what is at stake, the kill-ring is instant undo, and the **F6** snapshots are the durable safety net. (Single-paragraph deletes skip the snapshot — it would only clutter the list, and the kill-ring already covers them.)

- Delete the cursor's node — paragraphs only.
- D · Ctrl+B D** Delete the cursor's node — branches (Ctrl+B D is kind-blind).
- Ctrl+B U** Restore the most recently deleted paragraph.
- Ctrl+V Shift+U** Open the kill-ring picker — choose which deletion to restore.
- F6** Snapshot picker — reach the pre-delete snapshots of a branch.

The leaf types

So far a leaf has mostly meant a prose paragraph. It need not. A leaf is any childless node, and Inkhaven ships six kinds of them, each stored as its own file on disk and each with a job. The rest of this chapter takes them one at a time. The through-line: **the manuscript is always plain files** — a prose paragraph is a **.typ**, a data paragraph a **.hjson**, a template a **.jinja**, a script a **.bund**, an image its own bytes — and the **content_type** field (or the node kind) is all that distinguishes them.

Prose — the .typ paragraph

The default leaf, and the one you will make thousands of, is a **prose paragraph**: a Typst **.typ** file holding the words of your book. It shows the ¶ glyph, counts toward your word totals, and is the surface every prose intelligence reads — the Inner Editor, the continuity watch, the fact-checker, the read-through. There is nothing to configure; **+** or **P** makes one, and you write. Everything the next chapter says about the Editor is about editing these. The three other **text** leaves below are all, on disk, close cousins — the same kind of node with a different **content_type** — and you convert between them with a single morph key covered at the end of the chapter.

HJSON — the data paragraph

Some content is **data**, not prose: a character's dossier, a ship's manifest, a glossary entry, a table of API fields. For these Inkhaven offers the **HJSON data paragraph**

— a paragraph whose `content_type` is `"hjson"`, stored as a `.hjson` file and shown with the `{` glyph. HJSON is JSON for humans: unquoted keys, comments, trailing commas forgiven. The Editor switches to an HJSON highlighter and header badge, and the prose intelligences stand down — a data blob is not prose, so it draws no style notes and does not inflate the word count.

● ● ● *An HJSON data paragraph — structured, not prose*

```

Editor · 01-aria [hjson]
1 {
2   name:    "Aria"
3   species: "fox"           // unquoted keys, //
4   role:    "scout"        // comments, no fuss
5   age:     19
6 }
```

Data paragraphs are useful on their own — the Characters, Places, and Language system books store their entries as HJSON — but they come into their own as the **input to a template**. That is the next leaf.

Jinja — the template paragraph

A **Jinja template paragraph** is a paragraph whose `content_type` is `"jinja"`, stored as `.jinja`, shown with the `?` glyph, and marked `[jinja]` in the Editor header. It is a Jinja2-style template — rendered by `minijinja` — that the assembler **compiles to Typst** before Typst compiles anything to PDF. The two layers are strictly sequential, never nested:

● ● ● *Two layers, one direction — Jinja out, then Typst*

```

your .jinja      assembly      typst
paragraph  → (minijinja) → .typ → PDF
              renders          compiles
```

Templates exist so that **structured** content stays in one place instead of being copy-pasted and left to drift: a character sidebar that reads a linked dossier, an endpoint table that loops over fields, a shared admonition written once and pulled into a

dozen chapters. The feature is **self-gating** — a project with no Jinja paragraphs assembles exactly as before, and there is no flag to turn on.

The render context. Every template renders with a fixed set of variables: its own `title` and `slug`; the enclosing `book.title` / `book.slug` / `book.genre` and `chapter.title` / `chapter.slug`; the project `language` and `genre`; and, most importantly, `linked[...]`.

TERM **linked**

The data-injection mechanism. Link an **HJSON** paragraph to a template with the ordinary add-link chord `Ctrl+V a`; at assembly its parsed data is exposed under `linked["<that paragraph's slug>"]`. Only HJSON-bodied links land here — a linked prose paragraph is skipped, because its raw Typst is not meaningful template data. This is how a template reads facts instead of repeating them.

● ● ● *A template reading a linked HJSON dossier*

```
= {{ linked["01-aria"].name }}

#block[
  A {{ linked["01-aria"].species }} who
  serves as {{ linked["01-aria"].role }}.
]

{% if language == "ru" %}Глава{% else %}Chapter{% endif %}
```

Reusable fragments — the Snippets book. A `.jinja` paragraph placed in the built-in **Snippets** system book is registered as a named template before any rendering, so manuscript templates can pull it in with `{% include "snippets/<path>.jinja" %}`. The include name is the snippet's tree path, lowercased — chapter and subchapter titles become path segments, the paragraph slug is the filename — so you read the name straight off the tree.

Assembly order. At `Ctrl+B A` (or `inkhaven build`) the assembler first registers every Snippets template, then renders the snippets to `.typ`, then renders each manuscript template to a `.typ` in the generated tree, then runs `typst compile`. Because registration precedes rendering, every `{% include %}` resolves in one

pass — no nesting, no two-pass surprises. By default a render failure — bad syntax, a missing include, a typo'd variable — **aborts the whole assembly** with the offending paragraph named, so a broken template can never silently drop content from the PDF; set `jinja.continue_on_error: true` to write a visible error block into that paragraph's place and keep going while you fix templates one at a time.

TWO KINDS OF "SNIPPET" — DO NOT CONFUSE THEM

A Jinja template paragraph generates structured Typst **at assembly**. The edit-time text-expansion snippets (the `bund:` expansions that fire **while you type**) are a different system entirely. Both coexist; a Jinja paragraph is not prose and is skipped by the Inner Editor, Inner Socrates, and the idle fact-checker.

Bund — the script paragraph

A **Bund script** is a first-class leaf — a distinct node kind, not a paragraph flavour — stored on disk as a `.bund` file and shown with the `λ` glyph. Its natural home is the built-in **Scripts** system book, whose `.bund` files are `eval`'d into Inkhaven's embedded Bund virtual machine when the project opens. That is where user-authored hook lambdas live — `hook.on_save` and its kin — so a script under Scripts can watch your saves, enforce a house rule, or extend the tool. Bund is Inkhaven's embedded scripting language; Part VIII is its full treatment. For the Tree's purposes what matters is that a script is a real node you can add, move, and delete like any other, and that you **reach** the bund rung by morphing a paragraph — the last stop on the type cycle described below.

Images — the picture leaf

Images are first-class nodes too — `NodeKind::Image`, shown with the `■` glyph, holding their own bytes on disk (`.png`, `.jpg` / `.jpeg`, `.webp`, `.svg`, `.gif`). You add one by importing it: the `F3` file picker routes any image file to the image-import path instead of treating it as prose, dropping a new Image node into the tree near your cursor. An Image node carries an optional caption and alt-text; at book assembly Inkhaven emits the right `wrap_image_*` calls and ships the bytes into the generated Typst tree, so the picture lands in the PDF with its caption and accessible alt-text intact.

Two conveniences ride on images. `Enter` on an Image row pops a **preview** — a real in-terminal rendering through `ratatui-image`, using your terminal's best protocol (kitty, sixel, iTerm2, or a half-block fallback). And from **inside** the Editor,

with the cursor in a Typst `#image("...")` call, **Ctrl+B P** opens a sibling-image picker so you can fill the path from the images already in the tree rather than typing it.

• • • *Enter on an image row — an in-terminal preview*

Image · harbour-map.png · 1240×860 · 214 KB



(rendered inline via kitty /
sixel / iterm2 / half-block)

caption: "The harbour at Quaymouth"
Esc close

Structural paragraphs — the `para:*` family

A technical or nonfiction chapter is not all prose: it has code listings, warning boxes, display math, numbered procedures, tables. Left as ordinary paragraphs these would all read as ¶, draw prose-style craft notes that make no sense on a code block, and inflate the word count with text that is not narrative. **Structural paragraphs** fix this. A structural paragraph is an **ordinary** `.typ` paragraph carrying a `para:*` tag — the tag, not a new content type, is the whole mechanism.

• • • *The structural subtypes and their glyphs*

<code>para:code</code>	▢	code listing
<code>para:admonition-note</code>	⚠	note box
<code>para:admonition-warning</code>	⚠	warning box
<code>para:admonition-tip</code>	⚠	tip box
<code>para:admonition-caution</code>	⚠	caution box
<code>para:math</code>	∫	display math
<code>para:procedure</code>	≡	numbered steps
<code>para:table</code>	▢	table

Marking a paragraph structural does three things: it swaps the tree glyph for the subtype's icon, it **excuses the paragraph from the prose companions** (the Inner Editor and Inner Socrates no longer fire on it — it is not prose), and it **removes it**

from the prose word count, counting it separately (Book Info, `Ctrl+B I`, shows a `structural: N` line). One subtype breaks the pattern: `para:procedure` is prose the author writes — real steps — so the companions **do** still run on it; it is only excluded from the word count.

Create one — i in the Tree. Press `i` and a picker of the subtypes opens. Choose one, `Enter`, name it, and you get a `.typ` paragraph tagged `para:*`, its glyph set, and **matching Typst boilerplate seeded** — a `#figure` code block, a coloured `#block` admonition, a `#table`, a `$ ... $` math block, or a `+`-stepped procedure — so you start from a working skeleton rather than a blank buffer.

● ● ● *The i picker — pick a structural subtype*

```
└─ Add structural paragraph · i ───
|   ▣ code listing
|   △ admonition: note / warning / tip / caution
|   ∫ math
|   ≡ procedure
|   ▤ table
|   ↕ select · Enter create · Esc cancel
```

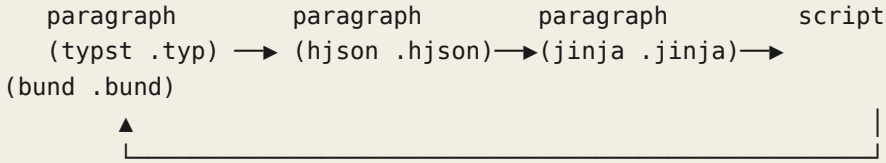
TAG, NOT TYPE — THE ESCAPE HATCH

Because a structural subtype is only a tag, you manage it like any tag. Add `para:code` to an existing prose paragraph with `Ctrl+B ]` to make it structural, or remove the tag to turn it back into prose — the body is untouched either way. There is no morph cycle to walk; the tag **is** the mechanism. (Poetry rides the same machinery: a stanza is a paragraph tagged `para:verse-*`, a sixth family with its own glyphs `|| ↓ = ∴ ⌘` and its own reader, the Inner Poet — the Poetry companion has the full treatment.)

The morph cycle — turning one leaf into another

The four **text** leaves — prose, HJSON, Jinja, and Bund — are close relatives on disk, so Inkhaven lets you cycle a leaf from one to the next in place. `t` (or `T`) **morphs the node type** through a fixed loop:

● ● ● *t / T cycles a leaf through four types*



With no rows marked, **t** cycles just the cursor's leaf (branches are skipped); with a set of paragraphs marked (**Space**), it cycles every marked leaf at once. The morph **only flips the type and renames the file's extension** — it does not seed a starter body. That is the practical difference from the two Tree **pickers**, which create a **fresh, seeded** leaf:

- **i** creates a new **structural** paragraph from the subtype picker, with boilerplate seeded (covered above).
- **e** creates a new **Jinja** template — seeded with a documented starter that lists the variables available to you — under a user book, or a reusable `{% include %}` fragment under the Snippets book. It is rejected elsewhere (Notes, Characters, and the other system books do not take templates).

So the rule of thumb is simple: reach for **e** (or **i**) when you want a **new leaf with a working skeleton**; reach for **t** when you have an **existing** leaf whose type you want to change and you will write the body yourself. To land on the Bund rung there is no seeded creator — you morph a paragraph around the cycle to **script**, or add a **.bund** under the Scripts book and let the tree hold it.

t / T	Morph the leaf's type: typst → hjson → jinja → bund (marked set, or the cursor).
i	Create a new structural paragraph (para:*) from the subtype picker, boilerplate seeded.
e	Create a new Jinja template ([?]), seeded — a manuscript template, or a Snippets fragment.
Ctrl+B]	Add / remove a tag — including a para:* structural tag — on the open paragraph.
g	Tag the marked set (or the cursor row) from the Tree.

WHAT YOU LEARNED

- The **Tree** is where the book has a shape. A row reads left to right — depth, mark, **kind glyph**, status letter, title, then **pips**: a target gauge, tag chips, a finding badge, and a dim word count. The open paragraph always wears a green ►.
- Navigate with the arrows; fold with → / ← (single branch), Z (enclosing subchapter), and X (everything). The **cursor** and the **open paragraph** are two different things — moving the cursor opens nothing until you press Enter.
- Build with B / C / A / + to **append at end** and V / S / P to **insert after** the cursor's same-kind ancestor; each has a Ctrl+B twin. New books slot above the protected system block.
- Reshape with U / J (reorder siblings), < / > in the Outline (promote / demote a level), and y / m / f (copy / move / affix across parents). The files on disk follow every move.
- Delete kind-specifically — - a paragraph, D a branch — behind a confirm that **counts the words at risk**. Recover with the **kill-ring** (Ctrl+B U, or the Ctrl+V Shift+U picker) and, for branches, the **pre-delete snapshots** in F6.
- Six leaf kinds: prose .typ (¶), HJSON data .hjson ({}), Jinja templates .jinja (?), rendered to Typst at assembly, fed by linked HJSON), Bund scripts .bund (λ), images (■, previewed with Enter), and the para:* **structural** subtypes (▣ ▴ ∫ ≡ ▢) — the last a tag, not a type.
- Turn one text leaf into another with the morph key t (typst → hjson → jinja → bund); use the **pickers** i and e when you want a fresh, **seeded** leaf instead.

CHAPTER 6

6

Snapshots and History

The last chapter taught you to write into a paragraph; this one teaches you to write **without fear**. Every long book is a graveyard of sentences the author was glad to lose and a handful the author would give a finger to have back, and the difference is never obvious in the moment you cut them. Inkhaven's answer is to make going back cheap and reliable at three different grains: a versioned history **inside** each paragraph, a project-wide index over **all** of it, and a recovery layer beneath both that survives the tool crashing under you. Learn this chapter and the delete key stops being a decision — it becomes a gesture you can always take back.

What a snapshot is

A snapshot is the smallest unit of Inkhaven's memory: a frozen, dated copy of one paragraph's prose, kept beside the paragraph but never confused with it. When you take a snapshot the current bytes of the open buffer are written into the project database as a document of their own, tagged with the paragraph they came from, the moment they were taken, a word count, a one-line preview, and an optional note. The paragraph goes on changing; the snapshot does not. It is a photograph, not a mirror.

TERM Snapshot

A **snapshot** is an independent, dated copy of a single paragraph's body, stored in the project database as its own document (`kind: "snapshot"`) with a `parent_id` back-reference to the paragraph it was cut from. It is not the live file, not an autosave, and not part of the search index — it is a point-in-time bookmark you can read, compare, and restore.

Three properties follow from that independence, and each one matters when you need a snapshot most.

First, **snapshots outlive saves**. Autosave and `Ctrl+S` overwrite the paragraph in place — they commit the **latest** version and keep no history. A snapshot is a separate document, so saving the paragraph a thousand times never touches it.

Second, **snapshots outlive the paragraph**. Because a snapshot is not stored inside the paragraph, deleting the paragraph does not delete its snapshots; they remain in the database, findable, as a recovery hatch when a whole branch goes. This is the seam that connects the two halves of this chapter — the versioning layer and the deletion layer meet here.

Third, **snapshots are never collected**. Inkhaven does not garbage-collect them, expire them, or cap them. A paragraph you have revised for a year carries its whole revision history, oldest to newest, until you delete a row by hand. They cost only disk, and disk at literary scale is free.

NOT IN SEARCH

Snapshots are deliberately excluded from the vector index — they carry no embedding and never surface in semantic search or the AI’s grounding. A history of forty drafts of one paragraph would otherwise drown the one live version in every “find similar” result. Snapshots are a versioning surface, not a corpus.

When snapshots are taken

You take snapshots two ways: deliberately, with a keystroke, and automatically, whenever Inkhaven is about to do something to your prose that you might regret. The second kind is the one that saves manuscripts, because it fires exactly at the moments you are least likely to think of it yourself.

The **manual** trigger is a single key, covered in the next section. The **automatic** triggers all share one rule — **nothing rewrites your prose without a restorable copy landing first** — and there are five of them:

- **Before an AI rewrite is applied.** When you accept a diff from any AI rewrite — the sentence-rhythm rewrite, an Editorial Pass fix, a reconciliation — the pre-rewrite paragraph is snapshotted before the new text replaces it, under an annotation naming the rewrite. Reject the diff and nothing is written at all.
- **Before a project-wide find-and-replace.** Each paragraph a book-scope replace touches is snapshotted first, annotated `replace: X → Y`, so **F6** is the undo for a substitution that went wider than you meant.
- **Before a branch is deleted.** Deleting a chapter, subchapter, or book snapshots **every paragraph leaf** inside it first, annotated `pre-delete`, so any single paragraph from a vanished branch can be found and restored even though the branch itself is gone for good.
- **Before a snapshot restore.** Loading an old snapshot over the live buffer first snapshotts the live buffer — the **pre-restore safety net**, described below. Restoring is itself a restorable act.
- **On demand from a Bund script.** The `hook.on_snapshot` hook fires on every snapshot however it was taken, so a script can mirror snapshots to git or stamp them with the paragraph’s workflow status.

THE THROUGH-LINE

Read the list once more and notice what it guarantees: there is **no path** in Inkhaven that overwrites or destroys prose without a dated, restorable copy existing first. Manual **F5** is the belt; the five automatic triggers are the braces. You never have to remember to protect yourself before a risky edit — the tool has already done it.

Taking a snapshot — F5 and the annotation prompt

Press **F5** (or its chord form, **Ctrl+B N**, for **new** snapshot) with a paragraph open. Rather than snapshot silently, Inkhaven floats a one-line **annotation prompt** — a chance to label this version with a note you will thank yourself for later.

● ● ● *F5 — the annotation prompt over the editor*

```

— Snapshot annotation · F5 —
|
| Snapshot `the-quay` — annotation:
| > before the lighthouse rewrite
|
| Enter commits (empty = no note) · Esc cancels
|

```

The prompt is forgiving in both directions. Type a note and press **Enter** to commit a **labelled** snapshot; press **Enter** on an empty line to commit an **unlabelled** one — the old reflex of “F5, Enter” still fires as fast as it ever did. Press **Esc** to cancel, and **no snapshot is written at all**.

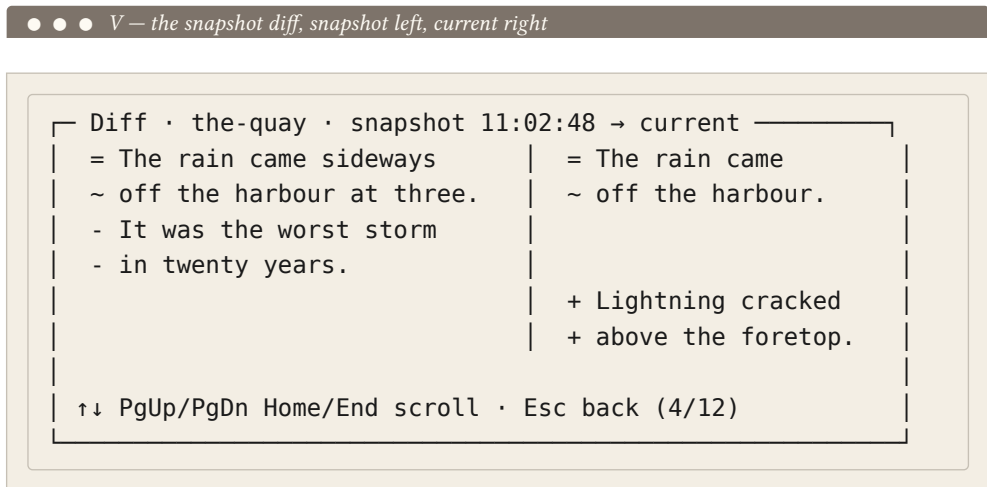
One subtlety earns its keep: the buffer to be snapshotted is captured the **instant the prompt opens**, not when you press **Enter**. You can keep typing in the editor behind the prompt — or take your time composing the note — without changing what gets saved. The annotation lives in the snapshot’s metadata, never in its body, so it never contaminates the prose you are versioning.

Home / End	Jump to the newest / oldest snapshot.
Enter	Load the selected snapshot into the buffer (safety-snapshotting the live buffer first).
V	Side-by-side diff of the snapshot against the live buffer.
D / Del	Delete the selected snapshot.
/	Filter the list by annotation text; Esc leaves the filter, keeping the query.
Shift+Enter	Pin the snapshot into the split-view secondary pane, read-only.
Esc	Close the picker.

The **/** filter narrows the visible rows to those whose annotation contains what you type — a case-insensitive substring match. It is filter-focus: typed characters build the query, **Backspace** edits it, and **Esc** exits the filter back to row navigation while keeping the narrowed list. The filter resets each time you open **F6**, so a previous session's query never haunts the next picker.

Viewing a diff — V

Reading a raw old version tells you little; seeing **what changed** tells you everything. Press **V** on a row and Inkhaven opens a side-by-side diff — the snapshot on the left, the paragraph's current text on the right.



The diff is computed with the Myers algorithm and sorted into four colour buckets: unchanged lines are dim, lines **removed** since the snapshot are red, lines **added**

since are green, and a one-line rewrite — a delete and an insert that sit adjacent — is **fused** into a single yellow “changed” row rather than shown as red-then-green. The result reads the way you think about an edit.

Esc in the diff returns you to the picker, **not** all the way out — the picker is stashed underneath so you can scan a row, diff it, scan the next, diff that, without ever losing your place in the list. One more **Esc** closes the picker.

Restoring an old version — Enter

Press **Enter** on a row to load that snapshot’s bytes into the editor. The buffer is replaced and marked **dirty** — nothing is committed to disk until you save, so a restore is a proposal you can still walk away from. The status bar reports both the load and the safety snapshot it took first.

Because every restore leaves the live buffer dirty rather than saved, and because every restore first snapshots what **was** there, you can flip back and forth between “what the snapshot has” and “what was in the editor” indefinitely, comparing them in place, until one of them is the one you save.

The pre-restore safety net

The single most important thing **F6** does happens where you cannot see it. When you press **Enter** to restore, Inkhaven first takes a fresh snapshot of whatever is currently in the editor — **then** replaces it. Without this, restoring an old version would silently discard any unsaved typing. With it, the state you were about to lose becomes one more row at the top of the history.

SAFETY BEFORE RESTORE, ALWAYS

If that pre-restore snapshot cannot be written — the disk is full, the store is offline — the restore **aborts entirely** and the buffer is left untouched. The whole point of the net is data safety, so Inkhaven would rather do nothing than do the replace without the copy. Fix the underlying cause and try again.

This is also the undo for a restore you did not mean. Pressed **Enter** on the wrong row? Press **F6** again: the safety snapshot of your real work is now the top row, newest timestamp. **Enter** on it and you are back where you started — and **that** restore made its own safety snapshot, so the chain never breaks.

Deleting a snapshot — D

D (or **Del**) removes the selected snapshot from the history. There is no confirmation dialog: snapshots are explicit creations, the list regenerates fresh after a delete,

and the safety-snapshot chain from any recent restore has you covered if you prune the wrong one. Use it to keep a decision-heavy paragraph's history readable — but you rarely need to, since nothing forces you to look at the old rows.

The project-wide snapshot browser — Ctrl+F6

The **F6** picker answers “what has this paragraph been?” The browser answers a different question — “what was I touching last Tuesday?” — because it shows every snapshot in the **whole project** at once, one flat list, newest first, regardless of which paragraph each came from. Press **Ctrl+F6** from **any** pane; you need no paragraph open and no particular place in the tree.

- ● ● *Ctrl+F6* — every snapshot in the project, newest first

Snapshots · all paragraphs				
2026-06-19 14:23	421w	the-quay	The storm scene	
2026-06-19 11:02	388w	the-quay	Earlier draft	
2026-06-18 16:40	205w	harbour-mist	Opening sketch	
2026-06-17 22:05	140w	chapter-head	Working title	
↑↓ move · / filter · V diff vs current · Enter open ¶				
Esc close				

Each row is **timestamp · words · paragraph · annotation**. Because the list is flat and sorted purely by time, one paragraph's history interleaves with every other's — which is exactly what you want when you are hunting by **when** you did something rather than **where**. The browser is the same engine as **F6**, widened; it is how you find the row, and it hands the acting-on-it back to **F6**:

↑ ↓ Move through the flat, project-wide list.

/ Filter by paragraph title OR annotation; Enter or Esc leaves the filter, a second Esc closes.

V	Diff the selected snapshot against its OWN paragraph's current text; Esc returns to the browser.
---	---

Enter	Open that snapshot's paragraph in the editor and drop straight into its F6 picker.
Esc	Close the browser.

Two of these repay a second look. The **V** diff resolves the snapshot's own paragraph and diffs against **that** paragraph's live text — even though you opened the browser from somewhere else entirely, the comparison is always apples-to-apples. And **Enter** is a hand-off, not an action: the browser deliberately **does not restore or delete** from the project-wide view, because those are destructive, paragraph-local acts. Instead it opens the owning paragraph and drops you into its **F6** picker, where the pre-restore safety net is already wired up and you can act with full local context.

The browser lists itself in the command palette (**Ctrl+V Space**) under “snapshot”, so you can reach it by name without remembering **Ctrl+F6**.

Deleted-paragraph history — the kill-ring

Snapshots version a paragraph that still exists. A different problem is the paragraph you **deleted** — cut from the Tree in a fit of restructuring, then wanted back. For this Inkhaven keeps a **kill-ring**: a short, ordered stash of the most recently deleted paragraphs, whole and restorable.

TERM Kill-ring

The **kill-ring** is a session-local stash of your most recently deleted paragraphs — up to `editor.deleted_paragraph_history` of them (default 10), oldest rolling off as new deletes arrive. Each entry captures the paragraph whole, enough to re-create it in place. It lives in memory only and is not written to `.session.json`, so restarting Inkhaven clears it.

When you delete a single paragraph from the Tree (**Ctrl+D**, then confirm), it is pushed onto the front of the ring, and the status bar tells you both that it is gone and that **Ctrl+B U** will bring it back. What comes back is not just the prose but the whole paragraph: its title and slug (so the file lands at the same path), its body, its tags, its workflow status, its per-paragraph word target, its content type, its outgoing paragraph links, and any timeline event data.

A NEW IDENTITY

A restored paragraph gets a **fresh** uuid. It returns to the same place with the same words, but paragraph links elsewhere that pointed at the **old** uuid stay broken — the status line is upfront about this. If preserving identity matters (an incoming link you need to keep live), reach instead for the branch-level snapshot restore, which keeps the uuid.

Restore the most recent – Ctrl+B U

Ctrl+B U restores the paragraph at the **front** of the ring — the one you just deleted — without opening anything. It re-creates the paragraph at its original position: back in its old slot if a sibling anchor survives, or at the end of the (now shorter) child list if it had been the first child. This is the one-key “undo that delete” you reach for the instant you realise you cut the wrong row. Press it with an empty ring and it is a harmless no-op — the status reads **nothing to restore**.

The restore picker – Ctrl+V Shift+U

Ctrl+B U only ever gives you back the **latest** delete. When the one you want is further down the ring — you deleted three paragraphs, then wanted the first of them — open the **restore picker** with **Ctrl+V Shift+U**. It lists the ring's entries, most recent first, each with its word count and where it came from, and **Enter** restores the highlighted one at its original position.

● ● ● *Ctrl+V Shift+U — pick which deleted paragraph to restore*

```

└─ Recently deleted paragraphs ───────────────────────────────────────────
└─ > morning          128w  Rillmark ▶ 2 The City
└─   the-ledger       64w  Rillmark ▶ 3 Departure
└─   a-false-start    212w  Rillmark ▶ 1 Arrival
└─
└─ ↑↓ move · Enter restore at original position · Esc

```

The word count on each row is the quiet detail that makes the picker usable — when three deleted paragraphs share a forgettable slug, the size is how you recognise the two hundred words you actually want among the throwaways.

What the ring does and does not cover

The kill-ring is for **single-paragraph** deletes only. Deleting a whole branch — a chapter, a subchapter, a book — is a larger and more consequential act that the ring does not try to undo, and a branch delete no longer clears the ring either, so older single-paragraph entries stay valid recoveries after one. What protects a deleted branch is the **other** half of this chapter: the automatic **pre-delete** snapshot of every paragraph in it, which you recover through the **Ctrl+F6** browser and the **F6** picker. The two systems divide the work — the kill-ring for the everyday single cut you want back in one keystroke, snapshots for the durable, branch-scale safety net.

THREE GRAINS OF UNDO

Keep the layers straight and you will always reach for the right one. A **typo** is **Ctrl+U** in the editor. A **deleted paragraph** is the kill-ring (**Ctrl+B U** / **Ctrl+V Shift+U**). A **deleted chapter** — or any older version of living prose — is a snapshot (**F6** / **Ctrl+F6**). The zip backup, from the health chapter, is the grain beneath all three.

The crash mirror and recovery

Everything so far assumes Inkhaven exits cleanly. The last layer assumes it does not — that the process panics, the host loses power, or something sends a **kill -9** between two keystrokes. The promise here is concrete: your dirty buffers survive it, and the next run picks them back up. You should never need this section; when you do, it should feel like a non-event.

The crash mirror

While you edit, a background task quietly mirrors every **dirty** buffer to disk on a short cadence — `editor.crash_mirror_seconds`, two seconds by default. This is not autosave and it is not a snapshot; it is a side copy that exists so a sudden death loses as little typing as possible. A lower value loses fewer keystrokes at the cost of more disk churn; `0` mirrors on every tick. The worst case is narrow: if the power fails between two mirror ticks, only the last couple of seconds of typing can be lost, and everything older is already on disk.

What a panic writes

If Inkhaven panics, its handler writes two kinds of artefact before it goes down, both atomically (temp file, fsync, rename, directory fsync), and both silent on failure — the tool would rather get partial state out than panic inside its own panic handler:

- **A crash report**, `inkhaven-crash-<UTC>.hjson`, in the launch directory: the panic message and location, the project root and the paragraph open at the time, a manifest of the rescued buffers, the last fifty actions you took, and a small environment fingerprint (`$TERM`, `$LANG`, OS, version).
- **A rescue file** per dirty paragraph, `<paragraph>.typ.inkhaven-rescue`, beside the real file — the in-memory buffer flushed to disk. These are the actual prose; the report is only the index over them.

The report is deliberately conservative about what it records. It does **not** include your LLM prompts or responses, your search queries, snapshot bodies, the project database, your full config, or any environment variable beyond `$TERM` and `$LANG`. You can read it yourself before attaching it to a bug report.

Running inkhaven recover

After a crash, restart Inkhaven on the project, then run `inkhaven recover` from a shell, pointing it at the crash report. It walks each rescued buffer, showing its size and the delta from what is currently on disk, and asks what to do.

● ● ● *inkhaven recover — the per-buffer walk*

```
$ inkhaven recover ./inkhaven-crash-20260805T0321.hjson

inkhaven version : 3.0.0
panic at         : src/foo.rs:42:7
message          : called Option::unwrap() on None

Found 3 rescued buffer(s).

[1/3] manuscript/01-arrival/the-quay.typ
      rescue : .../the-quay.typ.inkhaven-rescue (4823 b)
      on-disk: .../the-quay.typ (4801 b, delta +22)
      apply? [y/N/diff]:
```

At each prompt, `y` applies the rescue — atomically replacing the on-disk file, but first copying the current on-disk contents to `<original>.pre-recover-<UTC>`

so you can roll the decision back later with a `mv . N` (or a bare `Enter`) skips the buffer. `d` or `diff` shows a side-by-side line diff of rescue against on-disk, then re-prompts. When the two are byte-identical — you saved just before the crash and the rescue is redundant — the walk skips the prompt entirely and reports “no action needed”, making no backup because there is nothing to back up. After the walk, the report and its rescues are moved into `<project>/ .inkhaven/recovered/` so a second run does not re-apply the same set; pass `--keep` to leave them in place. For scripted cleanup, `--yes` applies every rescue without prompting, and a path-traversal guard rejects any rescue entry whose path would escape the project root — a crafted crash report cannot be used to write arbitrary files.

THE BACKSTOPS BENEATH RECOVERY

If a `kill -9` lands inside the mirror window, the last couple of seconds may not be in the rescue. The established backstops still hold under it: `Ctrl+S` saves the open paragraph immediately and atomically; `editor.autosave_seconds` flushes dirty buffers after an idle window; and `inkhaven backup` (or `Ctrl+B Shift+B`) writes the whole project to a time-stamped `.zip`, surfacing an overdue warning in the health chip. Recovery is the floor, not the only net.

The configuration knobs

Four settings in the `editor` block of `inkhaven.hjson` govern the machinery in this chapter. The defaults are chosen so you never have to touch them, but they are here when your workflow wants otherwise.

● ● ● *The history and recovery knobs, with their defaults*

```

editor: {
  crash_mirror_seconds:      2      // dirty-buffer mirror
cadence
  deleted_paragraph_history: 10      // kill-ring depth
  external_change_auto_reload: true // silent reload of clean
files
  autosave_seconds:        5      // idle autosave window
}

```

`crash_mirror_seconds` sets how often the crash mirror flushes dirty buffers — lower narrows the worst-case loss window at the cost of more writes; `0` mirrors every tick. `deleted_paragraph_history` sets how many deleted paragraphs the kill-ring holds before the oldest rolls off. `external_change_auto_reload` decides what happens when a **clean** open file changes on disk under you — `true` reloads it silently, `false` warns and leaves your view untouched, which is the setting you want when an external `git pull` or a script rewrites files while you work. `autosave_seconds` is the idle window before an unsaved buffer is written for you; `0` disables idle autosave, leaving the quit-time and paragraph-switch autosaves in force. Together they tune how eagerly Inkhaven protects your words against how much it writes to your disk to do it.

WHAT YOU LEARNED

- A **snapshot** is a dated, independent copy of one paragraph's prose — separate from the live file, excluded from search, and never garbage-collected. It **outlives saves and even the paragraph's own deletion**.
- Snapshots are taken manually with **F5** (the annotation prompt; **Ctrl+B N**), and **automatically** before every risky op — AI rewrites, book-scope replace, branch deletes, and each restore — so **no path overwrites prose without a restorable copy landing first**.
- **F6** (**Ctrl+B R**) is the per-paragraph picker: **Enter** restores (behind a **pre-restore safety snapshot**), **V** diffs against the live buffer, **D** deletes, **/** filters by annotation. **Ctrl+F6** is the project-wide browser over every snapshot, which hands acting-on-a-row back to **F6**.
- The **kill-ring** restores deleted paragraphs — **Ctrl+B U** brings back the most recent, **Ctrl+V Shift+U** opens a word-counted picker over the last `deleted_paragraph_history` (default 10). It is single-paragraph and session-local; deleted **branches** are recovered through their **pre-delete** snapshots instead.
- The **crash mirror** copies dirty buffers to disk every `crash_mirror_seconds` (default 2); a panic writes a crash report plus per-paragraph `.inkhaven-rescue` files, and **inkhaven recover** walks them back with per-buffer `y / N / diff`, `.pre-recover` backups, and a path-traversal guard.

CHAPTER 7

7

Search

There are two questions a writer asks a long manuscript, and they are not the same question. The first is “**where did I write that exact phrase?**” — you remember the words, you just cannot find the paragraph. The second is harder and more interesting: “**where did I write the scene where the light goes out and she loses her way?**” — you remember the **feeling** of the passage, the shape of what happened, but not one word of how you actually put it. A tool that can only answer the first question makes you the index. A tool that can answer the second reads with you.

Inkhaven answers both, but it leans hard on the second, because that is the one worth building. This chapter is the whole of finding things: the two kinds of search

and what each is for; the Search bar you type a query into and the ranked results it drops down; the machinery underneath that lets a search for words you never wrote land on the paragraph you meant; searching inside a single part of the book; keeping the index honest; and reaching the same search from a shell or a script. By the end you will treat “where is that passage?” as a keystroke, not a hunt.

Two kinds of search

The word **search** covers two genuinely different operations, and knowing which one you want is most of using them well.

TERM Full-text (literal) search

Literal search looks for a run of characters exactly as you typed them — the string `lantern`, the phrase `she counted the lamps`. It is precise and unforgiving: it finds every occurrence of those characters and nothing else, and it misses a paragraph that means the same thing in different words. This is the `Ctrl+F` find inside the open paragraph, and `rg` (ripgrep) or `grep` over the `.typ` files on disk from a shell.

TERM Semantic (meaning-based) search

Semantic search looks for passages whose **meaning** is near your query, whether or not they share any words with it. It represents both the query and every paragraph as points in a “meaning space” and returns the paragraphs that sit closest to the query. This is the project-wide Search bar, and it is the one this chapter is mostly about.

Hold the division in mind, because it draws a clean line down the middle of the tool. Literal search is **inside** the text you are looking at (the open buffer) or **outside** it entirely (the files on disk, via your shell). Inkhaven’s **project-wide** search — the one that reaches every paragraph, note, fact, place, and character at once — is **semantic only**. There is no inverted full-text index over the whole project, by design: the meaning-based index is the one that earns its keep for a book, and a second literal index would be a large thing to maintain for a job `Ctrl+F` and `rg` already do well.

WHEN TO REACH FOR WHICH

Want a **specific word or spelling** — a character's name, a coined term, a phrase you are hunting to replace? Use `Ctrl+F` in the editor (it does regex on the open paragraph) or `rg <pattern> books/` from a shell. Want **the passage about a thing**, remembered by sense rather than wording? Use the Search bar. The rest of this chapter is the Search bar.

The classic case — the lantern that never appears

Here is the example that shows why semantic search is worth the machinery. Suppose, three hundred pages into a draft, you want the scene where the last light fails and your protagonist is left to find her way in the dark. You type into the Search bar:

● ● ● *A query for a scene you remember by feeling, not by words*

└─ Search

| the moment the lantern fails|

Press `Enter`, and the top hit is a paragraph that reads:

● ● ● *The paragraph the query finds — which never says any of it*

The last lamp guttered and went out, and Mara counted the dark the way she had once counted the lamps: one, then the next, then nothing at all, the length of the quay gone to soot and the harbour with it.

Read the two side by side. Your query said **lantern**; the paragraph says **lamp**. Your query said **fails**; the paragraph says **guttered and went out**. Your query said **the moment**; the paragraph names no moment at all. A literal search for

`lantern fails` would have returned **nothing** — not one of those characters is in the passage. Yet it is unmistakably the paragraph you meant, and semantic search puts it first, because **the meanings are neighbours** even though the words are strangers. That is the whole promise in one example: you search by what a passage is **about**, and the tool bridges the gap between how you remember it and how you wrote it.

The Search bar — asking the whole book a question

The Search bar is the thin input that runs along the bottom of the window, and it is the front door to project-wide search. You reach it, type, run, read, and jump — five small moves you will wear a groove into.

Opening the bar — Ctrl+/

`Ctrl+/` focuses the Search bar from anywhere; `Ctrl+4` does the same, for hands that prefer the number row. The bar takes your keystrokes and a cursor appears in it. (You can also click it, if a hand is already on the mouse.) Nothing else about the window changes — the panes stay where they were; you have simply pointed your typing at the search input instead of the prose.

Typing a query

Anything goes in the bar — there is no syntax to learn and no operators to remember. A single keyword (`mariner`), an exact-ish phrase you half-recall (`stood at the rail at dawn`), or, best of all, a loose paraphrase of the thing you are after (`the moment the storm broke and someone left the deck`). The query is read as a natural-language description of the passage you want, not as a pattern to match, so write it the way you would describe the scene to a friend.

LONGER QUERIES SEARCH BETTER

It is tempting to type three terse words. Resist it. A fuller query — a sentence, even two — gives the model more to work with and usually retrieves a sharper result than a key-word does. `a quiet argument between them just before the funeral` will beat `argument funeral` almost every time. Semantic search rewards description, not keyword-thrift.

Running it and reading the results

Press **Enter**. A ranked results overlay drops down over the body — the nearest paragraphs, best match first.



Every row is three lines and tells you three things. The first line carries the **score**, the **kind** of node (a **paragraph**, a **note**, a fact, a place), and a human-readable

breadcrumb built from the node titles — **Rain > The City > the-quay**, the trail you would walk down the Tree to reach it. The second line is the node's **title**. The third is a one-line **snippet** from the body, enough to recognise the passage on sight.

TERM The score

The number at the head of each row is a **cosine similarity** — how close the paragraph’s meaning sits to the query’s, from roughly **0.0** (unrelated) toward **1.0** (nearly identical in meaning). It is a **relative** ranking signal, not an absolute grade: read it to compare hits against each other in one result set, not to set a universal pass mark. A **0.72** top hit in a sparse book can be exactly right; a **0.91** in a dense one can still be the wrong scene.

The list reaches across the **whole** project in one ranked column — manuscript prose, your Notes, the Research book, the Places and Characters entries, the Facts. A search for a mood can surface a research note you took weeks ago alongside the scene it informed, because both were indexed the same way. That breadth is a feature: the thing you are looking for is not always in the chapter you think it is in.

Jumping to a hit

↑ and **↓** move the selection through the results. **Enter** on the highlighted row **opens that paragraph in the Editor** — focus moves there, the Editor loads the body, and the Tree cursor repositions onto the very same row, so you land in context and can move to neighbours immediately. One keystroke took you from “the scene where the light fails” to the cursor blinking inside it.

Dismissing the overlay

Esc once closes the overlay while leaving focus in the Search bar, so you can refine the query and run again. **ESC** a second time steps focus on — to the Editor if a paragraph is open, otherwise onward through the focus ring. Nothing **Esc** does here is destructive; it only peels away the overlay.

Ctrl+/	Focus the Search bar (top). Ctrl+4 does the same.
Enter	Run the query; on an open result, open that paragraph.
↑ / ↓	Move the selection through the ranked results.
Esc	Close the results overlay; press again to move focus on.
Ctrl+F	Literal regex find inside the open paragraph (not project-wide).

What makes a good query — and what does not

Semantic search is powerful and a little unlike the search boxes you are used to, so it pays to learn where it shines and where it is the wrong tool.

The cases it is best at

Paraphrase is its home ground — the lantern example above. Describe the passage in your own words and it bridges to however you actually wrote it.

Mood and situation work beautifully, because a passage's emotional register is part of its meaning. Try **a quiet pause in the action, someone delivers bad news**, **the protagonist doubts herself** — you will find the passages that fit even when you have forgotten every literal word.

Character viewpoint is findable by name plus a verb phrase — **Mara realises, the captain's hand** — because the embedding learns that a passage is **about** a person from far more than the bare occurrence of their name.

The long query beats the short one, as the callout above warned: a sentence of description retrieves better than a keyword, every time you are patient enough to type it.

Multilingual, by construction

The search is **multilingual** down to its bones, not as a bolt-on. Set the project language to Russian and a query like **утренний рассвет на палубе** finds Russian prose written in different inflections; the model that powers the search understands roughly a hundred languages, and it maps meaning across the morphology that trips up a literal search. A book written in Russian, French, German, or Spanish searches exactly as well as one in English — the same model, the same index, the same bar. (You will meet the details of **why** in the next section.)

SEARCH ACROSS THE LANGUAGES OF A TRANSLATION

Because meaning is language-agnostic in the index, a query in one language can surface near-neighbours written in another — useful when a manuscript and its translation live in the same project. It is not a translation tool, but it will often land you on the corresponding passage in the sibling book.

What it will not do

It is as important to know the edges. **Literal single-word** search is **not** its strength: a query for one specific word may skip a paragraph that contains that exact word, if the paragraph's overall meaning lands far from the query. When you truly need “every paragraph containing this string,” that is a job for `Ctrl+F` in the editor or `rg` over `books/` — not the Search bar. **Boolean operators** (`AND`, `OR`, `NOT`) do nothing; the interface is natural language, and the way to combine ideas is to describe them together in one query or to run two queries. **Field-restricted** search (`title:foo`) is not supported, and **regex** belongs to the in-buffer `Ctrl+F`, not to the project-wide bar. None of these are oversights; they are the deliberate shape of a meaning-first search.

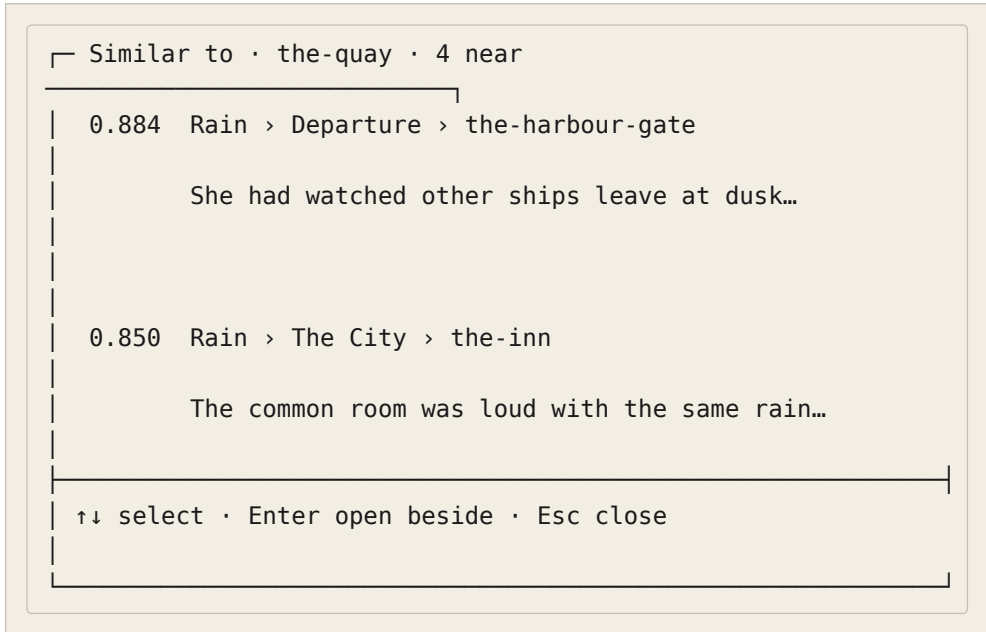
Finding the paragraph like this one — similar-paragraph mode

The Search bar starts from a query you type. There is a second, quieter way into the same vector index that starts from **a paragraph you are already reading**: similar-paragraph mode, on `Ctrl+V S`.

TERM Similar-paragraph mode

`Ctrl+V S` (from the editor) treats the **open paragraph itself** as the query. It saves the buffer, runs a vector-similarity search for the paragraphs nearest it in meaning, and opens the chosen hit in a second editor **side by side** with the first. Press `Ctrl+V S` again to save both and leave. It answers “what else in the book reads like this?” without your having to phrase the question.

● ● ● *Similar-paragraph mode — nearest neighbours of the open buffer*



This is the tool for a different kind of question than the Search bar answers. The Search bar is for **retrieval** — you know roughly what you want and you go get it. Similar-mode is for **discovery** — you are standing in one passage and want to know what else in the book rhymes with it: the other rain scene, the echo you did not plan, the note that anticipates this beat. It is also how you catch **unintended** repetition, the paragraph you have written twice in different chapters without noticing. Both surfaces read the same underlying index; they differ only in where the query comes from — your fingers, or the paragraph under your cursor.

Under the hood — how meaning becomes a search

The magic in the lantern example is worth demystifying, because it is not magic and understanding it tells you when to trust it. Two ideas do all the work: an **embedding** that turns text into numbers, and a **vector index** that finds nearby numbers fast. Both run **entirely on your machine**.

Embeddings — text as a point in meaning space

TERM **Embedding**

An **embedding** is a passage of text rendered as a list of numbers — a **vector** — such that passages with similar meanings produce vectors that sit close together. Inkhaven’s default model outputs a vector of roughly 384 numbers per passage. “The last lamp guttered” and “the lantern fails” land near each other in that 384-dimensional space; “the wedding was at noon” lands far away. Nearness of vectors **is** nearness of meaning — that is the entire trick.

The model that computes these vectors is **MultilingualE5Small**, run locally through fastembed. It is a small, fast, multilingual model — the source of the roughly-a-hundred-languages reach — and it is what makes a Russian query find Russian prose and an English paraphrase find English prose without a word of either being sent anywhere. The ONNX weights load once per session (a one-time cost of about half a second, paid lazily the first time you actually save or search, never at launch), then stay resident and cheap.

ON-DEVICE AND PRIVATE

Every embedding — of your prose and of your query — is computed **on your own machine**. No paragraph, no search term, no scrap of the book leaves the computer to be searched. There is no cloud call, no API key, and no network round-trip in the search path; it works on a train with the wifi off. The model weights live in a per-user cache after their first download, and an air-gapped install can be pre-seeded with them. Search is a local capability, full stop.

The HNSW vector index — finding neighbours fast

Comparing a query against every paragraph’s vector one by one would be slow in a big book. Instead the vectors live in a structure built for exactly this.

TERM HNSW index

HNSW (Hierarchical Navigable Small World) is a graph that links each vector to its near neighbours, so a search can **navigate** toward the closest points instead of checking every one. Inkhaven stores its vectors in vecstore, an HNSW index kept beside the metadata database. A query embeds once, then walks the graph to the nearest paragraphs in time that barely grows as the book does.

The index returns **cosine distance** (lower means closer), which Inkhaven flips into the **cosine similarity** score you see in the results (higher means closer), so the number reads the natural way — bigger is better. Each node actually carries **two** vectors in the index: one for its metadata fingerprint and one for its content, so both the substance of a paragraph and its identifying details are findable.

How a paragraph gets embedded — on save

The index stays current because embedding is wired into saving. Every time you save a paragraph — **Ctrl+S**, or the autosave that fires when focus leaves the Editor — Inkhaven does two things beyond writing the **.typ** file to disk: it feeds the prose to the embedding model and updates that paragraph’s vector in the HNSW index, and it refreshes the paragraph’s metadata (word count, modified time, a derived title if the title was still a placeholder). By the time your focus has reached the next pane, the words are on disk **and** the search index already knows about them. This is why a passage you wrote a moment ago is findable a moment later, and it is why the tutorial’s advice to “just keep writing” costs the search nothing — the index maintains itself, one save at a time.

WHY RE-EMBED ON EVERY SAVE

Re-embedding on each save (rather than once, long ago) means searches always run against the **live** model and the **current** text. Change the embedding model in config and every future save re-embeds with it; edit a paragraph and its vector moves to match the new words. The index is never a stale snapshot of last week’s draft — it is a running mirror of what is on disk now.

Searching within a scope — Facts search

Project-wide search is the default, but sometimes the question belongs to **one part** of the book. The clearest case is the Facts book — the store of things your world holds to be true — and it has its own scoped search on `Ctrl+B Shift+S`.

The Facts search modal — `Ctrl+B Shift+S`

`Ctrl+B Shift+S` opens a two-phase modal that runs a semantic search **restricted to the Facts book**. It uses the very same vector index as the project-wide search and similar-mode, then post-filters the results to the Facts subtree — so you are searching by meaning, but only among your facts.

● ● ● *Facts search — semantic search scoped to the Facts book*

└ Facts search

| query: siege supplies at the north gate|

| [x] 0.891 Facts > Logistics > grain-stores

| Three weeks of grain remained in the keep...

| [] 0.864 Facts > Defences > the-north-gate

| The north gate had not been rehung since...

| ↑↓ select · Space mark · Enter → chat · Esc close

The first phase is the query: type a description (multi-word is fine) and press `Enter` to run the scoped search. The second phase is the ranked matches: `↑↓` navigate, `Space` marks several facts for multi-select, and `Enter` sends the marked facts — or, if you marked none, the cursor's row — into a **targeted Facts chat** grounded in exactly those facts. Any printable key or `Backspace` in the results drops you back to refine the query; `Esc` closes.

How it differs from the Search bar

Three differences matter. It is **scoped** — it never returns prose or notes, only facts, so a big Facts book does not drown your answer in manuscript hits. It is **two-phase** — search then act, with multi-select in between, where the Search bar is search-then-jump. And its payoff is not “open the paragraph” but “**ground a conversation** in these facts”: the marked handful becomes the context for a focused chat, which is the scalable way to reason over a large fact base — you pull in the relevant few rather than loading the whole book into the model. Where the Search bar takes you **to** a passage, Facts search hands a passage **to the assistant**. Same index underneath; a different destination on top.

Keeping the index healthy — reindex

The index maintains itself for everything you do **inside** Inkhaven. It cannot know about changes made **outside** it, and it does not automatically follow a change of model. For those two situations there is one command.

What re-embeds automatically

Saving a paragraph re-embeds it. Renaming a node re-embeds it (the title is part of what is indexed). Creating, editing, and deleting through the TUI all keep the index in step. In normal writing you never think about the index at all — it is already current.

When to reindex

TERM **inkhaven reindex**

`inkhaven reindex` walks every `.typ` file under `books/` and reconciles the store with what is on disk — re-reading each file’s content and re-embedding it. It is the command that re-aligns the index with reality after something changed the files without going through Inkhaven, or after you changed the embedding model. It is idempotent: safe to run as often as you like.

Reach for it after any of these:

- You **switched `embeddings.model`** in `inkhaven.hjson`. The new model must re-embed every paragraph, since old vectors were made by the old model and are not comparable to new ones. Search quality looks wrong until you do.

- You **edited a .typ file in another editor**, or a `git checkout` brought back paragraphs the database had forgotten.
- You **moved or deleted files** under `books/` from a shell, outside the TUI.
- You simply want a “**are my files and the index aligned?**” sanity check.

Two flags refine the walk. `--prune` removes store records whose `.typ` file has gone missing from disk (use it after deleting files by hand). `--adopt` finds `.typ` files on disk the store does not yet know about and registers them under the matching hierarchy branch (use it after dropping new files into a chapter directory). They combine: `reindex --prune --adopt` does both passes.

● ● ● *Re-embedding the whole project after a model switch*

```
$ inkhaven --project ~/Books/my-novel reindex
  re-read 412 paragraphs · re-embedded 412 · pruned 0 ·
  adopted 0
  index aligned with disk.
```

SWITCH THE MODEL, THEN REINDEX

The one sequence to remember: change `embeddings.model` in the config, then run `inkhaven reindex`. The bigger multilingual models (`MultilingualE5Base`, `MultilingualE5Large`, or `BGEM3`) buy better recall at the cost of disk and inference time; whichever you choose, the reindex is what makes existing paragraphs searchable under it. Skip the reindex and old and new vectors mix, and the scores stop meaning anything.

Search from the shell — the CLI

Everything the Search bar does is available from a terminal, which is what you want when you are scripting around a manuscript or piping results into other tools. The subcommand is `search`.

● ● ● *The same semantic search, from a shell*

```
$ inkhaven --project ~/Books/my-novel \
  search "the moment the lantern fails"

0.907 [paragraph ] rain/the-city/the-quay
      The quay at dusk
      The last lamp guttered and went out, and Mara counted...
      id: 7b3e9a04-...

0.861 [paragraph ] rain/arrival/the-crossing
      Losing the road
      With the beacon dark she had only the sound of the...
      id: alf6c2d8-...
```

Each hit prints four lines: the score with the node kind and the slug path; the title; a one-line snippet; and the node's `id`. That last line is the point of the CLI form — the `id` is a stable handle you can feed to other `inkhaven` commands or grep out of the stream. The default is the top ten hits; `--limit N` changes the count.

● ● ● *Capping the result count*

```
$ inkhaven --project ~/Books/my-novel \
  search "someone delivers bad news" --limit 3
```

An empty result set prints `No results.` to standard error and exits cleanly, so a script can tell “nothing matched” from “the command failed.” Typical uses: listing the top matches to a file before a manuscript-wide edit, piping to `grep` to pull the `id` column, or feeding a shortlist of paragraphs into a batch job.

Search from a script — the Bund word

Inkhaven's embedded Bund language exposes the same search to your scripts, so a macro can find passages and act on them. The word is `ink.search.text`.

TERM `ink.search.text`

Stack effect (`query limit -- list`). Takes a query string and a positive integer limit, runs the same semantic `search_text` the Search bar and CLI use, and pushes a list of hit dictionaries — each with `id`, `title`, `score`, `document` (the body), and `kind`. It is the scripting front door to the vector index.

● ● ● *Searching from Bund, then reading the top hit*

```
"the moment the lantern fails" 5 ink.search.text
; -> a list of 5 hit dicts: { id title score document kind }
```

A companion word, `ink.search.load`, goes one step further: given a query and an index, it runs the search and **opens the Nth hit in the editor**, returning false when the search came back empty. It is the scripted equivalent of running a search and pressing `Enter` on a result — useful for a macro that jumps you to “the passage about X” as one bound key. Between the two, a Bund script can find passages by meaning and either process them or navigate to them, with the same index the whole rest of the chapter has been about.

WHAT YOU LEARNED

- There are **two kinds of search**: **literal** (exact characters — `Ctrl+F` in the buffer, or `rg` over the files) and **semantic** (meaning-based). Inkhaven’s **project-wide** search is semantic only.
- Semantic search finds a passage by **meaning**, not words: a query for the moment the lantern fails lands on a paragraph that says “the last lamp guttered and went out” and never uses one of your words.
- The **Search bar** opens with `Ctrl+/ (or Ctrl+4): type a query — longer and more descriptive is better — Enter runs it, ↑↓ picks a ranked hit, and Enter opens that paragraph in the Editor. Ctrl+V S searches from the open paragraph, for what reads like it.`
- Under the hood: an **embedding** (fastembed’s `MultilingualE5Small`, 384 numbers per passage) turns text into a point in meaning space; an **HNSW** vector index finds the nearest points fast. It is **multilingual** and runs **entirely on-device** — nothing leaves your machine. Every save re-embeds the paragraph.
- **Facts search** (`Ctrl+B Shift+S`) is the same index **scoped to the Facts book**, two-phase: search, multi-select, then ground a targeted chat in the chosen facts.
- Run `inkhaven reindex` after switching `embeddings.model` or after any change to the files made outside Inkhaven; `--prune` and `--adopt` reconcile deletions and additions. The CLI `inkhaven search "<query>" [--limit N]` and the Bund word `ink.search.text` reach the same search from a shell or a script.

CHAPTER 8

8

The Style Overlays

The last chapter left you writing prose into the Editor. This one is about the faint marks that appear **underneath** it — amber underlines beneath a hedge word, a purple one beneath a word you have used three times in as many paragraphs, a teal one beneath “was angry,” a chip in the status bar naming whose eyes you are seeing through. Inkhaven calls these the **style overlays**, and they are the most easily misunderstood thing in the whole instrument, so let us be plain at the outset about what they are for.

They are not a grammar checker, and they are certainly not a style **authority**. Every one of them is a question the tool poses and then leaves entirely to you: **you used “just” here — did you mean to?** The overlay has no opinion about

the answer. A filter word is sometimes exactly the right word; a repeated image is sometimes a deliberate refrain; a line of pure telling is sometimes the fastest way across a dull stretch of plot. The overlays exist to make those choices **visible** so they become choices at all, rather than habits you never see. The whole family is built on one principle, and it is worth stating in its own box before we open a single one.

TERM An overlay is advisory

A **style overlay** marks a pattern in your prose — a word, a phrase, a construction — that **often** rewards a second look. It never changes a character of your text, never blocks a save, never scores you. It is a coaching mark you toggle on when you want the second pair of eyes and off when you want the page clean. You question it; you do not obey it.

Two more things are true of the whole family, and both matter enough to say up front. First, every overlay is **multilingual** — its word lists and its stemming are keyed to the project's top-level `language` field, so a Russian manuscript is measured against Russian hedges and a French one against French, never against English words bleeding into foreign prose. Second, every overlay has a **master switch** in `inkhaven.hjson` and a **session-local chord** that flips it for the current run without touching the file. The chords are how you audition an overlay; the config is how you decide, once, that you want it on for good. We will meet each overlay in turn and close on the switchboard that governs them all.

The filter-word overlay — Ctrl+B Shift+F

The oldest and busiest of the overlays flags **filter words**: the intensifiers and hedges that leak into a first draft and dilute it — `just`, `really`, `very`, `quite`, `actually`, `simply`, `seem`, `perhaps`, and their kin. Press `Ctrl+B Shift+F` in the Editor and every one of them in the open paragraph gains a faint amber underline. Press it again and they vanish. That is the whole gesture; the depth is in what gets underlined, and in which language.

TERM Filter word

A word that **filters** the reader’s experience through a layer of the author’s hedging or emphasis instead of letting the image land directly. “The room was very cold” asks the reader to trust an intensifier; “the room was cold” and a shiver ask them to feel it. Not wrong — but almost always worth a glance.

The amber underline

The mark is a coloured underline drawn beneath the word in the Editor, in the amber `theme.style_warning_filter_word_fg` (default `#f9c44e`). It is deliberately quiet — a hint at the edge of vision, not a red-pen slash — and its weight is yours to set: `theme.style_warning_filter_word_modifier` accepts `underline` (the default), `bold`, `dim`, `reversed`, `italic`, `none`, or `+`-combined forms like `underline+bold` for terminals where the plain underline reads faint.

● ● ● *The amber filter-word underline in the Editor*

```

Editor · the-quay [modified]
1 She just wanted to see the harbour, and it
2 really did seem very far, almost too far.
(amber      = a filter word – question, don't obey)

```

Four words are marked here — `just`, `really`, `seem`, `very` — and two words that look like candidates are **not**: “almost” and “too” are not on the English list, and “harbour” is plainly innocent. The underline is a prompt to reread the sentence, nothing more. Strike `Ctrl+B Shift+F` again and the page goes clean for the next stretch of drafting.

The per-language word lists

Inkhaven ships a curated built-in list for each of its five first-class languages — English, Russian, French, German, Spanish — and picks the one that matches your project’s `language`. The English list runs to about thirty entries and gathers the usual suspects into two families: the **intensifier crutches and hedges** (`just`, `really`, `very`, `pretty`, `quite`, `rather`, `fairly`, `somewhat`, `slightly`,

actually, basically, literally, simply, definitely, absolutely, totally, completely, and the notoriously load-bearing that) and the **sensory or hedging verbs** listed in base form (seem, feel, look, appear, sound, notice, begin, start), plus suddenly, perhaps, and maybe. The Russian list is its own vocabulary, not a translation of the English one — очень, просто, именно, довольно, слишком, весьма, крайне, вполне, достаточно, and so on — because the hedges of one language are not the hedges of another.

DOES IT WORK IN RUSSIAN?

Yes, and by design rather than accident. Tokenisation is UAX-29-aware (Cyrillic, Latin, Greek, and Devanagari word boundaries all resolve), the columns are counted in **characters** not bytes so a multi-byte word never shifts the underline, and a `ë → e` fold is applied so a list entry spelled `ещё` still catches the `еще` you actually typed. A language Inkhaven ships no list for is left **silent** — never measured against English words — until you give it a list of its own.

Replacing the list versus adding to it

Two config fields shape the active list, and the difference between them is the one thing to get right. The per-language array — `editor.style_warnings.filter_words.english` (or `.russian`, and so on) — is a **replacement**: leave it empty (the default) and Inkhaven uses its built-in list; put anything in it and your list **replaces** the built-in one wholesale. The `extra_words` field is an **addition**: whatever you list there is unioned on top of whichever list is active, in every language. So the everyday move — “keep the defaults, but also flag my own two crutch words” — is `extra_words`, not the language array. Reach for the language array only when you want to curate the whole list yourself.

FICTION

A novelist who over-uses “some-how” and “a little” adds just those two to `extra_words` and keeps the thirty built-in English defaults underneath — the fastest way to hunt a personal tic without losing the general net.

NON-FICTION

A technical writer who finds the literary defaults noisy in reference prose can **replace** the English list with a short array of the three or four hedges that actually creep into documentation, and leave the rest clean.

There is a third door for a language outside the curated five. The `filter_words.languages` map takes any language by its lowercased name — an Italian project can be enabled with a `languages: { italian: ["molto", "solo"] }` entry, typically written for you by `inkhaven lang bootstrap`. A mapped entry takes precedence over everything; without one, a non-curated language simply stays quiet.

Stemming — one entry catches every inflection

You will notice the built-in lists carry **lemmas**, not every surface form — `seem`, never `seemed` / `seems` / `seeming`. That is because the detector stems both the list and your prose through the project’s Snowball algorithm before matching, so a single entry catches the whole inflectional family. `seem` on the list flags `seemed`; Russian `казаться` flags `казался`, `казалась`, `казалось`, and `казались` alike. The stemmer is also what keeps the overlay honest at the edges: English Snowball reduces `justice` to `justic`, not `just`, so “justice” is never flagged for containing “just.” If you would rather match exact forms — listing every inflection yourself — set `filter_words.use_stemming = false`.

The siblings under the same switch

`Ctrl+B Shift+F` is labelled “toggle style warnings,” in the plural, because it governs more than filter words. The `style_warnings` master switch (and the one chord that flips it) covers a small family of always-on inline detectors that share the same rendering machinery and the same multilingual foundation:

- The **repeated-phrase** detector slides an `n`-word window (default `n = 4`) across the open paragraph and marks, in magenta (`style_warning_repeated_phrase_fg`, default `#eb6f92`), any phrase that recurs `threshold` times or more (default `3`) — with stop-words excluded so “the dog and” does not inflate a count, and stemming applied so “lifted her shoulders” and “lifting her shoulders” are recognised as the same gesture.

- The **show-don't-tell** inline detector marks telling constructions in teal; it has its own section below.
- The **anachronism** detector marks, in an era-caution orange (`style_warning_anachronism_fg`, default `#eba672`), any word that post-dates your manuscript's setting — a `wristwatch` in an 1840 novel. It is off until you set `editor.style_warnings.anachronism.year`, and it ships a built-in lexicon (each term carrying its earliest plausible year) that you extend with a project `terms` list.

Each has its own per-detector `enabled` flag beneath the master switch, so a writer who wants filter-word marks without repeated-phrase magenta can silence just that one. The master switch turns the whole family off at once.

Ctrl+B Shift+F Toggle the style-warning overlays (filter words, repeated phrase, show-don't-tell, anachronism) for the session.

The echo overlay — Ctrl+B Shift+K

The filter-word family looks **within** the open paragraph. The **echo** overlay looks **between** paragraphs — it is the live, in-editor companion to the `echo-repetition` doctor scan, and it catches the revision-stage tic where a distinctive word gets reused a few paragraphs apart: “she **walked** to the window... he **walked** across the room... they **walked** out.” Each sentence reads fine on its own; the cluster clangs. Press `Ctrl+B Shift+K` in the Editor and every word in the **open** paragraph that echoes across nearby paragraphs of the chapter gains an underline in its own muted purple (`style_warning_echo_fg`, default `#b48ead`) — deliberately distinct from the repeated-phrase magenta, so a within-paragraph repeat and a cross-paragraph echo read as two different findings.

● ● ● *The echo overlay — a distinctive word reused nearby*

```

— Editor · the-market —
| 12 The lantern swung. A second lantern hung by
|
| 13 the door, and a third lantern by the dry well.
|
| echo: "lantern" x3 within a 5-paragraph window

```

Cross-paragraph, and live

The overlay reads the chapter’s paragraphs around the one you are editing — using the **open** paragraph live, unsaved edits and all, so a fresh repeat you type this instant is caught without a save. Its sensitivity is governed by three tunables under **editor**, shared with the doctor scan so the live overlay and the batch report agree:

- **echo_window** (default 5) — how many consecutive paragraphs count as “nearby.” A word must cluster within this span to flag.
- **echo_min_repeats** (default 3) — how many occurrences within the window it takes to raise the echo. Lower is more sensitive.
- **echo_max_global** (default 40) — the distinctiveness ceiling. A word used more than this many times across the chapter is treated as ordinary vocabulary an author legitimately reuses, and skipped even when it clusters.

That last knob is the clever part, and it is why the overlay does not need a frequency dictionary for every language. Distinctiveness is measured **relative to this chapter’s own distribution**: a word must repeat but not be common. A word used two hundred times across the book is vocabulary, not an echo, however it clusters; a word used three times total, all within four paragraphs, is a glaring one. The measure is corpus-relative and language-agnostic, which is what lets the same detector serve every language honestly.

MULTILINGUAL, WITH A DOCUMENTED EDGE

Echo detection stems through the project’s Snowball algorithm (with the Russian `ë → e` fold applied first), so `walked/walking` collapse to one echo and Russian `корабль / корабля / корабле` likewise. Languages **outside** the Snowball set — Japanese, Chinese — degrade gracefully to exact-surface matching: inflected variants will not collapse, but identical repeats still flag. The behaviour is documented and consistent with the other overlays’ fallback, never a silent failure.

The master switch is `editor.echo_overlay` (default `false` — it is an opt-in); `Ctrl+B Shift+K` is the session-local override that flips it without rewriting your config.

Ctrl+B Shift+K Toggle the echo overlay — words in the open paragraph echoing across nearby paragraphs of the chapter.

Show, don't tell — the overlay and the AI scan

The most quoted rule of prose craft gets two tools, and the split between them is the model for how Inkhaven pairs a cheap deterministic detector with an optional LLM pass throughout the program. One is the always-on inline overlay under the `Ctrl+B Shift+F` family; the other is a deliberate AI scan on `Ctrl+B Shift+T`.

The inline overlay — three telling patterns

The `show-don't-tell` detector marks, in teal (`style_warning_show_dont_tell_fg`, default `#94e2d5`), three specific constructions that **tell** the reader a feeling instead of **showing** the behaviour that would let them infer it:

- A **copula and an emotion adjective** — a linking verb (`be`, `seem`, `feel`, `appear`, `look`, `become`, `remain`, `grow`, `sound`) followed by an emotion word (`angry`, `sad`, `afraid`, ...). “She was angry” is flagged across both words; “she was running” is not, because “running” carries no emotion label.
- A **manner-of-emotion adverb** — `angrily`, `sadly`, `nervously` — flagged on its own, because such adverbs almost always name the feeling outright.
- A **cognition verb** — `realised`, `knew`, `understood`, `wondered`, `decided` — flagged on its own, because it reports a character’s interior state directly to the reader.

● ● ● *Show-don't-tell — teal marks on three telling patterns*

```

— Editor · the-quay —————
4  She was angry. He nodded, realised she meant
   |
5  every word, and turned away, angrily, at last.
   |
   copula+adj · cognition verb · manner adverb

```

Curated built-in lists ship for all five first-class languages; a language Inkhaven does not ship lists for produces an empty (silent) detector rather than noise. Matching is Snowball-stemmed by default, so “seemed nervous” is caught through the `seem` lemma; set `show_dont_tell.use_stemming = false` for exact forms. The detector is `enabled` by default **under** the master switch — so it rides `Ctrl+B Shift+F` with its siblings.

The AI scan — `Ctrl+B Shift+T`

The inline overlay catches the obvious two-grams; it cannot catch the paragraph that tells you a character is grief-stricken through a page of narrated summary with no flagged word in it. That is what `Ctrl+B Shift+T` is for. It sends the open paragraph to your configured model with a prompt asking for telling passages **and suggested rewrites**, and streams the answer into the AI pane. The two tools are complementary by intent: the regex overlay is free, instant, and always on, catching the mechanical cases; the AI scan costs a call and a moment, and earns it by catching the subtle instances and proposing alternatives. Neither touches your prose — the scan is advisory, its rewrites are suggestions in the AI pane, and any change you make from them is yours to type or to route through the Editorial Pass. The mnemonic is **T for tell**.

Ctrl+B Shift+T AI show-don't-tell scan — send the open paragraph to the model for telling passages plus suggested rewrites, streamed into the AI pane.

The terminology overlay — `Ctrl+V z`

Where the filter-word overlay is about literary reflex, the **terminology** overlay is about discipline: keeping one name for one thing. It reads the **Glossary** system

book — a book of canonical terms, each with a list of **banned synonyms** — and red-underlines any synonym that appears in your prose, so the canonical form reads clean while its rivals are flagged. Define “access token” as canonical with “auth token” among its banned synonyms, and every “auth token” you type earns a red underline (`style_warning_banned_synonym_fg`, default `#e05a5a`) while “access token” stays untouched.

● ● ● *The terminology overlay — a banned synonym flagged*

```
Editor · setup-guide
3 Paste the auth token into the header field,
4 then reload. The access token never expires.
   (canonical — clean, no mark)
terms: "auth token" → use "access token"
```

The overlay is **self-gating**: with an empty Glossary it flags nothing and costs nothing, so a project that does not govern terminology never sees it. Synonyms may be one to three words, matched longest-first over UAX-29 word tokens (so “auth-token” tokenises and matches the two-word form), and it works in Cyrillic as readily as Latin. When your cursor sits on a flagged word the Editor footer names the fix — `terms: "auth token" → use "access token"` — so the correction is one glance away. It is on by default within the master style toggle; `Ctrl+V z` is its own session-local override, distinct from `Ctrl+B Shift+F`.

The deliberate-variant escape hatch

Sometimes the variant is intentional — a character who says “auth token” in dialogue because that is how they speak. With the cursor on a red-underlined synonym, `Ctrl+V Shift+Z` records that canonical term as a **deliberate variant**

in the intent ledger, and both the overlay and the CI-ready `check` stop flagging it. It is the “I meant to write it this way” door, and it is a first-class part of the design rather than a way to silence the tool wholesale.

Ctrl+V z Toggle the terminology overlay — red-underline Glossary banned synonyms of canonical terms.

Ctrl+V Shift+Z Declare the banned synonym under the cursor a deliberate variant, so it stops being flagged.

The POV chip — Ctrl+B Shift+P

Not every overlay draws under your words; some speak from the status bar. The **POV chip** answers a question a long manuscript makes surprisingly easy to lose track of: whose eyes is this paragraph seen through? Press **Ctrl+B Shift+P** and the status bar shows, for the open paragraph, the most-mentioned character as its heuristic point-of-view figure, followed by up to three other named characters present in the scene.

● ● ● *The POV chip riding the status bar*

```
the-quay ● 214w POV Mara ·Corin ·Tyr scope=None
└ breadcrumb ┘ dirty+words ┘ └ heuristic POV chip ┘
```

The chip is a **heuristic**, and honest about it. It counts mentions against the project's existing **characters** lexicon — no separate per-paragraph tagging needed — takes the most-mentioned name as POV, and breaks ties by order of first mention. It will be wrong in a scene where the viewpoint character is silent while another is named repeatedly, which is precisely the kind of place worth a second look: a mismatch between who the chip names and who you **intend** is a prompt, not a verdict. The master switch is `editor.pov_chip_enabled` (default `true`); **Ctrl+B Shift+P** is the session-local flip. Its colours are `theme.pov_chip_bg` / `pov_chip_fg` (defaults `#8b1d88` on `#ffffff`).

Ctrl+B Shift+P Toggle the status-bar POV chip — the most-mentioned character in the open paragraph, plus up to three others present.

The sentence-rhythm gauge — Ctrl+B Shift+H

Prose has a heartbeat, and monotony in it — every sentence the same length — drones however clean the words are. **Ctrl+B Shift+H** opens the **sentence-rhythm gauge**, a modal that splits the open paragraph into sentences (with a hand-rolled walker that suppresses abbreviations — `Mr.`, `Mrs.`, `Dr.`, `e.g.`,

i.e., Ph.D. — so they do not cut a sentence short), measures each one's word count, and reports the mean, the standard deviation, and above all the **coefficient of variation** (CV) — the standard deviation divided by the mean, a single number for how much your sentence lengths vary.

TERM Coefficient of variation

The CV is the spread of your sentence lengths **relative to their average**, so it compares fairly across dense and sparse prose alike. A low CV means every sentence is about the same length — a drone. A high CV means you mix the short and the long — the rhythm most strong prose has.

The gauge maps the CV to one of four verdicts: **Monotone** (CV < 0.25 — the prose drones), **Steady** (0.25 – 0.45), **Varied** (0.45 – 0.80 — the sweet spot of strong rhythm), and **Choppy** (CV ≥ 0.80 — so jagged it may jar). It lists each sentence as a bar and calls out the three shortest and three longest so you can see the outliers at a glance.

● ● ● The sentence-rhythm gauge — CV to verdict

```

Sentence rhythm · the-quay
| mean 14.2 words · stdev 4.1 · CV 0.29
| verdict: STEADY (Monotone <0.25 · Varied >0.45)
|
| s1 █████ 6
| s2 ██████████ 15
| s3 ████████████████████ 24
| s4 ███ 4
| shortest: s4 (4) · longest: s3 (24)
|
| ↑↓ scroll · Ctrl+B Shift+M rewrite · any key close

```

The gauge diagnoses; it does not fix. But because the natural next step from a **MONOTONE** verdict is to break the rhythm up, **Ctrl+B Shift+M** — the AI sentence-rhythm rewrite — fires from **inside** the gauge as well as from the Editor, so the diagnose-then-rewrite path needs no extra keystroke: open the gauge, read the verdict, and if you want it, ask for a rewrite that mixes short and long while

preserving your voice. That rewrite, like every AI prose change in Inkhaven, arrives as a reviewable diff you accept or reject — snapshot first, never an unconfirmed write. The mnemonic is **H for heartbeat**.

Ctrl+B Shift+H Open the sentence-rhythm gauge — per-sentence bars, mean/stddev/CV, and a Monotone/Steady/Varied/Choppy verdict.

Ctrl+B Shift+M AI sentence-rhythm rewrite — mix short and long sentences while preserving voice; arrives as a reviewable diff. Also fires from inside the gauge.

The reader-pace preview — Ctrl+B Shift+E

The last overlay is the one that changes not how your prose **looks** but how you **read** it. You read your own draft at editing-glance speed — skimming, because you already know what it says — and at that speed a run-on that drags or a beat that lands too abruptly is invisible. **Ctrl+B Shift+E** opens the **reader-pace preview**: a teleprompter that advances a highlight word by word through the open paragraph at a reader's speed, so you experience the prose the way a first reader will.

● ● ● *The reader-pace preview — prose at reading speed*

```

┌ Reader pace · 200 wpm ────────────┐
│ The rain came sideways off the harbour, and Mara      │
│ counted the ┌lamps┐ as they went dark, one by one.    │
│                                                       │
│ dim = already read · ┌reverse┐ = now · plain = ahead │
├────────────────────────────────────────────────────────┤
│ word 9/48 · 00:12 left · Space pause · ↔ step · r    │
└────────────────────────────────────────────────────────┘

```

The words behind the highlight dim, the current word is reverse-highlighted, and the words ahead read normally; the footer shows your position and the time left. The pace is `editor.reading_wpm` (default `200`, the common silent-reading average — drop it toward `150` to feel an audiobook narration). **Space** pauses and resumes (elapsed time carries across cycles), **←** and **→** step the highlight by a word, **r** restarts from the top, and **Esc** closes. It reads **clean** prose — the same markup-

stripping pass the audiobook export uses — so your Typst markers never break the flow. The mnemonic is **E for experience**.

Ctrl+B Shift+E Reader-pace preview — a word-by-word teleprompter through the open paragraph at `editor.reading_wpm`, to feel your prose at a reader's speed.

Master switches and session overrides

Now the switchboard. Every overlay in this chapter has the same two-level control, and understanding the pattern once saves you configuring each one from scratch. The **master switch** lives in `inkhaven.hjson` and is the persistent default — on or off every time you open the project. The **session-local chord** is the audition: it flips the overlay for the current run only and never rewrites your config, so you can turn something on to check a passage and let it lapse when you quit. When you decide, once, that you want an overlay on for good, you set its master switch; day to day, you reach for the chord.

The overlays cluster under a handful of config keys, most of them inside the `editor` block:

● ● ● *The overlay config, gathered in inkhaven.hjson*

```
editor: {
  style_warnings: {
    enabled: false           // master – Ctrl+B Shift+F
    filter_words: {
      enabled: true
      use_stemming: true
      extra_words: [ "somehow", "a bit" ]
      english: []           // [] = use the built-in list
    }
    show_dont_tell: { enabled: true }
    repeated_phrases: { enabled: true, n: 4, threshold: 3 }
    anachronism: { year: 1840 } // off until set
  }
  echo_overlay: false       // master – Ctrl+B Shift+K
  echo_window: 5
  echo_min_repeats: 3
  echo_max_global: 40
  pov_chip_enabled: true    // status chip – Ctrl+B Shift+P
  reading_wpm: 200         // reader pace – Ctrl+B Shift+E
}
```

A few defaults are worth committing to memory because they surprise people. The `style_warnings.enabled` master is `false` out of the box – the whole filter-word family is **opt-in**, so a fresh project shows no underlines until you either flip the master or press `Ctrl+B Shift+F`. The per-detector flags beneath it (`filter_words.enabled`, `show_dont_tell.enabled`, `repeated_phrases.enabled`) default `true`, so once the master is on the family comes on together; you silence an individual detector by setting **its** flag false. The echo overlay's master (`echo_overlay`) is likewise `false` by default. The POV chip's master (`pov_chip_enabled`) is `true`. And the terminology overlay is on within the master style toggle but self-gates to silence on an empty Glossary.

EDIT THE CONFIG WITHOUT LEAVING THE EDITOR

You do not have to leave Inkhaven to change any of this. **Ctrl+B 0** opens `inkhaven.hjson` in a full-screen editor with syntax highlighting; **Ctrl+S** saves, and because these keys are read at launch a **Restart required** notice appears when a change needs the next run to take effect. The session chords are there precisely so you rarely need the round-trip mid-draft.

Here is the whole family in one table — the chord, and the master key it overrides — so you can find any overlay from any pane.

Ctrl+B Shift+F	Style warnings (filter word / repeated phrase / show-don't-tell / anachronism) — master editor.style_warnings.enabled.
Ctrl+B Shift+K	Echo overlay — master editor.echo_overlay.
Ctrl+B Shift+T	AI show-don't-tell scan (per-invocation; complements the inline overlay).
Ctrl+V z	Terminology overlay — on within the style master; self-gates on an empty Glossary.
Ctrl+V Shift+Z	Declare a banned synonym a deliberate variant.
Ctrl+B Shift+P	POV chip — master editor.pov_chip_enabled.
Ctrl+B Shift+H	Sentence-rhythm gauge (modal; Ctrl+B Shift+M rewrites from inside it).
Ctrl+B Shift+E	Reader-pace preview (modal; paced by editor.reading_wpm).

The overlays are the quietest intelligence in the tool — no reports, no letters, no scans you wait on — just a set of faint marks you can raise and lower over the page as you draft. Turn them on when you want the second pair of eyes on a finished paragraph; turn them off when you want to write without one watching. They only ever ask; the answer, in every case, is yours.

WHAT YOU LEARNED

- The style overlays are **advisory** — they mark patterns worth a second look (filter words, echoes, telling, banned synonyms, POV, rhythm, pace) and never edit your prose, block a save, or score you. You **question** them.
- **Ctrl+B Shift+F** toggles the **filter-word** family — amber underlines on intensifiers and hedges, keyed to the project `language`, Snowball-stemmed; `extra_words` **adds** to the list, the per-language array **replaces** it. The same switch governs repeated-phrase, inline show-don't-tell, and anachronism.
- **Ctrl+B Shift+K** toggles the **echo** overlay — a distinctive word reused across nearby paragraphs — tuned by `echo_window` (5), `echo_min_repeats` (3), and the `echo_max_global` (40) distinctiveness ceiling.
- **Show-don't-tell** is two tools: the always-on inline overlay (copula+emotion, manner adverb, cognition verb) under **Ctrl+B Shift+F**, and the AI scan on **Ctrl+B Shift+T** for the subtler cases with rewrites.
- The **terminology** overlay (**Ctrl+V z**) red-underlines Glossary banned synonyms and self-gates to silence when the Glossary is empty; **Ctrl+V Shift+Z** marks a variant deliberate. The **POV chip** (**Ctrl+B Shift+P**), **rhythm gauge** (**Ctrl+B Shift+H**), and **reader-pace preview** (**Ctrl+B Shift+E**) round out the family.
- Every overlay has a **master switch** in `editor.*` (the persistent default, most of them off) and a **session-local chord** (the audition that never rewrites config). **Ctrl+B 0** edits the config in place.

PART III

The AI Assistant

CHAPTER 9

9

The AI Assistant

Every other chapter so far has been about you and the manuscript. This one adds a third party to the desk: a language model you can consult without leaving the window. Inkhaven's stance on that model is stated in a single line of its own README — *AI is a co-author you steer* — and the word that matters is *steer*. The assistant never acts on its own, never edits a word you did not ask it to edit, and never sees more of your book than you decide to show it. Everything in this chapter is machinery for that steering: how you point the model at a slice of the manuscript (the **scope**), how much of its own training you let it draw on (the **mode**), where its answer is allowed to land (the **destination**), and which of six providers is doing

the thinking. Learn these four dials and the AI pane stops being a chat box and becomes an instrument.

The pane itself you already met in Chapter 3, as one of the three surfaces the right-hand region can show. Here we open it up.

Opening and focusing the pane

The AI feature is really two surfaces working as a pair: the **AI pane**, where answers stream in and the conversation scrolls, and the **AI prompt bar**, the thin input line along the bottom where you type. They are bound together so tightly that Inkhaven treats moving between them as a bounce rather than a full focus change.

To reach the prompt from anywhere, press **Ctrl+I** (or **Ctrl+5**) — focus drops to the AI prompt bar and you can start typing immediately. To read and scroll the answer, **Ctrl+3** focuses the AI pane itself. Once you are on one of the two, **Esc** bounces you to the other: **Esc** from the prompt bar lifts focus up into the pane so you can scroll the history; **Esc** from the pane drops focus back to the prompt so you can type a follow-up. This AI ↔ AI-prompt pairing is its own little loop, independent of the Tree / Editor / Search rotation that **Tab** walks.

TERM The bounce

From the AI prompt bar, **Esc** moves focus to the AI pane (to read); from the AI pane, **Esc** moves focus back to the prompt bar (to type). You ping-pong between asking and reading with one key, and never fall out of the AI surfaces by accident.

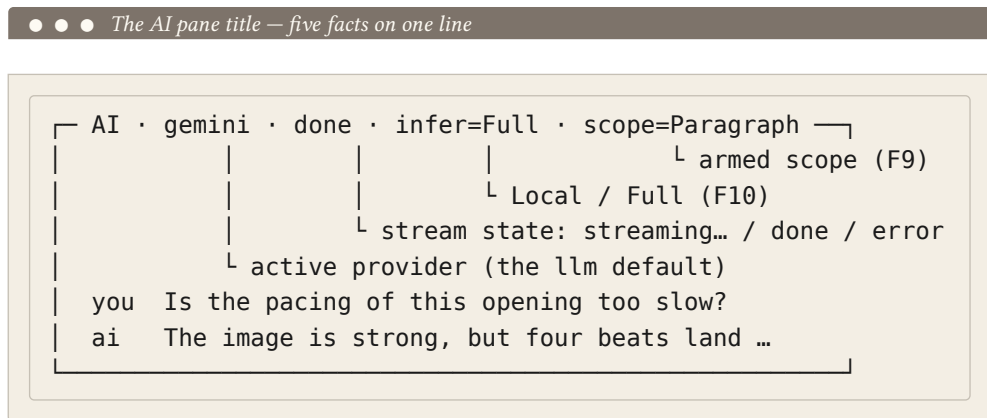
If the right region is currently showing the Output or Thoughts pane instead, **Ctrl+B Tab** cycles it around to the AI pane (Output → AI → Thoughts), or **Ctrl+3** jumps straight there and surfaces it. And when a conversation grows long enough that the four-pane layout feels cramped, **Ctrl+B K** fullscreens the AI pane — the chat history and the prompt bar spread across the whole window for an extended session. Press it again to return to four panes.

Ctrl+I / Ctrl+5	Focus the AI prompt bar (bottom). Start typing at once.
Ctrl+3	Focus the AI pane and surface it in the right region.
Esc	Bounce between the AI pane and the AI prompt bar.

Ctrl+B Tab	Cycle the right region Output → AI → Thoughts.
Ctrl+B K	Fullscreen the AI pane with its history and prompt (toggle).
Enter	Send the prompt (from the prompt bar).

The title bar — the pane's state at a glance

The AI pane wears a dense title strip, and it repays a glance. Read left to right, it folds in everything about the pane’s current state: the provider doing the work, whether a stream is in flight, the inference mode, the armed scope, and how deep the conversation has run.



While tokens are arriving the stream field reads `streaming...`; when the model finishes it flips to `done`; if the call fails it shows `error` and the failure text lands in the pane body. Once a conversation is under way the strip also carries the turn depth — `3 turn(s)` — so you know how much history the next prompt will replay. Two of the fields are colour-coded by default so an accidental setting is obvious at a glance: the scope chip is peach (`theme.ai_scope_fg`) and the inference-mode chip is teal (`theme.ai_infer_fg`). When prompt-language resolution is active a `lang=` chip joins them, reading `ru (book)` or `ru (paragraph)` depending on how the language was decided (see the last section of this chapter).

The prompt bar has a title of its own, and it echoes the armed scope so you see it while you type: `AI prompt · scope: Paragraph`. You rarely read either strip word for word, but between them you are never guessing what the next `Enter` will do.

Streaming inference

Press `Ctrl+I`, type a question, press `Enter`, and the answer begins to appear immediately — not a spinner, then a wall of text, but characters arriving left to right as the model produces them. Inkhaven streams the tokens into the pane as they land. This is worth internalising: the model is *literally* writing the answer one piece at a time, and you are watching it think, so you can start reading — or decide the answer is going the wrong way and clear it — before it finishes.

● ● ● A streaming answer, tokens arriving live

```
AI · gemini · streaming... · infer=Full
| Several darker alternatives:
|
| • stunned sailor — keeps the alliteration
| • shaken helmsman — slightly nautical
| • dazed deckhand — leans on disorientation
|
| "Thunderstruck" reads pre-20th-century; if the |
```

The pane renders the response as *markdown* as it streams: `*bold*`, `_italic_`, `inline code`, `# headings`, bullet and numbered lists, code fences, and block-quotes all display as formatted text rather than raw asterisks and hashes. That rendering is purely a display convenience — the raw markdown is what Inkhaven actually stores and what flows through the apply pipeline described below.

Underneath, the status bar narrates the call. While the tokens arrive it reports something like `streaming from gemini (chat turn #3 · scope=Paragraph)...`; when the model finishes it prints the elapsed time, `gemini responded in 1.8s`. If the call fails — a missing API key, a mistyped model name, a network hiccup — the status flips to an error, the title carries `error`, and the error text sits in the pane body. Fix the cause and re-send; nothing is lost.

READING WHILE IT WRITES

You do not have to wait for `done`. Scroll the pane (`Ctrl+3`, then the arrows or the wheel) while a stream is still running, or type your next question into the prompt bar — Inkhaven queues nothing behind your back, so a follow-up sends a fresh call once the current one settles.

Scope — telling the AI what to look at

By default the model sees only the prompt you typed plus the running chat history. It knows nothing about your manuscript unless you attach a slice of it. The dial that attaches that slice is the *scope*, and you cycle it with a single key: `F9`, from any pane.

TERM Scope

The *scope* decides what chunk of your book is prepended to the next prompt as context. `F9` cycles it. A non-`None` scope prepends its matching content to the query you send, then — for the manuscript scopes — *auto- resets to* `None` after that one submission, so a follow-up prompt is never surprised by stale context you forgot was armed.

`F9` walks a ten-stop ring and wraps:

● ● ● *The F9 scope ring*

None → Selection → Paragraph → Subchapter → Chapter →
Book → Facts → Socrates → Editor → Graph → None

The armed scope shows in three places while it is set: the AI pane title (`scope=Paragraph`), the prompt-bar title (AI prompt · `scope: Paragraph`), and the status bar, which spells out `AI scope: Paragraph (will prepend matching context to next prompt)`. After you send, a manuscript scope's chip disappears as it resets to `None`.

The manuscript scopes

The first six stops draw text from the tree around your cursor — progressively wider slices of the book you are actually writing.

• • • What each manuscript scope sends

Scope	Sends to the model
None	Nothing extra — the prompt + prior chat.
Selection	The current editor selection. Errors if no selection is active.
Paragraph	The whole open paragraph. In split-edit, both the snapshot and the live buffer.
Subchapter	Every paragraph under the cursor's enclosing subchapter.
Chapter	Same, for the enclosing chapter.
Book	Same, for the enclosing book — but retrieval-grounded (see below).

Two of these have a wrinkle worth stating plainly. *Selection* errors out with a status message if there is no active selection — it has nothing to attach, so it refuses rather than send an empty scope. And *Book* does not blindly stuff an entire novel into one prompt; that would blow past every model's context window and cost a fortune. Instead it is *retrieval-grounded*: Inkhaven retrieves the passages most relevant to your question from the book's semantic index and cites those, rather than sending the whole text. A book-wide question therefore stays affordable and pointed, at the price of the model seeing the relevant passages rather than literally every word.

FICTION

Match the scope to the question's reach. Brainstorming one sentence? Selection or None. Tightening a paragraph? Paragraph. Checking that a scene stays consistent with itself? Subchapter. Chasing a plot thread across a chapter? Chapter. A whole-book sweep? Book — long, and best on a large-context provider.

NON-FICTION

For reference and documentation the same ladder holds: Selection to reword a clause, Paragraph to fact-check a claim in place, Chapter to ask whether a section contradicts an earlier one, Book to ask a question that ranges across the whole document and let retrieval find the relevant passages.

The structured scopes — Facts, Socrates, Editor, Graph

The last four stops on the ring are a different animal. Where the manuscript scopes attach *prose from around your cursor*, these attach a *curated body of knowledge* and pre-seed the conversation with a matching system prompt. And unlike the manuscript scopes, they are *sticky*: they persist across follow-up prompts until you cycle F9 away from them, because they open a standing conversation rather than a one-shot attachment.

● ● ● The four structured scopes

Scope	Opens a conversation with...
Facts	every paragraph of the Facts system book — the world's invariants — as ground truth, seeded with a fact-analysis system prompt.
Socrates	the open paragraph read by the active Reader Persona (the Inner Socrates).
Editor	the Inner Editor's craft observations on the open paragraph.
Graph	your knowledge graph — relevant passages plus the edges that connect them (contradicts / sourced_from / cites / ...).

These are the doorways into Inkhaven's reading intelligences from the AI pane, and each has a whole world behind it. *Facts* turns the AI pane into an interrogation of your worldbuilding — ask whether a claim holds against the established canon and

the model answers from your Facts book, not from the internet. *Graph*, like *Book*, is retrieval-grounded, but it additionally folds in the graph relations touching each retrieved passage, so the answer is grounded in how the book's parts *connect*, not just in the prose. *Socrates* and *Editor* are the conversational faces of the Inner Socrates and Inner Editor respectively. All four are covered in full in Chapter 10 and the companion *Know Your Book* here it is enough to know that **F9** reaches them and that they stay armed until you cycle past *Graph* back to *None*.

STICKY VS. RESETTING

The six manuscript scopes reset to *None* the instant you send — one prompt, one attachment. The four structured scopes (*Facts*, *Socrates*, *Editor*, *Graph*) stay armed across every follow-up until you press **F9** to leave them. The pane title's `scope=` chip is your reminder of which world you are still talking to.

Mode — local-only RAG versus full knowledge

Scope decides *what the model sees*. Mode decides *how much of its own training it may lean on*. There are two settings, and **F10** toggles between them from any pane.

● ● ● The two inference modes

Mode	The model is told...	Reach for it when...
Local own checking worldbuilding, inside	use ONLY the supplied context and prior chat – refuse rather than fall back on general knowledge.	summarising your canon, fact- against working strictly the manuscript.
Full craft the reading.	treat the context as ground truth where present, but general knowledge is fair game.	brainstorming, questions, anything that benefits from model's wider

The default is *Full*, because most fresh chats are exploratory and you *want* the model to suggest a trope, name a craft book, or reach for a comparison. Flip to *Local* when you are asking the model to work only from what you have shown it – “summarise what the Facts book says about the northern climate” should never be answered from the model’s guess about real-world northern climates. The active mode is always shown in the title (`infer=Local` / `infer=Full`, teal by default), precisely so an accidentally- armed `Local` mode is never a silent surprise.

TERM Local mode is not privacy

Local mode constrains what the model may *draw on* – its own training versus your context. It does *not* mean the call stays on your machine. With an external provider, a `Local`-mode prompt still travels to that provider’s servers. On-device privacy is a *provider* choice (Ollama), not a mode choice – see the providers section below.

Two flows override the toggle. Help inferences (F1, or a **Help!** prefix in the prompt bar) and grammar checks (F7) are *pinned to Local* no matter where F10 sits — each has its own strict system prompt, and neither should ever invent a feature or paraphrase a rule from general training data.

Destination — where an answer lands

An answer in the pane is inert until you decide what to do with it. When an inference is *done* and has non-empty content, a row of action chips appears in the pane’s footer, and each key sends the response somewhere different.

● ● ● *The apply chips, shown when an inference is done*

r replace i insert t top b bottom c copy g grammar

● ● ● *What each destination does*

Key	Destination
r/R	Replace — overwrite the editor selection with the AI text. With no selection, replace the whole paragraph.
i/I	Insert — drop the text in at the cursor.
t/T	Top — prepend to the paragraph (blank-line gap).
b/B	Bottom — append to the paragraph.
c/C	Copy — to the clipboard only; the buffer is not touched.
g/G	Grammar — lift the corrected text from an F7 grammar-check result (see Chapter 6 / grammar).

Three things hold for every apply. First, *markdown becomes Typst* on the way in: when the answer lands in the editor (r, i, t, b), Inkhaven converts its markdown to Typst syntax — # headings become =, ***bold*** stays ***bold*** (Typst’s own bold), 1. lists become +, and so on — so the pasted text is valid manuscript markup, not raw markdown. Only c keeps the raw markdown, precisely so you can paste it somewhere outside Inkhaven. Second, an apply sets the buffer *dirty* press **Ctrl+S**

to commit it (or let idle autosave do so). Third, after **r** / **i** / **t** / **b** the *focus jumps to the editor* so you land where the change did and can review it in context.

THE GRAMMAR CHIP IS SPECIAL

g is not a general “apply”. It only works on the output of the **F7** grammar check, whose response is wrapped in markers; **g** lifts the corrected text from between those markers and applies just that. On any other inference **g** has nothing to lift. The full grammar-review flow, with its change highlighting, lives in Chapter 6.

Chat history — the conversation is continuous

Every ordinary inference (everything except a one-shot Help or grammar call) appends a **(you, ai)** turn to an in-memory chat history, and the next prompt replays the whole history to the model. The conversation is therefore *continuous*: when you say “now do another pass, especially on the dialogue tags,” the model knows what “this” was because the previous turn is replayed along with it. The title’s **N** **turn(s)** chip tracks the depth.

A typical multi-turn pass reads like a dialogue with an editor who remembers:

● ● ● A continuous revision loop

1. **F9** → Paragraph. Prompt: “Tighten this.” Enter.
2. **r** – replace the paragraph with the tightened text.
3. Prompt: “Now another pass, harder on the dialogue tags.” Enter. (History knows what “this” was.)
4. **r** – apply again.
5. **Ctrl+B C** – clear the thread and start fresh.

Ctrl+B C clears the chat history along with the currently displayed inference — the way to start a genuinely fresh conversation when the old context would only confuse the next question. From the AI pane it doubles as the cancel key for a stream you no longer want. (Help answers never enter the history at all — they are deliberately one-shot, so a question about Inkhaven itself never pollutes the thread you are having about your prose.)

Prompt templates and the Help! prefix

Two shortcuts live in the prompt bar itself. Typing `/` opens the *prompt picker*, a list of reusable templates drawn from two places: the `prompts.hjson` file in your project root (shown as *system* prompts) and the paragraphs under the *Prompts* system book (shown as *book* prompts). Arrow keys move, `Enter` or `Tab` expands the chosen template into the bar — with any `{{selection}}` / `{{context}}` substitutions already applied — and you can edit the expanded text before sending. Direct invocation works too: type `/tighten` and `Enter` with no picker at all. `Esc` closes it without expanding. The full template schema is its own subject; the point here is that a prompt you type often need never be typed twice.

The second shortcut is the `Help!` prefix. Typing `Help!` followed by a question routes the rest of the line through the `F1` help-manual flow — a retrieval-grounded answer over Inkhaven’s own Help book, pinned to `Local` so it never invents a feature — without your leaving the prompt bar. The capitalisation is exact: `Help!`, not `help!` or `HELP!`, so an actual sentence that happens to start with the word “help” is never hijacked.

Providers — six bundled, and any the router reaches

Behind the pane sits a *provider*: the company or runtime that actually runs the model. Inkhaven speaks to all of them through the `genai` crate, which picks the right adapter from the *model name* alone — so adding a new model is a matter of naming it, not writing code. Six providers are bundled as defaults.

● ● ● The six bundled providers

Provider	Default model	API-key env var
gemini	gemini-2.5-pro	GEMINI_API_KEY
claude	claude-sonnet-4-5	ANTHROPIC_API_KEY
openai	gpt-4o	OPENAI_API_KEY
deepseek	deepseek-chat	DEEPSEEK_API_KEY
grok	grok-2-latest	XAI_API_KEY
ollama	(a local model)	— none (on-device)

You configure them in the `llm` block of `inkhaven.hjson`. Each entry names a `model` and, for the cloud providers, the environment variable that holds your key; `default` names which one the TUI uses. Local runtimes like Ollama omit the key entirely — the auth check is simply skipped when `api_key_env` is absent.

● ● ● *The llm config block*

```
llm: {
  default: gemini          # which provider the TUI streams
  from
  auto_fallback: true      # use any working key if default's
  unset
  providers: {
    gemini: { model: gemini-2.5-pro, api_key_env:
GEMINI_API_KEY }
    claude: { model: claude-sonnet-4-5, api_key_env:
ANTHROPIC_API_KEY }
    deepseek: { model: deepseek-chat, api_key_env:
DEEPSEEK_API_KEY }
    ollama: { model: llama3.2 }      # no key – runs locally
  }
}
```

Set a key by exporting its variable before you launch — `export GEMINI_API_KEY='...'` — and Ollama needs nothing but the daemon running and a model pulled (`ollama pull llama3.2`). The `default` provider is the one the TUI streams from; to change it, edit `default` and relaunch. There is no per-inference provider switch inside the TUI today — the model is a session-level choice — but the CLI takes a one-shot override: `inkhaven ai "summarise this" --provider deepseek`.

TERM `auto_fallback`

When `auto_fallback` is `true` (the default) and your `default` provider's key is unset, Inkhaven quietly uses any other configured provider whose key is available — including a keyless local one like Ollama — rather than failing. It is the “use whatever works” setting. Set it to `false` if you want the configured provider or a clear error, and nothing else.

PRIVACY — EXTERNAL PROVIDERS SEE YOUR PROMPTS

This is the one privacy fact to internalise. When you use an external provider (Gemini, Claude, OpenAI, DeepSeek, Grok), your prompts — including whatever manuscript context the scope attaches — *travel to that provider's servers* under their terms. Inkhaven adds no inherent privacy over that channel, and Local mode does not change it. For genuine on-device privacy, set the default to `ollama` and run a local model: the prompt never leaves your machine. Every *other* Inkhaven subsystem — the RAG embeddings, semantic search, snapshot diffs — is already fully local, so switching to Ollama makes the whole tool on-device.

Prompt-language resolution

Inkhaven is multilingual to its core, and the AI pane is no exception: a prompt about a Russian paragraph should be answered in Russian, keyed off Russian word lists and a Russian-aware system prompt. The dial that decides which language a prompt resolves against is *prompt-language resolution*, set by `editor.prompt_language_mode` in HJSON and toggled at runtime with `Ctrl+B Shift+N`.

There are two strategies. *book_defined* (the default) resolves *every* prompt against the project's top-level `language` field — simplest, and right for a manuscript written in one language. *paragraph_detected* instead runs the `whatlang` detector over the *live* paragraph body and resolves against whatever language it finds — the mode for a genuinely bilingual project where one chapter is English and the next is Russian. Because `whatlang` is unreliable on very short text, *paragraph_detected* only attempts detection once the paragraph has at least `prompt_language_detection_min_chars` of non-whitespace text (default 50); below that floor it silently falls back to the book language.

`Ctrl+B Shift+N` cycles the mode *for the session* without rewriting your HJSON: `None` (defer to the config) → `book_defined` → `paragraph_detected` → `None`. The AI pane title's `lang=` chip reflects the outcome so you always know which way a prompt was resolved — `ru (book)` when the book language decided it, `ru (paragraph)` when detection did — and the status bar echoes the new mode as you toggle.

● ● ● *The lang chip records how the language was chosen*

```
┌ AI · gemini · done · infer=Full · lang=ru (paragraph) ┐
│ you Перепиши этот абзац живее.                        │
│ ai Дождь хлестал наискось с гавани, и Мара ...         │
└────────────────────────────────────────────────────────┘
```

WHAT YOU LEARNED

- The AI feature is a *pair*: the AI pane (answers stream in, history scrolls) and the AI prompt bar (you type). `Ctrl+I` focuses the prompt, `Ctrl+3` the pane, and `Esc` *bounces* between them; `Ctrl+B` `K` fullscreens the pane.
- *Scope* (`F9`) decides what the model sees. Six *manuscript* scopes — `None` / `Selection` / `Paragraph` / `Subchapter` / `Chapter` / `Book` — attach prose and auto-reset after one send (`Book` is retrieval-grounded). Four *structured* scopes — `Facts` / `Socrates` / `Editor` / `Graph` — open sticky conversations with the reading intelligences (Chapter 10).
- *Mode* (`F10`) decides how much the model leans on its own training: *Local* uses only the supplied context, *Full* (the default) may augment with general knowledge. Help and grammar are pinned to *Local*. *Local* is *not* privacy.
- *Destination* chips land a done answer: `r` replace, `i` insert, `t` top, `b` bottom, `c` copy (raw markdown), `g` grammar-apply. Editor applies convert markdown → Typst and set the buffer dirty.
- Six providers are bundled — Gemini, Claude, OpenAI, DeepSeek, Grok, Ollama — plus any model genai routes; the `llm` block sets `default`, per-provider models and key env vars, and `auto_fallback`. External providers *see your prompts* Ollama keeps them on-device.
- Prompt-language resolution (`Ctrl+B` `Shift+N`) picks the language a prompt answers in: *book_defined* (the project language) or *paragraph_detected* (whatlang over the live paragraph, above a 50-char floor); the title's `lang=` chip records which decided.

CHAPTER 10

10

Chat With Your Book

An assistant that has never read your book can only ever guess at it. Ask a general model whether Mara doubts the captain and it will write you a fluent, confident paragraph about a character it has never met — plausible, ungrounded, and wrong as often as not. The panes you met in Chapter 3 already give you a conversation; this chapter is about what that conversation is **grounded in**. The whole point of writing inside your own material is that the assistant can be made to answer **from** it — to retrieve the passages that actually bear on your question, quote them back by name, and refuse to invent what the text does not say. That is the difference between chatting **with** a model and chatting **with your book**.

The control that governs all of this is a single key, **F9**, which cycles the AI pane’s **scope**. You have already met the near end of that dial — the scopes that send the model a slice of prose around your cursor. This chapter is about the far end: the **structured scopes**, where the pane stops shipping raw text and starts **retrieving**, grounding each answer in evidence it can cite. Book, Facts, Graph, and the two reader conversations all live there, and each grounds in a different part of what Inkhaven knows about your work.

The scope dial — F9 and what an answer stands on

Every question you send the AI pane carries a scope, and the scope decides what context travels with it. Press **F9** from anywhere and the scope advances one notch; the pane’s title and the status bar both name the current one (**scope=Book**). Ten stops sit on the ring, and they fall into two families.

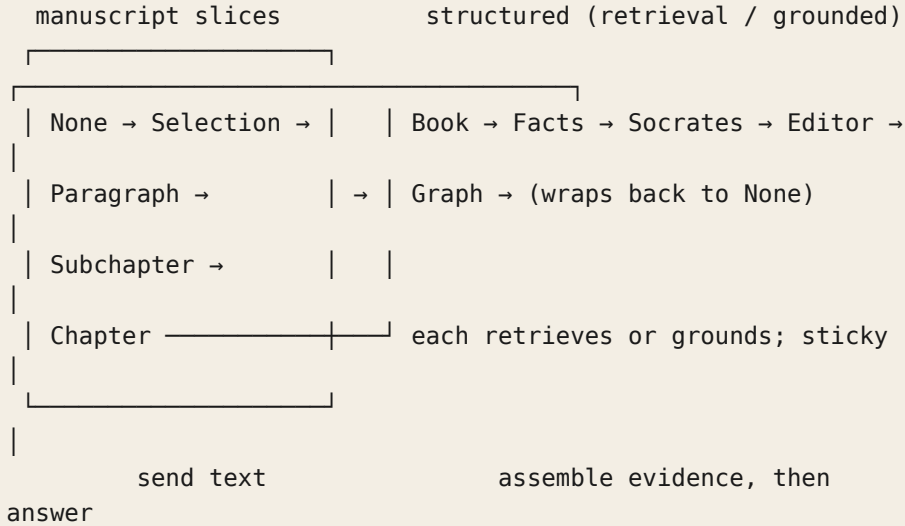
TERM AI scope

The **scope** is what the AI pane sends the model alongside your prompt. **F9** cycles it. The first five stops are **manuscript slices** — a literal span of prose around your cursor. The last five are **structured scopes** — Book, Facts, Graph, Socrates, and Editor — which retrieve or assemble grounding rather than sending text verbatim, and are **sticky**: the conversation stays in that scope, reasoning over the same evidence, until you cycle away.

The first family is the plain one. **None** sends nothing but your words; **Selection** sends the highlighted span; **Paragraph**, **Subchapter**, and **Chapter** send exactly the prose those names enclose. What you see is what the model gets. These are covered where they belong, in the editor chapters; they are unchanged and unsurprising.

The second family is the subject of this chapter, and it behaves differently on purpose. A Book-scope question does not send your book — it **searches** it. A Facts-scope question loads the world’s invariants as ground truth. A Graph-scope question folds in how your book’s parts connect. Socrates and Editor seat a particular reader across the table from you. None of these send a fixed span of text; each **builds** its grounding from a different store, and each is worth understanding on its own.

● ● ● *F9 walks the scope ring; the last five are structured*



STICKY SCOPES RETRIEVE ONCE

A structured scope holds its ground across a whole conversation. It gathers its evidence when you enter it (or send the first question), and your follow-ups reason over that same evidence rather than re-gathering on every turn. To make it look again — a new line of inquiry, or freshly edited prose — clear the chat history, and the next question re-grounds from scratch.

Book scope — retrieval-augmented conversation

Book scope is the flagship of this chapter, and the feature it implements has a name worth learning, because it is the same idea that quietly powers half the tool.

TERM Retrieval-augmented generation (RAG)

Instead of handing a model everything and hoping it finds the relevant part, **retrieval-augmented generation** first **retrieves** the handful of passages that actually bear on the question — by semantic similarity, not keyword match — and hands the model only those, as labelled evidence to reason from. The answer is grounded in real, quotable text rather than in the model’s diffuse memory of everything it has ever read.

Book scope used to be the tool’s weakest surface. It sent your whole manuscript along with every question — fine on a ten-page draft, ruinous on a real book. The model skimmed a hundred pages, grounded in none of them, and answered from a blur. Modern Inkhaven replaces that with RAG, and the change is total: Book scope **is** retrieval now. There is no switch to flip and no “send the whole book” mode to fall back to, because the whole-book mode never worked at scale.

Here is what happens behind a Book-scope question. Inkhaven runs a semantic search for your query over the same on-save vector index every other semantic feature uses, keeps the most similar paragraphs, widens each one with a neighbour or two so it reads in context rather than as a fragment, drops duplicates, and packs the result into a focused, token-budgeted block of evidence — best hits first, ordered by their position in the book. That block, and only that block, is what the model sees. Then it answers.

● ● ● *A question in Book scope, and a cited answer*

— AI · Scope: Book —
| > does Mara ever doubt the captain?_

She does, twice. Her first real doubt surfaces on the crossing, when the captain orders the night run despite the glass falling — she calls it "a wager with other men's lives" [act-two/the-night-run]. By the storm chapter that doubt has hardened into open defiance [act-three/storm-at-sea].

You asked a question about your book and got back specific passages, by name, instead of a plausible paragraph the model invented. Every claim it makes about the book is anchored to a citation — and those citations are the heart of why the answer is trustworthy.

Citations you can trust — and the ones you can't

Each bracketed label is a passage's **location path** — `[chapter-slug/scene-slug]` — not an opaque database id. You can read **where** a claim comes from and jump straight to it. The system prompt makes the contract explicit and ships localised for the five baseline languages (English, Russian, Spanish, French, German, with English as the fallback): ground every claim about the book in a retrieved passage, cite it by repeating its location label exactly, and never invent a label you were not given.

That last clause is not left to the model's good behaviour. Inkhaven scans the answer for citation-shaped tokens and checks each against the passages it actually retrieved. Any location the model cites that **was not** in what it was handed is flagged inline, right where it appears:

● ● ● *A hallucinated citation, caught and flagged*

She returns in `[act-two/the-storm]` and again later in `[act-three/the-reckoning]` `[citation could not be validated: act-three/the-reckoning]`.

The first citation was in the evidence, so it stands untouched. The second was not — the model reached past what it was given — and the flag says so plainly. A Book-scope answer whose citations are all clean is one you can trust down to the paragraph; one with a flag is telling you exactly which sentence to distrust. This is a structural guarantee, not a stylistic nicety: the check runs on every answer, and it leaves ordinary bracketed prose (a `[note]`, an `[aside]`) alone, because a real citation contains a slash and no spaces and an aside does not.

When nothing in your book addresses the question, the model is instructed to say so — “the retrieved passages don't address that directly” — and then either ask you to sharpen the question or offer general knowledge **clearly marked as not from the book**. It confabulates far less readily when the honest answer is built into its instructions.

Seeing what it retrieved — the transparency toggle

You never have to take the grounding on faith. Above the conversation, a collapsed line reports how many passages grounded the answer; press **p** to expand it (and **p** again to collapse). Expanded, it lists each passage's similarity score, a ★ for a **direct hit** versus a context-expansion neighbour pulled in around one, the passage's location path, and its opening line.

● ● ● *p expands the retrieved-passages transparency section*

```
▼ Retrieved passages (6) · p to collapse
★ 0.851 manuscript/act-ii/storm-at-sea
    On the ninth night the storm took them.
    Waves the height of..
  0.806 manuscript/act-i/the-harbour
    The harbour at Vell was a forest of masts...
...
(retrieved once for this chat – clear history to
retrieve again)
```

That closing line is the rule that keeps a conversation coherent. Retrieval runs **once per chat session**, so your follow-ups reason over a stable set of passages instead of yanking the ground out from under the thread with every question. The transparency section is the honest window onto that set: if an answer surprises you, expand it and see the evidence the model was actually standing on.

Re-grounding after you edit

Because a chat holds its retrieval, editing the book underneath a live conversation creates a small hazard — the passages the model has are now the **pre-edit** prose. Inkhaven watches for exactly this. Save an edited paragraph while a Book conversation is open and it nudges you, once:

● ● ● *A gentle nudge when the book moves under a chat*

```
book changed since retrieval – clear chat to re-ground
on the new text
```

It is a reminder, not an interruption. The open conversation stays valid and usable; when you want the model to see the new prose, clear the history and the next

question retrieves afresh. Re-grounding is always your call, never a surprise mid-thread.

What's in the pool — the retrieval scope

Retrieval is anchored to the **user book your cursor is in** — its chapters, subchapters, and paragraphs. But a question about your story often wants your **notes** about the story, so a curated set of author-content system books joins the pool. By default that means **Notes, Research, Places, Characters, Artefacts, World, and Language**. Ask “what is the significance of the brass key?” and a Characters or Artefacts entry can ground the answer right alongside the prose that mentions it.

The internal and meta system books stay out. Scripts, Prompts, Typst, Help, Intent, Sources, Glossary, and Snippets never enter retrieval — they hold the machinery of the project, not its content, and grounding a story question in a Bund script or a Typst template would only add noise. The split between “author content, searchable” and “project machinery, excluded” is what keeps a Book-scope answer about the **book**.

ONLY PARAGRAPHS GROUND AN ANSWER

Retrieval considers paragraph nodes only. A chapter or book node is a container, not prose, so the pool is always the actual writing — never a heading standing in for the scene beneath it. Each hit is expanded with its sibling paragraphs, so a retrieved passage arrives with the lines that surround it in the book.

Tuning retrieval — the `book_rag` config block

Every dial on the retrieval sits in one config stanza, `book_rag`. There is no on/off knob — Book scope **is** RAG — so the block only shapes **how** it retrieves, never **whether**. Five keys, each doing one thing:

● ● ● *The book_rag block, shown with its defaults*

```
book_rag: {
  top_k: 5
  context_expansion: 1
  max_context_tokens: 8000
  include_system_books: [notes research places
    characters artefacts world language]
  exclude_system_books: [scripts prompts typst help
    intent sources glossary snippets]
}
```

`top_k` is how many paragraphs the semantic search keeps as direct hits — raise it to ground a broad question more widely, lower it to keep the evidence tight. `context_expansion` is how many sibling paragraphs are pulled in **around** each hit (a value of `1` means one on either side), so a passage reads in context instead of as an orphaned line. `max_context_tokens` is the hard ceiling on the assembled evidence block, estimated at roughly four characters to the token; when hits would overflow it, the best-scoring ones win and the rest are dropped. The two book lists are named by each system book's lowercase tag: `include_system_books` is the author-content set that joins the manuscript in the pool, and `exclude_system_books` is the meta set that never does — and exclusion wins, so a tag in both lists stays out. Trim the include list to a leaner pool, or extend it if you keep story content in a book not listed by default.

THE PERMISSIVE PRINCIPLE, HERE TOO

None of these are guardrails. They shape cost and focus — a smaller `top_k` and a tighter token budget make a cheaper, narrower call — but nothing in `book_rag` will ever refuse a question or cap a conversation. As everywhere in Inkhaven, the numbers inform; they never block.

Inspecting retrieval from the terminal

Before you spend a single model call, you can see precisely what a question would retrieve — and confirm the grounding is sound — with `inkhaven book-rag retrieve`. It runs the **exact same** retrieval core the pane uses (they share one function, so they can never drift), but stops before the LLM: no answer is generated, no config is written, nothing is charged. It is the inspect-what-the-model-sees tool.

● ● ● *book-rag retrieve — the passages, no model call*

```
$ inkhaven book-rag retrieve \
  "what does Mara fear about the voyage?"
Book-RAG retrieval — `The Long Road`
(6 passages, 3 direct hits, ~288 tokens)

★ 0.806 manuscript/act-i/dawn-departure
    id: 019efc4b-ff42-7603-9a5d-4f83eedf50cb
    The ship slipped its moorings before sunrise...
  0.806 manuscript/act-i/the-harbour
    id: 019efc4c-05b9-7033-b7a2-7623b3112421
    The harbour at Vell was a forest of masts...
★ 0.851 manuscript/act-ii/storm-at-sea
    id: 019efc4c-0c27-7d61-9c1e-8df95471c961
    On the ninth night the storm took them...
```

Three flags shape the run. `--book-name <name>` picks the book in a multi-book project by title or slug (optional when there is only one). `--top-k <n>` overrides `book_rag.top_k` for this run only, leaving your config untouched — useful for feeling out how wide a question's grounding gets before you commit. And `--context` prints the literal composed evidence block the model would receive — passage headers and full bodies, the exact **— Retrieved passages —** prefix — rather than the summary listing. When you want to know why an answer came out the way it did, that block is the ground truth it was built on.

Facts scope — chat against the world's invariants

Cycle **F9** one past Book and you reach **Facts**, the odd sibling among the scopes. Where Book scope retrieves a **slice** of the manuscript, Facts scope loads the **whole** Facts system book — every established invariant of your world: its climate and geography, its distances and seasons, its chronology — as ground truth, and seeds the conversation with a fact-analysis system prompt. It is not a part of the book; it is the reference the whole book answers to, which is why it sits **after** Book on the ring rather than among the manuscript slices.

Reach for Facts scope when your question is about the world's rules rather than its prose. “How long is the ride from Vell to Rillmark?” “Which season is it when the fleet sails?” “Is it plausible for snow to fall in the southern reach?” The answer is grounded in what you have **established**, so the model reasons from your canon instead of a genre cliché.

TWO SIBLINGS SHARE THE FACTS GROUNDING

The same fact grounding powers two nearby chords. **Ctrl+B Shift+X** runs a one-shot **fact-check** of the open paragraph against every world fact, flagging any claim that contradicts the canon. **Ctrl+B Shift+S** opens a semantic **Facts search**: query the Facts book, mark the handful that matter, and send just those into a targeted chat — the scalable path when the Facts book grows too large to load whole. Both reuse the fact-analysis grounding the **F9** Facts scope is built on. The world, and everything you can do with it, has its own chapter.

Graph scope — chatting with how your book connects

The last stop on the ring is **Graph**, and it answers a different kind of question than any other scope: not what your book **says** but how its parts **connect**. It is the conversational face of the knowledge graph — the typed-edge layer over your nodes that records which fact contradicts which source, which paragraph links to which, how a scene is sourced. A flat manuscript cannot tell you what contradicts what; the graph can, and Graph scope lets you ask it in plain language.

A Graph-scope question retrieves the relevant passages exactly as Book scope does — same semantic search, same location-path citations, same hallucination

flag — and then, beneath each passage, folds in the graph edges touching it: `contradicts`, `sourced_from`, `links_to`, `cites`, and the rest. The answer stands on both the prose **and** those relations. Like Facts, it is a sticky scope that retrieves once per chat and re-grounds when you clear history, and pressing `p` expands a transparency section showing the passages **and** the subgraph the answer was built on.

Walking the graph — one hop, or many

Graph scope reads **one hop** — the edges immediately touching each retrieved passage. When a question needs the model to **follow** the graph — chaining from node to node, several relations deep — there is a heavier instrument: `graph ask`, the traversal loop.

● ● ● *graph ask — the model walks the graph turn by turn*

```
$ inkhaven graph ask \  
  "which of my claims about the harbour  
  contradict each other?"  
  
(walk prints to stderr; the answer to stdout)
```

Given a question, `graph ask` searches for seed nodes, then issues read-only graph queries turn by turn — neighbours, contradictions, loci, paths — until it can answer, grounding the reply in what it actually observed. The exploration transcript prints to stderr so you can watch the path it took; the answer prints to stdout so you can pipe it. It is honest by construction: when the relations do not record what you asked, it says so rather than inventing a connection — the graph is only as complete as you have made it.

The same walk runs **inside the editor**, streamed live. Type a question in the AI prompt, then press `Ctrl+B z → w`, and the AI pane shows each step unfold — a search, a neighbours query, a contradiction check — before the grounded prose answer lands as a normal chat turn; `Esc` stops the walk at any point. Because a walk is several model calls where the one-hop Graph scope is one, it is an explicit action you opt into per question. Both the CLI and the in-editor walk are bounded by two knobs — `ask_max_steps` (the most LLM turns before a forced answer) and `ask_search_width` (seed nodes per search) — which, true to the permissive principle, cap the cost of a question without ever refusing it.

The reader conversations — Socrates and Editor

Two scopes on the ring are not about grounding in data at all but in a **perspective**. Cycle past Facts and you reach **Socrates**, then **Editor** — the conversational modes of two of Inkhaven’s inner readers. Where Book, Facts, and Graph ground the model in your **material**, these ground it in a particular way of **reading** that material, seated across the table to talk with you about the paragraph in front of you.

Socrates scope seats the active Reader Persona. The model reads the open paragraph in that persona’s voice, carries in whatever questions the Inner Socrates has already raised about it, and discusses them with you — asking, never prescribing, and never rewriting your prose. It is the scope for interrogating a passage’s assumptions rather than polishing its surface.

Editor scope seats the Inner Editor instead. It carries in that reader’s literary and stylistic observations of the open paragraph — its notes on richness, tautology, vocabulary, and craft — and talks them through with you as a thoughtful editor would, observing rather than commanding. When you cycle into either scope, the status bar names it and tells you how many of that reader’s findings are queued for the conversation; both are sticky, and re-cycling **F9** refreshes the seed from the paragraph you are now on.

THE READERS HAVE THEIR OWN CHAPTERS

Socrates and Editor scopes are the **conversational** face of the Inner Socrates and Inner Editor. What those readers notice, how they engage a paragraph, and how you tune them each fill their own chapters. Here it is enough to know that **F9** seats them for a chat: the same reader you meet as a pass over your prose, now willing to talk about what it found.

Living with the chat — cost and habits

Every structured-scope answer is one retrieval (cheap and local — embeddings and a vector query, no network) plus one grounded model call, far less than shipping a whole manuscript ever cost. Those calls are tagged in the cost dashboard — `inkhaven cost`, or **Ctrl+B \$** — under the `book_rag` category, so you can see

across a day what chatting with your book actually costs. As with every figure in Inkhaven, it **informs**; no cap will ever refuse a question.

A few habits make the whole surface fluent. Cycle **F9** deliberately and read the scope off the status bar before you send — the same question means very different things in Book, Facts, and Graph. Press **p** when an answer surprises you and read the evidence it stood on. Clear the history when you change subject or edit the prose, so the next question re-grounds. And cycle **F9** back toward **None** when you want the model's plain help with no grounding at all — the far end of the dial is always one key away.

F9	Cycle the AI scope one notch; the status bar names it.
F9 → Book	RAG chat over the manuscript + author-content system books.
F9 → Facts	Chat grounded in the whole Facts book (the world's invariants).
F9 → Graph	Chat grounded in prose + the graph edges that connect it.
F9 → Socrates	Converse with the active Reader Persona about this paragraph.
F9 → Editor	Converse with the Inner Editor about this paragraph's craft.
p	In the AI pane: expand / collapse the retrieved-passages section.
Ctrl+B z → w	Walk the graph to answer the AI prompt, streamed live.
Ctrl+B Shift+X	One-shot fact-check of the open paragraph against the Facts book.
Ctrl+B Shift+S	Semantic Facts search → ground a chat in a chosen handful.
Ctrl+B \$	The cost dashboard — see the book_rag category's spend.

Scripting retrieval — the `ink.book_rag.*` words

The same retrieval that grounds the pane is reachable from a Bund script, so an automation can retrieve passages, compose the exact grounding block, inspect the config, and even validate an answer's citations without touching the TUI. Every word is **read-only** — nothing here mutates the store or calls a model; the retrieval is local, and generating the answer is left to the script author. All eight gate under the default-allowed `store_read` category, so a cautious project can disable the whole surface at once.

● ● ● *Retrieve and compose the grounding block from Bund*

```

"manuscript" "does Mara doubt the captain?"
  ink.book_rag.context .          # the composed evidence block

"manuscript" "the harbour market"
  ink.book_rag.retrieve          # list of {id, breadcrumb,
                                #          body, score,
is_hit}
  dup ink.book_rag.cited_ids     # the valid citation tokens

```

`ink.book_rag.retrieve` takes an anchor (any book, or a node inside one) and a query and returns the passages as dicts; `ink.book_rag.context` returns the composed grounding block a model would receive; `ink.book_rag.scope` returns the node ids in the retrieval pool; and `ink.book_rag.config` returns the live `book_rag` block as a dict. Four pure helpers round it out: `ink.book_rag.system_prompt` gives the localised grounding contract for a language, `ink.book_rag.estimate_tokens` the rough token count of a string, `ink.book_rag.cited_ids` the valid citation tokens of a passage set, and `ink.book_rag.validate_citations` runs the same hallucination check the pane runs — flagging, in a candidate answer, any bracketed location the passages do not support. The knowledge-graph side is scriptable too, through the `ink.graph.*` words, though the LLM-driven `graph ask` is not a synchronous word.

WHAT YOU LEARNED

- The AI pane's **scope**, cycled with **F9**, decides what an answer is grounded in. The first five stops send a **slice of prose**; the last five — **Book, Facts, Graph, Socrates, Editor** — are **structured scopes** that retrieve or assemble evidence and are **sticky** until you cycle away.
- **Book scope** is retrieval-augmented generation: it semantically searches your manuscript, keeps the top passages, expands and token-budgets them, and grounds the answer with **location-path citations** — flagging inline any citation the passages do not support.
- Retrieval runs **once per chat**; press **p** to see the passages it stood on, and clear the history to **re-ground** after you edit — Inkhaven nudges you once when the book changes under a live conversation.
- The pool is your **user book** plus the **author-content system books** (Notes, Research, Places, Characters, Artefacts, World, Language); the meta books are excluded. Tune it all in the **book_rag** block — **top_k**, **context_expansion**, **max_context_tokens**, and the two book lists.
- **inkhaven book-rag retrieve** shows exactly what a question would retrieve with **no model call**; **--top-k** and **--context** inspect the grounding, and the CLI shares the pane's retrieval core so they never disagree.
- **Facts** scope grounds in the world's invariants; **Graph** scope grounds in how the book's parts connect (with **graph ask / Ctrl+B z → w** to **walk** deeper); **Socrates** and **Editor** seat an inner reader for a conversation about the open paragraph.
- The whole surface is scriptable through the read-only **ink.book_rag.*** Bund words — **retrieve**, **compose** the grounding block, **inspect** the config, and **validate citations** — and its cost is tagged under **book_rag**, informing but never blocking.

CHAPTER 11

11

Reusable Prompts

You will find yourself typing the same instruction to the assistant again and again. *Tighten this without changing the meaning. Make the tone darker but keep every fact. Read this paragraph for places I am telling instead of showing.* A good instruction is a small piece of craft — it took you three tries to word it so the model stops adding new plot — and retyping it from memory each time is both a chore and a slow erosion of the phrasing that worked. Inkhaven's answer is the **reusable prompt**: write the instruction once, give it a short name, and from then on summon it in two keystrokes, with the passage you are working on already stitched in. This chapter is the whole of that system — where prompts live, how you reach them, what gets

substituted into them, and how the same machinery quietly powers the assistant's built-in passes like grammar-check and fact-check.

Why keep a prompt around

A reusable prompt is nothing more than a saved block of text with two conveniences bolted on. First, it has a **name** you can type — `/tighten`, `/darker` — so you never scroll a history buffer looking for the wording you liked. Second, it carries **placeholders** that Inkhaven fills in at the moment you send it, so the same template works on whatever paragraph is open without you pasting anything. Between those two, a prompt you tuned once becomes a verb: you *tighten* a passage the way you *save* a file.

TERM Prompt

A named, reusable instruction to the AI assistant. It is plain text plus the placeholders `{{selection}}` and `{{context}}`. When you dispatch it, the placeholders are replaced with the passage you are working on and its location in the book, and the result lands in the AI prompt bar ready to send — nothing is sent until you press Enter.

Prompts are advisory, like everything the assistant does. Expanding a prompt fills the input bar; it never edits your prose on its own. You see the fully substituted text, you can edit it, and only your Enter sends it. That is the same contract the whole AI layer honours (Chapter 9): the model proposes, you dispose.

Two libraries: the file and the book

Inkhaven draws prompts from two places at once, and the picker shows them merged into a single list.

The first is a plain file, `prompts.hjson`, in the root of your project. These are your **system prompts** — portable, versionable templates you carry between books. `inkhaven init` seeds the file from a shipped default, and you edit it with any text editor (or the built-in editors described later). In the picker they wear a cyan `[ system ]` chip.

The second is a system book named **Prompts**, sitting in the Tree alongside Notes, Facts, and the rest. Every paragraph you file under it becomes a prompt. These are your **book prompts** — project-local, authored in the TUI like any other paragraph, and carried inside the manuscript itself. They wear a green [book] chip.

TERM The Prompts system book

One of Inkhaven's built-in system books. Paragraphs nested under it are not prose — they are prompt templates. A paragraph's **slug** supplies its `/name`; its **title** supplies the description shown in the picker; its body is the template. System books are excluded from the manuscript corpus (word counts, concordance, exports), so your prompts never leak into the book.

The two libraries serve different instincts. Keep in `prompts.hjson` the **signature** templates you want in every project — your house style for tightening, your favourite critique framing. Keep in the Prompts book the templates that only make sense *here*, because they lean on this manuscript's voice, its glossary, its world. When the two disagree — a `/tighten` in both — the book copy wins, on the principle that the project-local, hand-authored version is the more deliberate one. The exact ladder is spelled out under *Localized prompts* below, because it interleaves with language; the short form is **Prompts book, then `prompts.hjson`, then Inkhaven's embedded default.**

Opening the picker

There is no chord for the prompt picker, and you do not need one: it is bound to the character that already means “I am about to name something,” the forward slash. Focus the AI prompt bar with `Ctrl+I` and type `/`. A floating magenta panel opens just above the bar, listing every prompt from both libraries. Keep typing and the list filters live — the text after the slash is matched, case insensitively, against both the name and the description of each prompt.

● ● ● The prompt picker, filtered to `/ti`

```
+-- Prompts -----+
| [ system ] /tighten |
|           Tighten the prose without changing meaning |
| [ book ]   /tighten-quay-voice |
|           Tighten, in the harbour-chapter register |
| [ system ] /timeline-health |
|           Review the story timeline for consistency |
+-----+
AI prompt > /ti|
```

The panel is routed to the prompt bar — it has no separate focus, so the same keys that edit the bar also drive the list.

- Ctrl+I** Focus the AI prompt bar (from any pane).
- /** As the first character, opens the picker; refiltered as you type.
- Up / Down** Move the selection through the list.
- Enter / Tab** Expand the selected template into the bar (substituted).
- Esc** Close the picker without expanding anything.

Selecting a prompt with Enter or Tab does **not** send it. It expands the template — running the substitutions, stripping any editor chrome — and drops the finished text into the prompt bar, leaving the picker. Now you can read exactly what the model will receive, edit it (append a one-off note, delete a clause), and press Enter when you are satisfied. The status line confirms the pick: `loaded prompt tighten [system] – Enter to send`.

When the filter is empty the list keeps its natural order — system prompts first, then book prompts. As you type, matches are ranked: a prompt whose **name** begins with what you typed sorts above one where a later **description** word begins with it, which sorts above a mere substring hit anywhere. So `/ti` surfaces `tighten` before it surfaces a prompt merely *described* as “review the timeline.”

Direct invocation: slash a name

Once a prompt's name is in your fingers, you can skip the picker entirely. Type the whole name yourself and send:

● ● ● *Dispatching a prompt by name*

AI prompt > /tighten

Enter looks the name up — system library first, then the Prompts book by slug or title — expands the template against the current selection, and sends in one motion. If nothing matches, the input is left alone and the status line says **no prompt** tighten — type / to see the list, so a typo never fires a blind inference.

Two subtleties are worth knowing. First, text you type *after* the name is ignored by the resolver — the template is what gets sent, not your trailing words. To add a one-off note, expand the prompt into the bar (Enter inside the picker) and edit it there before submitting. Second, two prefixes are special and are **not** prompts: a leading **Help!** (case-sensitive) routes the rest of the line through the grounded Help search (Chapter 9), and a bare line with no leading slash is simply sent as an ordinary question.

The substitutions

A template is plain text until you dispatch it, at which point Inkhaven replaces two placeholders. There are exactly two — no others — and both are filled from the editor state at the instant you expand the prompt.

TERM **{{selection}}**

The text you are working on. If you have an active selection in the editor, that selected span is substituted verbatim. If nothing is selected, the **entire open paragraph** is substituted instead. This is the working text most prompts operate on, so a prompt “just works” whether you highlighted a sentence or want the whole paragraph.

TERM {{context}}

A breadcrumb of titles locating the open paragraph in the book, joined with `>` — for example `Rain > The City > the-quay`. It lets you tell the model *where* in the manuscript you are without flooding it with the surrounding prose. Cheap orientation, a few tokens, no retrieval.

You can use either, both, or neither. A template frames them however you like; a common shape wraps the working text in visible fences so the model can see exactly where it starts and stops:

● ● ● *A template using both placeholders*

```
Context: {{context}}
```

```
Tighten the passage below without changing meaning or
voice. Aim for 10-20% fewer words. Keep Typst markup
verbatim.
```

```
--- begin
{{selection}}
--- end
```

The substitution happens once, in memory, the moment you expand or dispatch the prompt. What the model receives is finished plain text with the placeholders already gone; the leading and trailing whitespace of the whole template is trimmed. Inkhaven never rewrites your `prompts.hjson` or your book paragraphs to do this — the source templates are untouched.

SELECTION BEATS PARAGRAPH

Because an empty selection falls back to the whole paragraph, you rarely need to select anything for a paragraph-level task. Reach for a selection only when you want the model to work on *part* of the paragraph — a single line of dialogue, one clause — and leave the rest out of its view.

System prompts in prompts.hjson

The system library lives at `<project-root>/prompts.hjson`. It is HJSON — JSON with comments, unquoted keys, and triple-quoted multi-line strings — and its shape is a single `prompts` array of entries:

● ● ● *prompts.hjson — one entry*

```
{
  prompts: [
    {
      name: "darker"
      description: "Make the tone darker, keep facts"
      template: '''
        Rewrite the passage in a darker tone -- somber
        palette, unease, understated dread. Keep every
        fact, name, and timeline intact. Preserve Typst
        markup.

        Context: {{context}}

        --- begin
        {{selection}}
        --- end
      '''
    }
  ]
}
```

Each entry has three required fields and one optional one.

name	The /name identifier. Lowercase, no spaces; hyphens are fine.
description	The one-line blurb shown in the picker. Keep it scannable.
template	The prompt body. Use <code>'''</code> triple-quotes for multi-line text.
language	Optional ISO 639-1 code (en, ru, fr, de, es) — see below.

To add a prompt, append an entry to the array and save. Inkhaven reads `prompts.json` when the TUI launches; it does **not** watch the file for live edits, so close and reopen the project (or restart the binary) to pick up a change. A malformed file is reported at load and the library falls back empty rather than crashing — fix the HJSON and relaunch.

Because the file is plain text in your project root, it versions with the rest of your work: commit it, diff it, branch it. That is exactly why it is the right home for the templates you want to *travel* between books.

Book prompts in the Prompts book

A book prompt is authored the way you author anything in Inkhaven — as a paragraph. Navigate to the **Prompts** system book in the Tree, add a paragraph (+, or **P** to insert after the cursor), give it a title in the Add modal, and write the body in the editor. Save with **Ctrl+S** and it appears in the picker immediately — no restart, because the book prompts are read live from the store on every keystroke in the picker.

Two fields fall out of the paragraph automatically. Its **slug** — the filesystem-safe form of the title — becomes the `/name`. Its **title** becomes the picker description. A title of `Worldbuilding consistency pass` slugs to `worldbuilding-consistency-pass`, so typing `/world` filters straight to it.

THE HEADING IS STRIPPED FOR YOU

Inkhaven seeds a new paragraph with a = Title Typst heading line. When a book prompt is expanded, that leading heading — and the blank lines after it — are removed before the template is used, so the model never sees editor chrome, only your instruction. Write the prompt body under the heading and forget the heading is there.

A book prompt earns its place over a file entry in four ways. It **travels with the manuscript**: anyone who clones the project gets your prompts for free, no side file to remember. It is **editable in the TUI**, with no round-trip through an external editor. It is **indexed like prose**, so semantic search can surface it. And it is **snapshot-able** — **F5** captures a version before you tune a prompt, **F6** picks an older one back out — so your best wording is never one careless edit from lost.

Localized prompts

Every AI feature in Inkhaven keys off the project’s working language, and prompts are no exception. A prompt can be tagged with the language it was written for, and the resolver prefers the tag that matches the language you are writing in.

On a `prompts.hjson` entry the tag is the optional `language` field, an ISO 639-1 code. On a book prompt it is a `lang:<code>` tag on the paragraph — for instance `lang:ru`. Untagged prompts (every prompt from before this feature existed, and every one you never bother to tag) are treated as language-neutral and always remain eligible.

When a name is resolved — whether from `/name`, the picker, or a built-in pass — Inkhaven walks three passes, and within each pass it checks the Prompts book before `prompts.hjson`, and tries both the hyphenated and the spaced form of the name (`grammar-check` and `grammar check`):

● ● ● *The three-pass resolution ladder*

```
Pass 1  strict  prompt tagged for the ACTIVE language
                  Prompts book -> prompts.hjson
Pass 2  untagged prompt with NO language tag (back-compat)
                  Prompts book -> prompts.hjson
Pass 3  any     prompt tagged for ANY other language
                  Prompts book -> prompts.hjson
                  ... then the embedded default (English
floor)
```

So a Russian `grammar-check` beats an untagged one, which beats an English-tagged one, which beats Inkhaven’s built-in English fallback — and at every rung a book prompt outranks a file prompt. This is the full form of the *book, then file, then embedded* precedence mentioned earlier: language sorts first, then source.

Which language counts as “active” is itself controllable. `Ctrl+B Shift+N` cycles the prompt-language mode through three settings: deferring to the `editor.prompt_language_mode` value in your project HJSON, using the project’s declared `language` for every resolution (`book_defined`), or detecting the language of the live paragraph and resolving to *that* (`paragraph_detected`). The AI pane’s title bar shows a `lang=` chip — `ru (book)` versus

`ru (paragraph)` — so you always know which language the next prompt will resolve against.

BOOTSTRAPPING A WHOLE LANGUAGE AT ONCE

You do not have to translate the built-in prompts by hand. From the command line, `inkhaven prompts bootstrap russian` asks your configured model to produce Russian variants of the seven embedded prompts and prints a ready-to-paste HJSON snippet; add `--update` to merge them into `prompts.hjson` in place (with a timestamped backup, never clobbering your own entries). Covered under *From the command line* below.

Named flows and the resolution ladder

The same resolver that answers `/tighten` also supplies the prompts behind Inkhaven's built-in AI passes — and that is deliberate, because it means every one of them is **customisable** by exactly the mechanism you have just learned. A pass looks up a well-known name, and if you have defined a prompt of that name (in the Prompts book, or in `prompts.hjson`, in the language you are working in), yours is used instead of the shipped default.

The clearest example is `fact-check` (`Ctrl+B Shift+X`, Chapter 14). It resolves the name `fact-check` through the ladder: a `fact-check` paragraph in the Prompts book, then a `fact-check` entry in `prompts.hjson`, then the embedded multilingual default. Override the first two to bend the house style of the check; leave them absent and the built-in prompt runs. Grammar-check (`F7`) works the same way against the name `grammar-check`, as do the show-don't-tell scan, the sentence-rhythm rewrite, the diagnostic explainer, the critique passes, and the timeline-health review.

● ● ● *The eight embedded prompt names you can override*

grammar-check	F7 copy-edit pass
show-dont-tell	Ctrl+B Shift+T telling-vs-showing
scan	
sentence-rhythm-rewrite	Ctrl+B Shift+M rhythm rewrite
explain-diagnostic	Ctrl+F12 explain a Typst error
critique-edit	F12 weakest-elements critique
critique-changes	F12 evaluate a revision
critique-compare	F12 compare two paragraphs
timeline-health	Timeline modal consistency review

Two flows deliberately stand outside this system. The Help question (F1, or a **Help!** prefix) uses a fixed strict-grounding prompt pinned to Local mode, so the model can never invent a feature — it is not yours to override. And the grammar-check apply key expects the corrected text between `<<<CORRECTED>>>` / `<<<END>>>` markers; if you *do* override the `grammar-check` prompt, keep those markers in your version or the one-key apply will have nothing to extract.

From the command line

Two subcommands touch prompts from outside the editor.

`inkhaven prompts bootstrap <language>` is the localizer described above. It sends the eight embedded prompt names to your configured model, asks for faithful variants in the target language (preserving Typst tokens and the `{{selection}}` / `{{context}}` placeholders verbatim), and prints an HJSON snippet to standard output. Nothing is written to your project unless you pass `--update`, which merges the results into `prompts.hjson` in place — overwriting only same-name, same-language entries, appending the rest, and leaving a time-stamped backup under `.config-backups/`. An optional `--genre` biases word choice toward the register you write in.

`inkhaven prompts-editor` launches a standalone four-pane workbench for `prompts.hjson` alone: the prompt list on the left, an editor with the same chord set as the main paragraph editor in the centre, an AI response pane on the right, and a prompt bar across the bottom — a focused place to draft and review your file prompts without the whole manuscript around them.

EDITING THE FILE INSIDE THE TUI

You can also open `prompts.hjson` — and the rest of the project config — from inside a running session with `Ctrl+B 0`, the in-app HJSON editor. It syntax-highlights, saves atomically, and warns you that prompt changes take effect on the next launch. For a book prompt there is no such delay: it is live the moment you save the paragraph.

Scripting: setting the system prompt

There is no `ink.` Bund word for the prompt *picker* — the picker is a keyboard affordance, not a scripted surface. The one prompt-adjacent scripting hook is `ink.ai.set_system_prompt`, which sets the **system** message the assistant runs under (distinct from the reusable templates this chapter is about). It takes one string off the stack:

● ● ● *Overriding the system prompt from Bund*

```
"You are a terse copy-editor. Never explain."
ink.ai.set_system_prompt

"" ink.ai.set_system_prompt \ empty string clears it
```

An empty string clears the override and the assistant falls back to the default system prompt for the current inference mode (Local versus Full, Chapter 9). This is a runtime, session-local change — it is not persisted, and it does not touch your prompt libraries. Use it in a Bund macro that puts the assistant in a specific frame of mind before a batch of work; reach for reusable prompts when you want a named, substitutable *instruction* instead.

How prompts sit beside scope and mode

Reusable prompts are one of three orthogonal dials on what the assistant sees, and it helps to hold them apart. A **prompt** supplies the instruction and, via its placeholders, the working text and its location. **Scope** (`F9`) prepends a separate context

block — the selection, the paragraph, the chapter, the whole retrieval-grounded book — *above* your prompt; the model sees both. **Inference mode** (F10) swaps the underlying system prompt, forbidding or permitting general knowledge.

For a one-shot task — tighten this, run a grammar check — the prompt’s own `{{selection}}` is usually all you need, and you can leave scope at None. For an ongoing conversation about the work — brainstorm a subplot, plan a chapter — lean on scope and let chat history accumulate instead, clearing it with `Ctrl+B C` when you want a fresh thread. Prompts and scope compose cleanly; you will reach for one, the other, or both as the task asks.

WHAT YOU LEARNED

- A **reusable prompt** is a named instruction with two placeholders; expanding it fills the AI prompt bar but never sends until you press Enter.
- Prompts come from two libraries: portable **system prompts** in `prompts.hjson` (cyan `[ system ]`) and project-local **book prompts** under the Prompts system book (green `[ book ]`).
- Open the picker by focusing the bar with `Ctrl+I` and typing `/` — there is no chord; `Enter/Tab` expand, `Esc` closes — or dispatch a template directly by typing `/name`.
- There are exactly two substitutions: `{{selection}}` (the selection, or the whole open paragraph) and `{{context}}` (the title breadcrumb).
- Resolution runs **Prompts book, then prompts.hjson, then the embedded default**, and within that a language ladder of `strict` → `untagged` → any keys the active-language variant first.
- Built-in passes like `fact-check` and `grammar-check` resolve named prompts through the same ladder, so you can override any of them; `inkhaven prompts bootstrap` and `inkhaven prompts-editor` manage the file from the command line.

CHAPTER 12

12

Watching the Cost

Every tool that talks to a large language model spends money doing it, and most of them are quiet about it — the meter runs somewhere you cannot see, and the bill arrives at the end of the month with no way to trace which feature ate what. Inkhaven takes the opposite stance. It keeps almost everything free, makes the handful of paid touchpoints explicit, records every model call the moment it happens, and gives you one screen that shows the day's spend at a glance. This chapter is about that visibility: the philosophy behind it, the dashboard that surfaces it, the daily caps that warn you before a big pass, and exactly which features cost anything at all. Read it once and you will never again wonder where your model budget went.

Cost informs, it never blocks

The single principle that governs money in Inkhaven is the one that governs everything else in the tool: it is a **permissive** program built for one writer who owns their own decisions. It informs; it does not gate. There is no feature Inkhaven will refuse to run because you have “used too much” today, and no hidden ceiling that silently drops a request. When you cross a budget, the tool tells you plainly and then does the thing you asked for anyway.

This is a deliberate reversal of how quotas usually work. A daily cap in Inkhaven is not a wall — it is a **speed bump**. It exists so that a runaway loop, or a habit you did not notice forming, becomes visible before it becomes expensive. The number in the config is a threshold for a warning, not a limit on your agency.

THE PERMISSIVE PRINCIPLE

Cost caps **inform**, they never **block**. Past a daily budget the model-using passes print a one-line notice and continue. The only things Inkhaven ever truly refuses are matters of safety — a sandbox boundary, a destructive operation — never a matter of spend. Your money is your business.

Two consequences follow, and they run through the rest of the chapter. First, because nothing is blocked, the caps can afford to be honest rather than conservative — they are set where a normal day of writing never reaches them, so that touching one actually means something. Second, because the tool never surprises you, it has to be scrupulous about **showing** you: every inference is counted, every counted call is visible, and the dashboard is one keystroke away.

Almost everything is free

The reason the bill stays small is architectural, not accidental. The overwhelming majority of what Inkhaven does for you is **deterministic** — it runs locally on your machine, from your text and your data, with no network call and no model behind it. Deterministic work is free, it is instant, it works with no network, and — the quiet virtue — it is **reproducible**: run it twice on the same input and you get the same answer, because there is no model in the loop to drift.

TERM Deterministic

A computation whose output is fixed by its input — the same paragraph in gives the same result out, every time, computed on your machine with no model and no network. The opposite of a **model call**, which asks a language model for a judgement and costs money to make. Inkhaven's default posture is deterministic; the model is reached only where a determinate answer is genuinely impossible.

Look across the reading intelligences and you find the same pattern again and again: a free deterministic **core** that runs all the time, and an optional model **pass** you invoke by hand when you want the deeper reading the patterns cannot give. The continuity ledger checks co-location, timeline, numbers, and character-fact drift with no model at all; its cross-paragraph coherence pass is a separate key you press only when you want it. The read-through measures your prose's intensity curve and walks the book for confusion and info-dumps deterministically; its synthetic first-read is opt-in. The voice profiler, the poetry scanner, the theologian's fast signals — all free, all local, all reproducible.

So the honest one-sentence summary of Inkhaven's cost is this: **the tool is free until you deliberately ask a model a question**. The rest of this chapter is about those deliberate asks — where they are, what they cost, and how to keep them in view.

Where the money goes

There is no long list to memorise, because the paid surfaces are few and each one announces itself. Every model touchpoint in Inkhaven is one of two shapes: a **slow track** you engage on a paragraph or a book, or a **conversation** you type into. Here is the whole territory.

The slow tracks

A slow track is the model pass that sits behind a deterministic reader and supplies the judgement the patterns cannot. Each is opt-in — you press its engage key or pass its flag — and each is capped and counted.

- **World fact-check (slow)** — reads a paragraph against your world's declared facts for contradictions a rule cannot catch. Default cap: 200 calls a day.

- **Inner Socrates (slow)** — the **J** → **E** engage pass, which surfaces the assumptions, tensions, and framings a paragraph rests on as questions. Default cap: 150 calls a day.
- **Inner Editor engagement** — the editor’s own coaching pass. Default cap: 200 engagement calls a day (and a session and monthly ceiling besides).
- **The coherence and synthetic passes** — the continuity ledger’s **k** coherence pass, the read-through’s **k** synthetic first-read, the Inner Stylist’s **E** coaching, the Inner Theologian’s session, and a **--deep** flag on the CLI where one exists. Each is a small number of model calls you asked for by name.

Every one of these is a single short model call per paragraph — one request, one response. With the default caps, an ordinary writing day never comes near the ceiling; you would have to engage the slow track on a hundred and fifty separate paragraphs to reach the Inner Socrates cap, and the tool would still let you.

The conversations

The other paid surface is anything you type into and get an answer back from: the AI pane’s chat, chat-with-your-book, the research assistant’s session, grammar and explain and continuation on a selection, the graph’s **ask**. These are not capped by a daily ceiling — they are metered instead, counted by **category** so you can see how many of each you have run today. The research assistant goes one step further and shows a running dollar estimate in its status bar as you work, priced from the model you are actually using.

WHAT A MODEL CALL COSTS

Inkhaven ships a per-model price table (USD per million tokens) so the research assistant can turn tokens into dollars — a Sonnet-class model is priced around \$3 in / \$15 out per million, a Haiku-class one far less, an Opus-class one more. A single slow-track paragraph pass is a few hundred tokens; the arithmetic is pennies. The caps exist to catch a **runaway**, not a normal session. Prices are informative and live in config — adjust them as the market moves.

The cost dashboard

Everything counted lands in one place. There are two doors to it — a command and a keystroke — and both render the exact same view, because the CLI and the modal share a single aggregator under the hood.

From a shell, `inkhaven cost` prints today's tally and exits. Inside the editor, the chord `Ctrl+B $` opens the same report as a scrollable modal titled **AI cost · Ctrl+B \$**. The report is strictly read-only: it changes no caps, enforces nothing, and computes its numbers fresh each time you open it.

Ctrl+B \$ Open the AI cost dashboard modal (read-only).

The report is in three bands. First the **daily budgets** — the capped slow tracks, each shown as calls-used over its cap with a twenty-cell usage bar and a percentage. Then **other AI calls** — every uncapped conversation, listed by category with a count and no bar, because there is nothing to measure against. Last a **total** line and the standing reminder that the budgets inform rather than limit.

● ● ● *inkhaven cost — a day's tally*

```
AI cost – LLM calls today (2026-08-05)
```

```
  daily budgets (informative):
```

```
    world fact-check (slow)    12 / 200
    [██████████████████] 6%
    inner socrates (slow)      3 / 150
    [██████████] 2%
```

```
  other AI calls today:
```

```
    chat                      7
    explain                   2
    grammar                   1
```

```
total AI calls today          25
```

```
Budgets are informative, not limits – past a budget the slow
tracks warn and continue. The tally resets per
goals.day_boundary.
```

A few things are worth reading off that screen. The bar fills proportionally and clamps at full, but the percentage always tells the truth — cross a cap and you will see the bar solid with, say, `150%` beside it, which is exactly the signal you want. The capped rows and the counted rows never double-count: the slow tracks record only against their own budget, so they do not also appear in the per-category list below. And the dashboard is **extensible by design** — any new analytical thread that

records its calls under its own sub-budget key shows up here automatically, with no change to the dashboard itself. The Inner Socrates and Inner Editor entries you see are exactly that mechanism: sub-budgets keyed by feature, surfaced dynamically.

ONE AGGREGATOR, TWO FACES

`inkhaven cost` and `Ctrl+B $` are the same code. The CLI is for a quick check from a shell or a script; the modal is for a glance without leaving your paragraph. Neither can change anything — to move a cap you edit config (below), and the next report reflects it.

The daily caps

The caps are the numbers the dashboard measures against, and understanding what they do — and do not — is the heart of the chapter. A cap is a per-day ceiling on one slow track’s model calls. It is **informative**: the tool checks it before a call, and if you are over it, it warns and proceeds.

TERM Daily call cap

A per-day threshold on a slow track’s model calls — 200 for world fact-check, 150 for Inner Socrates, 200 for Inner Editor engagement by default. Crossing it triggers a one-line warning, not a refusal. “Today” is defined by `goals.day_boundary`, so the caps, the streak, and the usage tallies all agree on when the day rolls over.

Before a slow-track call, a **preflight** estimates the request’s size and checks your standing against the cap. When you are under budget, it prints a quiet line naming the model, the rough token count, and where you stand — then reads:

● ● ● *A preflight, under budget*

```
inner socrates (slow) · model: claude-sonnet · ~420 tokens
· 3/150 calls today · reading...
```

When you are **over** budget, the preflight does not stop. It prints the notice and continues into the very same call — the permissive principle made literal:

● ● ● *A preflight, past the cap — it continues*

```
inner socrates (slow): past today's slow-track budget
(150/150 calls) – continuing (the cap is informative,
see `inkhaven cost`).
```

That is the whole behaviour: a warning, a pointer to the dashboard, and the work done regardless. There is a second, separate guard worth knowing about — a per-call **soft cap** on estimated tokens, which catches a single pass that is unusually large (a very long paragraph, say) and asks you to confirm with `--force`. That one can decline a call, because its job is to stop a single accidental monster request, not to ration your day; but it too yields the moment you insist.

THE ONE EXCEPTION, AND IT IS NOT A WALL

The per-call soft cap is the only place a model pass will pause rather than proceed, and it pauses for size, not for your daily total. Re-run with `--force` and it goes through. The daily caps never pause at all.

The ``cost`` config block

The caps and the retention window live in one small config block. Edit your project's config to move any of them; the change takes effect on the next report and the next preflight.

● ● ● *The cost block — defaults shown*

```
cost: {
  world_daily_call_cap: 200           // world slow-track
  ceiling
  inner_socrates_daily_call_cap: 150  // Inner Socrates
  ceiling
  usage_retention_days: 30            // days of tallies kept
  in                                  // .inkhaven/
  ai_usage.json
}
```

The Inner Editor keeps its own richer set of ceilings under its own block — a per-session cap, a “confirm above” threshold, a per-day cap of 200, and a monthly cap — because a coaching conversation has more shapes to bound than a one-shot paragraph pass. All of them are informative in the same way.

Two of these numbers deserve a note. `usage_retention_days` governs how much history the per-category tally file keeps before the oldest days are pruned — it is a housekeeping bound on a small JSON file, not a spend control. And the day boundary that all the caps share is `goals.day_boundary: utc` by default, or `local` so an evening session far from UTC is attributed to the right day and the caps reset at your midnight, not Greenwich’s.

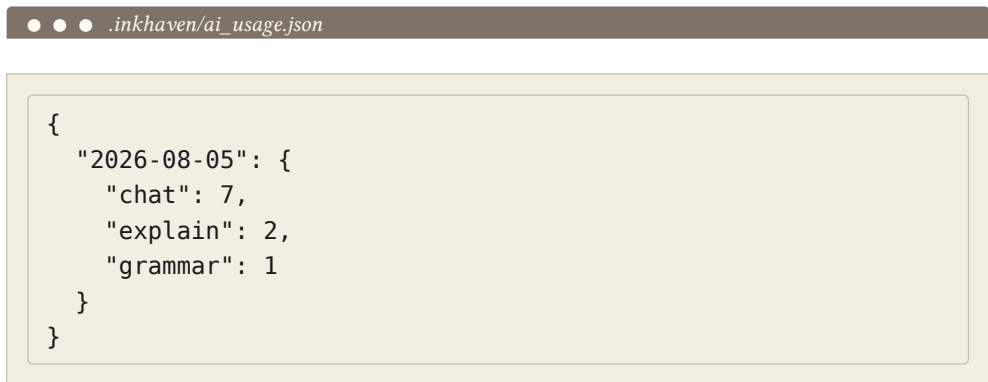
How usage is tracked

The dashboard can only show what the tool bothered to record, so the recording is deliberate and complete. Every inference Inkhaven makes — chat, grammar, explain, continuation, critique, the retrieval-augmented answers, all of it — passes through a single chokepoint that records one tally under a `(day, category)` key the instant the call is made. Nothing that reaches a model goes uncounted.

TERM The usage ledger

A small JSON file at `<project>/inkhaven/ai_usage.json`, keyed `day → category → calls`. Every model call increments its category's count for the current day. The dashboard reads it; the slow tracks keep their capped counts in their own stores so they are not double-counted here.

The file is plain and human-readable — you can open it and see exactly what ran:



```

.inkhaven/ai_usage.json

{
  "2026-08-05": {
    "chat": 7,
    "explain": 2,
    "grammar": 1
  }
}
```

Three properties keep this trustworthy. It is counted by **calendar day** on the same clock the caps use, so a day's tally resets exactly when the caps do. It is written **atomically** and under a lock, so two inferences racing to record at once never lose a tally — the count is exact, not approximate. And it is **pruned on write** to the retention window, so the file never grows without bound; the oldest day falls off once you pass `usage_retention_days` of history.

One boundary is worth stating plainly: the ledger counts **calls**, not tokens or dollars, because a call count is exact and free to keep while a token count would need the model to tell it the truth. For dollars you have the research assistant's live session estimate and the price table behind it; for volume you have this. Together they answer the two questions a writer actually asks — “how much am I leaning on the model today?” and “what is this session costing me?” — without either one pretending to a precision it does not have.

BEFORE A PROJECT IS OPEN

The tracker is pointed at your project at startup, so a call made headless or before any project is open is simply not recorded — there is nowhere to write it. In practice every call you make from inside a project lands in that project's ledger, which is exactly the scope you want: cost is accounted per-book, not globally.

A field guide to free versus paid

To close, the practical map — what you can lean on all day at no cost, and what to reach for deliberately. When in doubt, remember the rule: if you did not press an engage key or pass a `--deep`-style flag and type into a conversation, you did not spend anything.

Free, always — the deterministic core: the editor and the tree; snapshots, search, and the semantic index; the continuity ledger's structural checks; the read-through's measured intensity curve and forward walk; the prose voice profiler; the poetry scanner; the theologian's fast signals; the graph's structural queries; assembly to PDF, EPUB, and the web. None of it touches a model.

Paid, on purpose — the deliberate asks: the world and Socratic and editor slow tracks; the continuity coherence pass; the synthetic first-read; the stylist and theologian engagements; the research assistant's session; chat and chat-with-your-book; grammar, explain, and continuation on a selection. Each is capped or counted, each shows in the dashboard, and each is one keystroke or one flag you chose to press.

That is the whole of watching the cost: a tool that stays free until you ask it not to, tells you the moment you do, and never once stands between you and the work.

WHAT YOU LEARNED

- Cost **informs, never blocks** — the daily caps are speed bumps, not walls; past a budget the slow tracks warn and continue.
- Almost everything is **deterministic and free** — the model is reached only where a determinate answer is impossible, and only when you engage it by hand.
- `inkhaven cost` and `Ctrl+B $` render the **same read-only dashboard**: capped slow tracks with usage bars, uncapped calls by category, and a daily total.
- The default caps are **200** (world), **150** (Inner Socrates), and **200** (Inner Editor engagement) calls a day; they and the `usage_retention_days` window live in the `cost` config block.
- Every inference is recorded by category in `.inkhaven/ai_usage.json`, keyed by day on the `goals.day_boundary` clock, written atomically and pruned to the retention window.

PART IV

The World & Its Facts

CHAPTER 13

13

Places and Characters

A long book is held together by two things it can never afford to get wrong: **where** it happens and **who** it happens to. Names of towns and rivers, of protagonists and the people they wrong, recur across hundreds of pages, and the reader keeps a quiet ledger of every detail. A city founded in one chapter must not be founded again in another; a character with grey eyes on page 12 must not have brown ones on page 280. Inkhaven gives you two dedicated places to keep that ledger — the

Places book and the **Characters** book — and then it does something no notebook can: it reads your prose against them, lights up every mention, and lets you ask an assistant a question grounded in your own canon.

This chapter is the tour of that machinery. It covers what the two system books are and how you fill them, the coloured overlays that make your world visible inside the editor, the multilingual stemming that matches an inflected name to its entry, the two RAG chords that turn a selected name into a grounded question, and — at an operator’s level — the character-arc tracker that watches a declared arc across a whole manuscript. It closes on the quiet part: the Characters roster is not a private notebook feature. Half of Inkhaven’s reading intelligences read it too.

Two books, seeded for you

Every project Inkhaven creates carries a small block of **system books** — Notes, Research, Prompts, Places, Characters, Help, and a handful more depending on your version. They are ordinary books in every visible respect: they hold chapters, subchapters, and paragraphs, and you edit their prose exactly as you edit your manuscript. What sets them apart is that Inkhaven seeded them at `init`, treats them as metadata rather than manuscript, and wires specific behaviour to their contents.

TERM System book

A book Inkhaven creates and reserves — marked internally by a *system tag* (`places`, `characters`, `facts`, and so on). You can add, edit, group, and delete the paragraphs *inside* it freely, but you cannot delete or rename the book itself. Its contents drive a feature rather than appearing in your finished document, and the concordance and other prose tools deliberately exclude it from the corpus.

Places and Characters are the two worldbuilding system books. They sit near the bottom of the Tree pane, below your own books and above Help:

● ● ● *The Tree — your books above, the system block below*

My Novel	(your book)
Interludes	(your book)
Notes	
Research	
Prompts	
Places	← cyan overlay · Ctrl+B P
Characters	← yellow overlay · Ctrl+B C
Help	

The two books are structurally identical twins. Everything this chapter says about a Place is true of a Character with three substitutions: the book, the overlay colour (cyan versus yellow), and the RAG chord (Ctrl+B P versus Ctrl+B C). We will lean on the Places book to teach the mechanics, then note where Characters differs.

Recording a place, recording a person

An entry is a paragraph, and **the paragraph title is the entity's name**. That single rule is the whole data model. To add one:

1. Move the tree cursor to the **Places** row and press → to expand it.
2. Optionally press **C** first to add a chapter for grouping — **Cities**, **Regions**, **Buildings** — if you expect many entries.
3. Press **+** to add a paragraph. The Add modal asks for a title. **Type the place name:** **King's Landing**, **Apartment 12**, **Москва**, **San Francisco — 1906**.
4. Press Enter; the new paragraph opens in the editor.
5. Type whatever you want to remember — physical description, history, who lives there, why it matters to the plot. This is ordinary Typst prose: headings (= History), bold (*important*), lists all work.
6. **Ctrl+S** to save.

You can put paragraphs directly under the book or nest them as deep as your project's depth allows. Every paragraph anywhere in the subtree is picked up — there is no separate “register this entry” step; saving is registration.

● ● ● *A Places book that mixes flat and grouped entries*

Places	
Cities	(chapter)
Major	(subchapter)
Москва	(paragraph)
Санкт-Петербург	(paragraph)
Minor	
Воронеж	
Regions	(chapter)
Сибирь	
Дача на Волге	(paragraph, directly under Places)

The Characters book works the same way, and the same grouping instinct applies — most authors organise it by role (Protagonists, Antagonists, Supporting), by house or family, or by the part of the story where a character enters. The title is the character's **canonical name**: Aragorn, Anna Karenina, Robb Stark, Дмитрий.

FICTION

Your Characters book is the cast list, and your Places book is the map's index. Put voice notes, relationships, secrets, and the chapters a character enters and exits into the body — the assistant reads it back when you ask, and the arc tracker checks against it.

NON-FICTION

For non-fiction the same two books hold recurring **entities**: the people your argument keeps naming and the sites, institutions, or regions it returns to. The overlay then doubles as a consistency check — a name you spelled two ways stops lighting up on the variant.

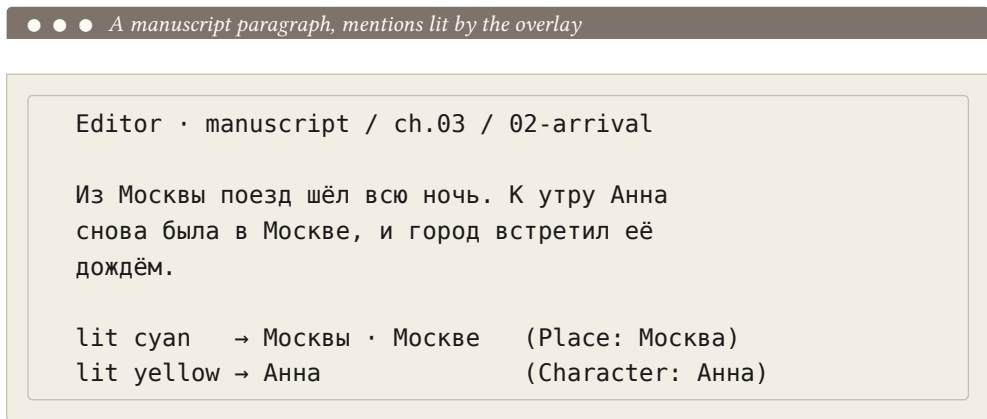
ONE ENTRY PER PARAGRAPH

Keep entries atomic. If you find yourself describing two locations in one paragraph, split them — the overlay, the RAG lookup, and the arc tracker all key off the paragraph title, so a paragraph that covers two things is only ever found under one name.

The highlight overlays

Open any paragraph in your manuscript and look at it. Every word that matches a recorded Place name renders in **cyan and bold**; every word that matches a Character name renders in **yellow and bold**. Your world becomes visible in the prose without any tagging on your part — the lexicon is compiled from the two books' titles and applied on render.

Because a monospace page cannot show colour, picture the overlay as a highlight laid over the matching words:



Note what happened there. The Place entry is titled **Москва**, but the prose says **Москвы** and **Москве** — two inflected forms — and both light up anyway. That is the **stemming**, covered next. Note too the collision rule: when a Place name and a Character name would land on the same word, **Place wins by design**. This matters for surnames that are also place names; if it bites you, give one of the two a more distinctive title.

The overlay refreshes at three moments, so it is always current:

- **Live as you type** — each render re-checks the visible buffer against the lexicon, so a sentence you just wrote lights up immediately.
- **On every save** — adding a new entry and pressing **Ctrl+S** starts highlighting that name everywhere else at once.
- **On project open** — the lexicon is compiled from scratch at startup.

The colours are yours to change. In **inkhaven.hjson**:

● ● ● *Overriding the overlay colours*

```
theme: {
  places_fg:      "#a6e3a1"    # green instead of cyan
  characters_fg:  "#fab387"    # peach instead of yellow
}
```

The defaults are `#89dceb` (sky blue) for Places and `#f9e2af` (yellow) for Characters. Restart the TUI to pick up a change.

Stemming: one entry, every form

Names inflect. English is gentle — `city` and `cities`, `Aragorn` and `Aragorn's` — but a language like Russian gives a single name six or more surface forms: `Анна`, `Анне`, `Анной`, `Анну`, `Анны`. You cannot be expected to file a separate entry for every grammatical case. So the matcher does not compare words literally; it compares **stems**.

Inkhaven runs Snowball stemmers (from the `rust-stemmers` crate), which reduce an inflected word to its root. The entry `Москва` and the prose word `Москве` stem to the same root, so the overlay lights the prose word even though it never matches the title character-for-character.

The stemmer language is driven, in order, by:

1. The top-level `language` field in `inkhaven.hjson` — it wins whenever it is non-empty. The default is `"english"`.
2. `editor.stemming.languages` — a fallback list, used only when `language` is empty, that runs several stemmers at once.

Set `language` to the dominant language of your manuscript:

● ● ● *A Russian project — one line does it*

```
language: russian
```

The supported set is `arabic`, `danish`, `dutch`, `english`, `finnish`, `french`, `german`, `greek`, `hungarian`, `italian`, `norwegian`, `portuguese`, `romanian`, `russian`, `spanish`, `swedish`, `tamil`, and `turkish`. If you write in a language not on the list, set `language: ""` to disable stemming — the overlay then falls

back to **exact** word matches, which is correct for an uninflected language and safe for any.

Multi-word titles are handled as a **sequence of stems**. `King's Landing`, `North Tower`, `Анна Каренина` — the matcher splits the title into tokens, stems each independently, and looks for the same run of stems in the prose. So `Анна Каренина` matches the full name in any case. It does **not**, however, match the parts in isolation: an entry for `Anna Karenina` lights the full name but not a standalone `Anna`. If both forms appear often, add a second short entry (`Anna`) or title the entry with the form the prose actually uses most.

DOES IT WORK IN RUSSIAN?

Yes — that is the point of stemming. The overlay, the multi-word matcher, and the RAG lookups all key off the project `language`, so a Russian, German, French, or Spanish project gets inflection-aware matching with no extra configuration. For a bilingual project (say, translation work) set `language: ""` and list both stemmers under `editor.stemming.languages` so a `Moscow` entry lights English inflections and a `Москва` entry lights Russian ones.

Asking the AI: Ctrl+B P and Ctrl+B C

The overlay is passive — it shows you what you have recorded, but it never talks back. To **ask** the assistant about an entry, using your own canon as the ground truth, use the two RAG chords. They are the active half of this feature.

TERM RAG context

Retrieval-Augmented Generation: before the model answers, Inkhaven retrieves the relevant entries from your Places or Characters book and prepends them to the prompt. The model answers from *your* text, not from whatever it half-remembers about a real or famous place. It is how you keep the assistant inside your canon.

The flow is the same for both chords:

1. In the editor, select the name in your prose, or just place the cursor inside the word.

2. Press **Ctrl+B P** for a Place, or **Ctrl+B C** for a Character.

Inkhaven sweeps the matching book for every paragraph whose title contains your term (case-insensitive), builds a context block from their bodies, and then behaves one of two ways depending on the AI prompt bar:

- **If the prompt bar is empty**, the context is **stashed** as the next RAG prefix and focus jumps to the prompt bar. The status line reads **Place RAG armed for 'Москва' – type your question and Enter.**
Type your question and press Enter; the model answers using the stash.
- **If the prompt bar already has text**, the inference fires **immediately** with the context prepended, and focus moves to the AI pane so you can watch the answer stream in.

The block the model receives looks like this:

● ● ● *The context block prepended to your question*

```
— Place context for `Москва` (1 match(es)) —

— Place: Москва —
= Москва

= History
Столица России. Основана в 1147 году.

= Geography
На реке Москве, в Центральной России.
— end place —
```

Two dials sharpen the answer. **F10** toggles the inference mode: with **Local**, the model is constrained to use **only** the context you supplied — ideal when you want a faithful summary of your canon and nothing invented from training data. Switch to **Full** if you want the model to reach beyond the entry. And **F9** sets the **scope** — combine it with the RAG chord to give the model both the surrounding chapter and the entry: put the cursor in a city's name, press **F9** to widen scope to Paragraph or Chapter, then **Ctrl+B P**, and ask “how does she feel about being back?” The model sees the scene **and** the city's recorded history.

If your selection matches no entry, the status reports `Place RAG: no` entry titled like 'XYZ' in the Places book and nothing fires — add the entry first. The chords route through the same meta-prefix dispatcher, so they work from any focus, but selecting in the editor first is the natural path.

Ctrl+B P	Place RAG — sweep the Places book for the selected name, arm or fire a grounded question.
Ctrl+B C	Character RAG — the same against the Characters book.
F10	Toggle inference mode Local ↔ Full (Local = answer only from the stashed context).
F9	Cycle the AI scope; stack it under a RAG chord for scene + entry grounding.

PRONOUNS ARE NOT NAMES

The lexicon is a noun-phrase index, not a coreference resolver. It highlights literal titles (stemmed), so pronouns — `he`, `she`, `они` — are never lit and aliases need their own entries. For a “who is she here?” question, widen `F9` scope so the model has the passage to reason over, then ask in the chat.

Character arcs — a first tour (CHAR-1)

The Characters book earns a second life as the ground truth for Inkhaven’s **character-arc tracker**. This is a large feature with its own deep treatment in the *Know Your Book* companion; here we give it an operator’s tour — enough to run it and read its output, with pointers to where the full story lives.

The premise: you **declare** the arc you intend, and Inkhaven watches the manuscript to see whether the prose delivers it. You declare an arc by adding a `character_arc` block to a Characters-book entry, written as HJSON in the paragraph body:

● ● ● *A declared arc in a Characters entry body*

```
{
  character_arc: {
    arc_type: "positive_change"
    desired_state_start:  "Mara defers to her family."
    desired_midpoint_state: "Mara's first open defiance."
    desired_state_end:     "Mara acts without permission."
  }
}
```

The `arc_type` names one of five structural shapes the tracker understands — `positive_change` (a false belief overcome), `flat` (the character holds firm while the world changes), `corruption`, `fall`, and `disillusionment` — plus any other string, which falls back to a generic probe. Only `desired_state_start` and `desired_state_end` are required; the midpoint is optional. Omit the whole block and the entry simply has no arc to check.

Everything else is computed from the prose. Three commands drive it from the terminal:

- `inkhaven character refresh` recomputes the **agency score** (fast, deterministic) and re-extracts the chapter-by-chapter **observable-state chain** (an LLM pass, run lazily — only chapters whose text changed are re-extracted). `--name` limits it to one character.
- `inkhaven character check` runs the **completeness checks** against the declaration and is built to be a pre-submission gate: it exits `1` on any gap or stall and `2` if the ending or the earned-arc check fails, so you can wire it into a script. `--json` emits machine-readable findings.
- `inkhaven character plan` reports **Planning-Board coverage gaps** deterministically — a declared arc that no scene card names, or one confined to the book's first half. It exits `1` on any gap.

TERM Agency score

A deterministic, zero-cost number between 0 and 1 for a character in a chapter: the share of their appearances where they *act* (their name precedes a transitive action verb) versus where they are *acted upon* (a passive construction, or their name follows the verb). Computed from a per-language action-verb list — English, Russian, German, French, and Spanish ship built in, and you can fold genre verbs in via `char.extra_action_verbs`. A protagonist whose agency sags for a stretch of chapters is a protagonist going passive.

The stall detector is the other deterministic half: it finds the longest run of chapters where the character shows **no observable change** after their baseline appearance, and flags it when the run reaches `char.stall_threshold` (default 4). The four alignment and earned checks — start, midpoint, end, and “was this arc earned?” — are the LLM half, judged only on observable evidence in the prose and run only for characters with a declaration and at least `char.min_chapters_for_check` (default 3) chapters of extracted state.

`inkhaven character arc <name>` prints the whole picture read-only from the cached `char.duckdb` — declaration, state chain, agency, stalls, checks, and planning gaps together:

● ● ● inkhaven character arc Mara

Character arc – Mara · `My Novel`

Declared arc: positive_change

start : Mara defers to her family.
 midpoint : Mara's first open defiance.
 end : Mara acts without permission.

State chain (6 chapters):

· ch. 1 agency 0.42 (3a/4p) Silent at dinner.
 · ch. 2 agency 0.38 (2a/3p) Runs an errand.
 + ch. 4 agency 0.61 (5a/3p) Refuses to lie.
 · ch. 5 agency 0.55 (4a/3p) Holds the line.

Arc checks:

[earned] Arc earned – the refusal in ch.4 is
 seeded by the small evasions before it.

A + marks a chapter where the character's state changed; a · marks one where it held. Inside the editor the same view lives one chord away: **Ctrl+V Shift+N** opens the arc modal for the nearest character (one named in the open paragraph, else the first tracked one) — the declared arc, the state chain with its agency scores, the completeness checks, and any planning gaps, scrollable with **↑↓**, **Esc** to close. It is read-only over the cache, so populate the cache first with a **refresh / check**, or run the zero-AI **Ctrl+B Shift+C** review pass, whose deterministic arc findings also land in the Output pane's `character` category.

WHERE THE FULL TREATMENT LIVES

This is the operator's tour. The arc taxonomy in depth, the agency windows and cross-system enrichment signals, the exact wording of the LLM checks, and the Planning-Board integration are covered in the *Know Your Book* companion and in Chapter 17 (Continuity and Knowledge). The `char:` config block tunes every threshold; see the configuration reference.

How the roster feeds the rest of Inkhaven

Here is the quiet payoff. The Characters book is not a private notebook that only the overlay and the RAG chord read. The list of entry titles — the **roster** — is the canonical cast of the whole project, and a surprising number of Inkhaven’s reading intelligences read it:

- **Continuity** (SENTINEL, Chapter 17) builds its “referenced-before- introduced” invariant on the roster, and the status-bar POV chip names the most-mentioned rostered character in the open paragraph.
- **Knowledge** (KEN, Chapter 17) grants each rostered character an epistemic timeline — who could know what, and when — keyed off their presence in timeline events, so a name absent from the roster gets no grant.
- **Voice at scale** (CHORUS, Chapter 18) resolves a scene’s POV to the most-mentioned rostered character and checks each one’s distinct voice against the others.
- **Dialogue attribution** resolves “who said this” to a roster-canonical name, and the arc tracker’s agency and stall scoring rides on the same roster.

The lesson is that filling in the Characters book is not busywork for one feature — it is the single act that turns on continuity, knowledge, voice, dialogue, and arc tracking together. A name you never record is a name none of them can watch. This is why the roster is drawn from the entry **titles** directly: record a character once, as a paragraph, and the whole reading apparatus knows who they are.

WHAT YOU LEARNED

- The **Places** and **Characters** system books are seeded at `init`; you record an entity as a paragraph whose **title is its name**, and saving is registering.
- Mentions light up in the editor — **cyan** for Places, **yellow** for Characters — with **Snowball stemming** keyed to the project `language`, so one entry matches every inflected form; Place wins a collision.
- `Ctrl+B P` and `Ctrl+B C` sweep the matching book and **arm or fire** a question grounded in your own canon; `F10 Local` keeps the answer inside that context, and `F9 scope` stacks scene grounding on top.
- The arc tracker (CHAR-1) checks a declared `character_arc` block against the prose: `inkhaven character refresh / check / plan / arc` and the `Ctrl+V Shift+N` modal, with a deterministic **agency score** and **stall** detector plus LLM completeness checks.
- The Characters **roster** is shared ground truth — **continuity, knowledge, voice, dialogue, and arcs** all read it, so one entry switches on many readers at once.

CHAPTER 14

14

The World

A long story happens somewhere. The somewhere has a sky with a certain number of moons, a climate that makes some weeks bitter and others sweet, distances that take real days to cross, and cities of a certain size in certain places. Get any of it wrong and a reader who is paying attention feels the floor tilt: snow in the desert, a moon that was full last night and full again tonight, a courier who rides four hundred miles between breakfast and supper. Inkhaven's answer is to let you **declare** the physics of your world once, in a single file, and then do two things with that declaration — build the rest of the world out from it by deterministic simulation, and watch your prose for the moment it forgets what you declared.

This chapter is the operator’s tour of that machinery: how you write the definition, what the compiler derives from it, how the map is drawn, the interactive worldbuilder you can shape a world in, the coherence checker for imagined societies, and the live fact-checker that reads over your shoulder in five languages. It is deliberately breadth-first. The full treatment — every field, the physics behind each layer, worked worlds from scratch — lives in the companion book **Building the World**; this chapter tells you what each piece is, how to reach it, and how to run it.

The two halves

The whole subsystem turns on one file at the root of your project: `world.hjson`. It is opt-in. A project without it behaves exactly as before; drop one in and the world layer wakes up. Everything then divides into two halves that share that one declaration.

TERM `world.hjson`

The single file, at the project root, where you declare a world’s **physics** — its star and planet and moons, and optionally its geology, geography, hydrology, economy, and any magic that breaks the rules on purpose. Only the world’s **name** and its **astronomy** are required; every other block is optional with sensible defaults. Scaffold one with `inkhaven realworld new <name>`.

The first half is **the compiler**. Give it the definition and a seed, and it derives a coherent world — astronomy, geology, climate, hydrology, demographics — in five layers, then materializes the result into a **World** system book, one chapter per layer, plus a rendered map. The second half is **the fact-checker**. As you write, Inkhaven reads each paragraph against the world you built — against its climate zones, its distances, its populations, its sky — and flags the sentences that contradict it, in whatever of five languages you wrote them in.

DETERMINISM

Everything the compiler produces is a pure function of (definition, seed). The same inputs give the same continents, the same rivers, the same cities, every run — driven by an in-tree SplitMix64 keyed to your seed, never the rand crate. The astronomy layer has no randomness at all; it is closed-form physics, re-derived each run. Keep the seed and your world — and its map — are reproducible forever; change it and the whole world regenerates.

Declaring the physics

You write the definition in HJSON — JSON with the ceremony removed: no quotes required on keys, comments allowed, trailing commas forgiven. There is one catch worth stating first, because it bites everyone once.

THE HJSON QUOTING RULE

In HJSON an unquoted string runs to the **end of the line**. So an inline enum value must be quoted — write `class: "G2V"` and `kind: "city"`, not `class: G2V`. Multi-line blocks are fine; it is only inline scalar strings that need the quotes. When a value comes out looking wrong, this is almost always why.

Only two blocks are load-bearing: the world's `name`, and its `astronomy`, which is required because everything about seasons, tides, and the calendar is derived from it. The rest are optional, and each one you add makes the world richer and gives the fact-checker more to check against.

● ● ● *world.hjson — the shape of a definition*

```
{
  name: "Aldoria"
  seed: 0x1A2B3C          // int or "0x..." hex; drives every
layer
  primary_language: "en"  // sets the fact-checker's language

  astronomy: {             // the only required block
    star: { class: "G2V", luminosity_solar: 1.0, mass_solar:
1.0 }
    planet: { axial_tilt_deg: 23.4, day_length_hours: 24.0 }
    orbit: { semi_major_axis_au: 1.0, year_length_days: 365 }
    moons: [ { name: "Luna", period_days: 27.32 } ]
    calendar: { months: 12, month_length_days: 30, weekdays:
7 }
  }

  geology: { generated: { plates: 7, continents: 4,
                        sea_level: 0.40 } }
  geography: { landmarks: [ { name: "Caer Dath", kind: "city",
                        climate_zone: "tundra", population: 27000 } ] }
  economy: { tech_level: "medieval", currency: "silver
mark" }
  magic: { enabled: false }
}
```

Read the blocks as a table of what each one drives:

name / seed	Identity; the seed drives every procedural layer.
astronomy	Required. Seasons, tides, calendar — closed-form.
geology	generated (procedural) or dem (a real heightmap).
geography	Named regions + landmarks → Setting + gazetteer.
hydrology	Named waters + rainfall → Setting chapter.
economy	Tech / currency / trade / resources → known goods.
magic	Declared exceptions the fact-checker respects.

A few blocks earn a word here because they change what the checker will accept. A `geography.landmark` that carries a `climate_zone` becomes a **gazetteer** entry the checker resolves by name — so it knows Caer Dath’s weather even before you `geology.`

compile and accept the procedural cities. The `sea_level` under `generated` is a **threshold on the generated heightmap, not a literal ocean fraction**; the terrain is not uniform, so a value near `0.40` gives an Earth-like, ocean-dominant world — raise it for a wetter one, lower it for more land. And `economy.resources` (like `geology.notable_minerals`) extend the checker’s list of known goods, so trading your world’s own metals is never flagged as an anachronism.

The `magic` block is the pressure valve. When your world breaks physics on purpose, you declare the exception so the checker respects it instead of flagging it in every chapter forever.

● ● ● *A magic rule — a declared exception to the physics*

```
magic: {
  enabled: true
  rules: [
    {
      kind: "messenger_birds"
      covers: ["travel_time"]      // categories this rule
      excuses
      description: "Royal pelicans fly day and night with
      relays"
      applicable_to: { roles: ["royal_messenger"] }
    }
  ]
}
```

Each rule names a `kind`, the categories it `covers`, and an optional `applicable_to` scope (`roles` / `regions` / `seasons`). The checker consults the ledger **lazily** — only after it already has a candidate warning — and a covered, applicable rule suppresses the finding with a note rather than hiding it silently. The rule materializes into the World book’s **Magic Ledger** chapter, and `inkhaven realworld magic` lists what is on the books.

START FROM EARTH

The repository ships a complete, heavily-commented real-Earth definition at `examples/realworld/Earth.hjson` — the exact Sun, Earth, and Moon, the Gregorian calendar, Earth-tuned geology, and a populated geography. Copy it in as `world.hjson` and compile to begin in a world you already recognise, then edit outward toward your own.

The compiler and what it derives

`inkhaven realworld compile` runs five layers in dependency order, each a pure function of the ones before it. You can run one layer or all of them, print a human summary or the full structured JSON, and — with `--materialize` — write the result into the World system book.

Astronomy	Kepler year, insolation by latitude, lunar periods, tides.
Geology	seed → plates → heightmap → continents, ranges, minerals.
Climate	Zonal model → Köppen biomes, rain shadows, winds.
Hydrology	D8 flow over the terrain → rivers, lakes, watersheds.
Demographics	Carrying capacity → a rank-size hierarchy of towns.

● ● ● *Compiling the layers from the CLI*

```
$ inkhaven realworld compile --layer demographics
$ inkhaven realworld compile --layer climate --json
$ inkhaven realworld compile --layer geology --materialize
```

```
Geology (seed 0x1A2B3C)
  plates 7 · continents 4 · sea level 0.40
  highest cell 4210 m · 3 mountain ranges clustered
  materialized → World / Geology
  wrote assets/world/heightmap.png
```

--layer is one of **astronomy** (the default) / **geology** / **climate** / **hydrology** / **demographics**. The astronomy layer also does one check worth knowing about: if you **declare** an **orbit.year_length_days**, the compiler computes Kepler's value and warns when the two disagree by more than a day — your calendar and your orbit are allowed to drift, but you will be told they have.

When you materialize, the writes are **idempotent** — re-running never duplicates, it updates in place — and they land in the World system book as one chapter per layer, plus the author-declared **Setting** chapter, the **Magic Ledger**, and the normalized heightmap under **assets/world/**. Nothing touches your manuscript.

Proposals and Places

The compiler models settlements, but it never writes places into your book behind your back. Instead its cities become **proposals** you accept or reject one at a time. Only an accepted proposal becomes a real **Place** record, and each one records a Place-to-World cross-reference.

● ● ● *The proposal queue closes the loop*

```
$ inkhaven realworld propose           # seed the queue
$ inkhaven realworld proposals list
$ inkhaven realworld proposals accept-all
$ inkhaven realworld places           # the accepted places
```

Re-running **propose** never re-offers a site you already resolved. These accepted Places are exactly what the fact-checker resolves place names against, which is what closes the loop: **compile** → **accept cities** → **write** → **check**. In the TUI the whole cycle is under one chord, which the next section names.

The World hub — Ctrl+B W

Every CLI operation has a home in the TUI under **Ctrl+B W**, the World hub. It opens a read-only, scrollable overview of the world — the definition, the compiled astronomy layer, and whether it has been materialized yet — and from there a handful of second keys drive the whole subsystem without leaving the editor.

Ctrl+B W Open the World overview (definition + astronomy + status).

→ **C** Compile + materialize all five layers, seed the proposal queue.

- **P** Open the Place proposal queue (Enter accept · r reject).
- **F** → **P/B/R** Fact-check the paragraph / book / recent edits.
- **M** Render the world map with plakat.
- **S** Toggle the idle auto slow-check (off by default).

`Ctrl+B W → C` is the one-key path through the whole compiler: it compiles and materializes all five layers **and** seeds the proposal queue in a single step, so a fresh world is one chord from existing. The other keys open the queue, arm the fact-checker's scope picker, draw the map, and toggle the background slow check — each covered in its own place below.

Maps — the plakat cartographer

`inkhaven realworld map` (or `Ctrl+B W → M`) turns the compiled layers into a **MapSpec** — a structured description of the map — and hands it to **plakat**, a separate cartographer you install once with `cargo install plakat`. `plakat` loads the spec and skips its own map-generation AI entirely, so the drawn map stays a pure function of the world and its seed, exactly like everything else.

● ● ● *Rendering the world map*

```
$ inkhaven realworld map
  features: assets/maps/world.features.png # the rendered
map
  geojson:  assets/maps/world.geojson      # coast/rivers/
roads
  spec:     assets/maps/world.mapspec.json # the emitted
MapSpec
```

Mountains come from clustering the heightfield's high cells; rivers run their real D8 watercourse; landmarks are your largest settlements, with coastal cities promoted to ports. `plakat`'s resolved landmark positions are read back to **refine each accepted Place's coordinates**, so the map and the gazetteer agree. Two flags tune the run — `--spec-only` writes the MapSpec without invoking `plakat`, and `--no-ingest` renders without touching Place coordinates.

PLAKAT IS OPTIONAL

These are called “plakat” maps after the cartographer that draws them. The binary is a genuinely external dependency, and it is the **only** one Inkhaven reaches for. A missing plakat never fails the run — it degrades to a notice, and everything else about the world still works. The rendered map is a convenience, not a load-bearing part of the pipeline.

The interactive worldbuilder — at a glance

Everything above is reachable from the CLI and the `Ctrl+B W` hub, editing `world.hjson` by hand. When you would rather **build** a world interactively — shaping the sky, adding nations, drawing the map, watching a plausibility score move as you go — there is a whole separate application for it: `worldbuilder`, `inkhaven` a full-screen TUI that is a third sibling beside `research` and the linguistic workbench.

TERM The worldbuilder

A companion TUI (introduced in 1.9.0) that is a **front-end** to everything in this chapter. Every change it makes lands in the same `world.hjson`, compiled by the same chain, checked by the same checker. It never generates your prose — the author decides, and the worldbuilder measures, validates, and records.

Its window is four regions: a **Facts** tree over a **World** tree down the left; a wide right pane that cycles **Chat** / **Research** / **Map** / **Ledger**; a full-width **Query** prompt; and a status bar carrying the world’s name and its live **plausibility score** (★ NN, with ▲ / ▼ deltas). Plain text in the Query prompt is a question to the AI; a line that opens with a slash is a command.

```

┌ Facts ◎ ───────────┐ Map ───────────┐
│ ▼ fact:world         │ . . ~ ~ ^ ^ . . biome minimap|
│   ◎ two moons        │ ~ ~ = = ^ M ^ ~ . after /compile|
│ ▼ World              │ ~ . = △ . . ~ ~ . |
│   ▶ astronomy        │ . . . . $ $ . . . |
│   ▶ geography        │ ───────────┐
└──────────┘         Ledger · Chat · Research (Ctrl+R) |
Query ▶ /compile|
└──────────┘
Aldoria      ★ 74 ▲2      pending: 3 edits      ? hints

```

The commands typed at that prompt are the shape of the whole session. A few of the load-bearing ones: `/interview` walks a guided five-stage world interview; `/set <dot.path> <value>` sets any `world.hjson` key, with shortcuts like `/star`, `/tilt`, `/moon`, and `/nation` for the common ones; `/wfact` records an author fact into the Facts book; `/compile`, `/validate`, and `/map` run the same chain, score, and cartographer you already met; and `/roll` compares candidate worlds on derived seeds so you can `/adopt` the one you like.

Crucially, shaping commands do not touch the file as you type. They accumulate into a **pending delta** — accepted-but-uncommitted edits you preview with `/diff`, undo with `/undo`, and only fold into `world.hjson` atomically with `/write`. The plausibility score, though, moves the moment an edit is **accepted**, before it is committed, so you can feel a change's effect before you keep it.

The map editor

Cycle the right pane to **Map**, press `e`, and you can draw the world directly — and every mark is another pending `world.hjson` edit, reviewed with `/diff` and written with `/write` like any other. A brief tool list:

- t / n** Place a town / named landmark → `geography.landmarks` (⬠).
- r** Draw a river source → mouth → `hydrology.rivers` (≈).
- g / o** Region from the cell's biome (\$) / road between towns (=).
- + / - , .** Raise / lower terrain under a brush · brush size.

d / f

Delete the feature under the cursor · jump to each flaw.

Drawing the terrain and running `/terrain` writes a sculpted grayscale heightmap under `assets/maps/` and points `geology.dem` at it, so the next compile rebuilds the whole world from the shape you drew. This is only the glance; the worldbuilder and its map editor get a chapter of their own in **Building the World**.

FICTION

For a **secondary world** — the invented planet of a fantasy or SF novel — the worldbuilder is where the whole setting is born: shape a plausible sky and geology, draw the map, and let the checker hold your chapters to it.

NON-FICTION

For **history, travel, or reportage** set on Earth, copy examples/ in `realworld/Earth.hjson` and lean on the fact-checker's real climate, distance, and demographics knowledge to catch a wrong season or an impossible journey.

Coherence for imagined societies — utopia

There is a second, narrower checker for a particular kind of world: the imagined **society** — a utopia or dystopia whose premises are supposed to hang together as an argument. You declare its premises as tagged paragraphs in the World book — `para:utopia-premise`, `-mechanism`, `-consequence`, `-elimination` (glyphs $\vdash \odot \Rightarrow \emptyset$) — and run the checker over them.

● ● ● *The utopia coherence checker*

```
$ inkhaven world utopia-check --stage 1      # chain logic
(free)
$ inkhaven world utopia-check --stage 2      # pairing
(explicit)
$ inkhaven world utopia-check --stage all
$ inkhaven world utopia-model                # the extracted
claims
```

Stage 1 (chain logic) is deterministic and free; Stage 2 (pairing premises for tension) is explicit-only because it spends LLM calls per pair; Stage 3 scans the prose for entailment. It reasons about **logical and systemic** structure only — does the mechanism actually produce the claimed consequence, does one premise quietly contradict another — and leaves moral and theological coherence to the Inner Theologian. Findings are advisory, cached in `.inkhaven/utopia.duckdb`, surface in the `Ctrl+B Shift+C` review pass, and ground the `utopian-architect` Socratic persona. `utopia-check` exits `1` on a chain-logic finding and `2` on an entailment violation, so it can gate a pre-submission script.

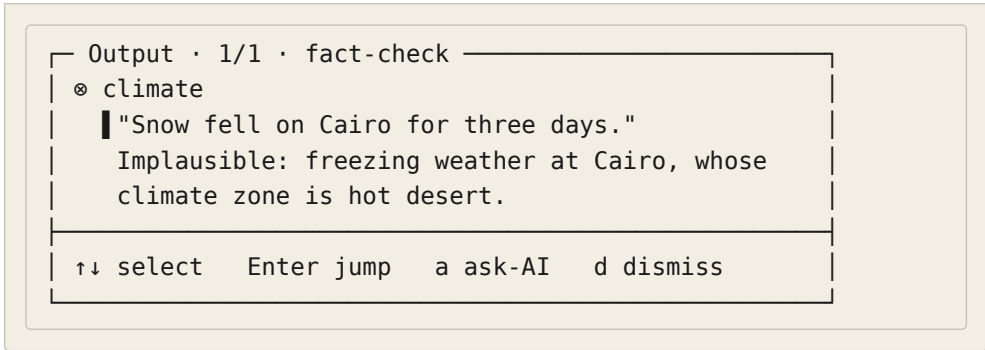
The live fact-checker

Now the second half of the whole subsystem: the reader that watches your prose. When a project has a `world.hjson`, a **fast track** runs automatically — pause a few seconds on a paragraph and any findings appear in the Output pane, with no chord and no focus stolen. Re-checking a paragraph replaces its own prior findings, so the board never accretes stale warnings.

travel_time	A distance + duration implying an impossible pace.
climate	Weather at a known place against its climate zone.
demographics	A population diverging sharply from the model.
astronomy	A moon count disagreeing with the world's sky.
economy	A metal worked that the geology does not yield.

Climate, demographics, and economy resolve place names through the **gazetteer** — your accepted Places plus any `geography.landmarks` you declared — so the checker knows which Cairo you mean and what its weather should be.

● ● ● *A fact-check finding in the Output pane*



From the CLI the same check runs on demand: `inkhaven fact-check --text "..."` checks a literal string, and `--paragraph <id>` checks a stored paragraph. In the TUI, `Ctrl+B W → F` arms a scope picker — `P` for the open paragraph, `B` for the enclosing book, `R` for the twelve most recently edited paragraphs.

Five languages, and graceful degradation

The checker works in **English, Russian, Spanish, French, and German**. It detects the paragraph's language, renders its warnings in that language, and resolves place names in their grammatical cases — Russian `B МОСКВЕ` matches `МОСКВА`, a German genitive matches its nominative. The detector is a built-in heuristic that needs no model, and when it is not confident it **degrades** — rendering in English rather than guessing wrong — and never panics. An optional enhanced parser can be pointed at with `INKHAVEN_LANG_MODEL`, but nothing ever requires one.

The slow track and coherence

The fast track is pattern-based and free. For the subtle contradictions patterns miss, `fact-check --slow` adds an LLM pass, and it is scrupulously cost-controlled: a preflight prints the estimated tokens and the day's call tally; a per-call soft cap (`--max-cost`, default 6000; `--force` overrides) refuses an oversized call; a daily ceiling holds; and a missing provider or a reached cap degrades to a notice. An opt-in **idle auto** variant runs it in the background after about 45 seconds of quiet — toggled with `Ctrl+B W → S`, off by default because it spends tokens.

Where a single-paragraph check asks “is this sentence possible,” `inkhaven realworld coherence <node>` asks whether a **run** of paragraphs holds together — a character in two places without the travel between, a fact asserted then reversed, a timeline that cannot follow. Give it a book or chapter node; it gathers the

paragraphs in document order and runs one cost-capped call, citing the paragraph numbers.

Timeline-aware checking

When your project also has a **timeline**, the fast checker learns **when** a paragraph happens, and its guesses become grounded facts. A paragraph tied to a dated event gains a **calendar-grounded season** — snow in a paragraph the timeline places in summer becomes a flat contradiction, not a guess — plus event-derived travel time (a prose “three days” against a 35-day gap between the traveller’s events), a `date_coherence` check on seasonal hints like a midsummer feast, and `co_location` for a character whose events put them in two places at once. These run automatically, in all five languages, and respect the magic ledger exactly as the world checks do; timeline-derived findings carry a small calendar marker in the

Output pane. Check the whole timeline at once with `inkhaven realworld co-location`.

Facts, and grounding the AI

There is one more sense of “the world” in Inkhaven, and it is worth keeping distinct from the simulated one. Alongside your manuscript lives a **Facts** system book — a place to write down the settled truths of your world in plain prose, whether or not you ever declare a `world.hjson`. These facts do two jobs: they give the AI ground to stand on, and they give a second fact-checker something to check against.

TERM The Facts book

A system book of established truths about your world, written as ordinary paragraphs. Distinct from the `world.hjson` simulation: the simulation **derives** a world from physics; the Facts book **records** the ones you have decided, including the ones no simulation could produce (“the queen has no heir,” “iron is taboo in the north”).

`Ctrl+B Shift+X` fact-checks the open paragraph against that Facts book. It locks the AI scope to the local paragraph, grounds the check against every established fact — climate, geography, seasons, distances, chronology — and streams a verdict into the AI pane, flagging any claim that contradicts the world you have written down. (With an empty Facts book it degrades to a generic local fact-check.) When it flags contradictions, `Ctrl+B Shift+J` cycles the editor cursor through them one

at a time, showing the violated fact on the status bar. Its mnemonic is X for fact eXamination.

The same facts ground the assistant proactively through the **Facts** AI scope. As Chapter 3 described, **F9** cycles the AI scope; **Facts** is one of the **sticky** conversation scopes, so once selected it stays selected across follow-ups. With it active, your prompts are answered against the Facts book — the assistant reasons from your world’s established truths instead of inventing fresh ones. For a large Facts book, **Ctrl+B Shift+S** opens a semantic search over it: type a query, mark the handful of relevant facts, and send just those into a targeted Facts chat, so you ground in the passage that matters rather than the whole book.

FICTION

In **fiction**, the Facts book is the series bible in miniature — the settled truths the AI must honour and the checker must guard, so the assistant never quietly retcons your world mid-conversation.

NON-FICTION

In **non-fiction**, it is the ledger of claims you have verified — dates, figures, definitions — that the assistant answers from and the checker holds your prose to, keeping the manuscript honest to your own research.

The ink.world Bund surface

Scripts written in Inkhaven’s embedded Bund language (Chapter 34) can read the timeline-aware checker’s view of the world, read-only, through the **ink.world.fact_check.timeline.*** family. It exposes exactly what the checker sees — the events near a point in world-time, the events for a character or a place, the season of a point, and a paragraph’s effective date — so a script can reason about **when** things happen without re-deriving the calendar.

events_near Timeline events near a point in world-time.

events_for_character The events that place a character in time.

events_for_place The events tied to a named place.

season_for The calendar season at a given point.

effective_date A paragraph's effective world-date.

These are all `store_read` words — safe, side-effect-free, allowed by default in the Bund sandbox. The utopia checker has its own small read-only surface, `ink.utopia.*` (`model`, `findings`, `violations`, and a `suppress` action), for scripting the coherence pass. Between them a script can fold the world's own knowledge into any automation you build.

WHAT YOU LEARNED

- The world layer is opt-in on a single root file, `world.hjson`, and has **two halves**: a deterministic **compiler** that derives a world from declared physics, and a **fact-checker** that reads your prose against it.
- Only `name` and `astronomy` are required; `geology`, `geography`, `hydrology`, `economy`, and `magic` are optional and each gives the checker more to check. Everything the compiler makes is a pure function of (`definition`, `seed`).
- `inkhaven realworld compile` derives five layers (astronomy, geology, climate, hydrology, demographics); `--materialize` writes them into the **World** book, and settlements become **proposals** you accept into **Place** records.
- `Ctrl+B W` is the hub — `C` compiles all layers, `P` triages proposals, `F` arms the fact-check scope, `M` draws the **plakat** map, `S` toggles the idle slow check.
- `inkhaven worldbuilder` is a whole companion TUI for building a world interactively — pending deltas, a live plausibility score, a map editor — all writing the same `world.hjson`. `inkhaven world utopia-check` grades an imagined society's **logical** coherence.
- The live fact-checker flags travel, climate, demographics, astronomy, and economy contradictions automatically in **five languages**, degrades gracefully, and grows timeline-aware when a timeline exists; the **Facts** book and the `F9 Facts` scope ground the AI in your world's settled truths.
- Bund scripts read the world through `ink.world.fact_check.timeline.*` and `ink.utopia.*` — read-only, sandbox-safe. The deep dive lives in the companion book **Building the World**.

CHAPTER 15

15

The Knowledge Graph

Every chapter so far has added **nodes**. A paragraph is a node; so is a chapter, a book, an image, a script, a fact, a character, a place, a source, a glossary entry. They all live as one uniform kind of thing — a UUID in one database, embedded for semantic search — and Inkhaven has quietly stored them that way since your first project. What it did not store, until now, was the **lines between them**: a first-class, persisted, typed way to say “this fact contradicts that source”, “this paragraph links to that one”, “this word means this concept”. That layer of lines is the knowledge graph, and this chapter is its complete tour — the shape of an edge, the three ways the graph gets filled, the `graph` command that reads it, and the two ways you can **talk** to it.

The graph does not replace the manuscript tree or the vector index you met in the search chapter. It **overlays** them, threading the nodes you already have into one interrogable whole — so that a question the flat text cannot answer, like “which of my claims about the harbour contradict each other?”, becomes a walk across the lines instead of a guess.

YOU NEVER LOSE DATA TO THE GRAPH

An edge is an annotation **over** your nodes, never a container of them. Deleting an edge never touches a paragraph; deleting a node cascades its edges away so none is left dangling. And most edges are **derived** — you can throw the entire graph away and rebuild it from your manuscript, your sidecars, and your installed dictionaries. The graph is powerful precisely because it is disposable.

The edge — a typed line between two nodes

Everything in the graph is built from one small object. An **edge** is a directed or symmetric relation from a source endpoint to a destination endpoint, carrying a kind and a little metadata about why it exists and how much to trust it.

TERM Edge

A single line in the graph: `(src) --kind--> (dst)`, plus a handful of fields — a **kind** (the typed relation), a **weight** (confidence), a **reason** (the rationale, when there is one), an **origin** (how it came to exist), and kind-specific **attrs**. Symmetric kinds are undirected and are found from either end.

● ● ● *The anatomy of an edge*

```
(src) --kind--> (dst)    + weight, reason, origin, attrs

src, dst    the two endpoints (a node, or an extern ref)
kind        the typed relation: links_to, contradicts, ...
directed    false for symmetric kinds (found either way)
weight      confidence in [0,1]  (1.0 = asserted)
reason      the human / LLM rationale, when there is one
origin      how it arose – & whether rebuild recomputes it
attrs       kind-specific extras (a locus key, a role, ...)
```

Endpoints — nodes, and the things that aren't nodes

Most endpoints are ordinary **nodes**: a paragraph, a fact, a character, addressed by its UUID. But an edge sometimes needs to point at something that is **not** a node and should never become one — you do not want ten thousand citations to force ten thousand phantom nodes into being. So an endpoint can also be an **extern reference**: an addressable non-node entity the graph can name without materialising it.

● ● ● *The endpoint kinds an edge can reach*

Node	any manuscript node, by UUID
Source	a Sources-book entry / @cite key
Work	an external work id (OpenAlex/arXiv/Wikidata...)
Locus	a canonical primary-source locus (bible:Jn 3:16)
Sense	a WordNet synset in a language (en: s-dog)
Ili	an interlingual index id – cross-lingual pivot
Grade	a fact-check verdict bucket (inaccurate)
Evidence	a labelled confront / relate evidence item
Declared	a declared world entity (character/symbol/...)

Under the hood the two endpoint columns are real, indexed columns — not a JSON blob — because the **reverse** index is the whole point. Every edge is stored so that “what points at this node?” is a single indexed query rather than a scan of the table, which is what makes a one-hop neighbourhood instantaneous no matter how large the graph grows.

Edge kinds — the vocabulary of relations

An edge’s **kind** is the typed relation it encodes, and the vocabulary is fixed and language-neutral. Each kind has a natural pair of endpoint types and a place it comes from in the rest of the tool — the kinds are, in effect, the five implicit ways Inkhaven already encoded relationships, unified into one table.

• • • *The edge kinds, and where each one comes from*

links_to	¶ → ¶	the editor's paragraph links
event_involves	event→char/place	timeline event markers
sourced_from	fact→source/work	fact provenance
graded_as	fact → grade	/factcheck verdicts
contradicts	claim ↔ evidence	confront / /contradict
in_tension	fact ↔ fact	/relate
qualifies, agrees	""	confront / /relate
cites	work → work	snowball / OpenAlex
cites_locus	¶ → locus	@key[locus] citations
hypernym/hyponym/antonym	sense↔sense	WordNet taxonomy
translates	sense → ili	cross-lingual pivot
mentions	¶ → sense	manuscript↔lexicon
declares	book → entity	cast/symbols/motifs
similar_to	¶ ↔ ¶	embedding similarity (see below)

The last one, `similar_to`, is special: it is deliberately **not** stored. Two paragraphs are “similar” only in the sense that a live nearest-neighbour query over the vector index says so this instant, and that answer changes every time you edit. Materialising it would be a cache that is stale the moment it is written, so embedding similarity stays a live HNSW query and never lands as an edge. Everything else in the list is a real, persisted line.

Origin and durability — will a rebuild keep it?

The single most important field on an edge is its **origin**, because origin answers the question that governs the whole graph’s economics: **if I run graph rebuild, does this edge survive?** Your decisions must survive; a recomputable cache need not.

• • • *Origins, ordered from most durable to recomputed*

authorial	you asserted it directly	kept, never GC'd
promoted	a judgement you accepted	kept
judged	an LLM judgement, advisory	kept until dismiss
imported	reference data (WordNet...)	its own command
structural	derived from your fields	RECOMPUTED
derived	recomputable (similarity)	RECOMPUTED

`graph rebuild` clears and recomputes only the **structural** projection — the edges it can re-derive from your project's own durable data. Your assertions (**authorial**), the judgements you have accepted (**promoted**), the LLM judgements still awaiting your triage (**judged**), and imported reference data (**imported**, rebuilt by its own command) are all preserved. This is why a rebuild is safe to run at any time and as often as you like: it can only touch the part of the graph that is, by definition, a function of your manuscript.

Populating the graph

A fresh project's graph is empty. Three actions fill it, and between them they cover the structural spine, the lexical bridge, and the judgements you accumulate while you work.

Structural edges — graph rebuild

• • • *Deriving the structural projection*

```
inkhaven graph rebuild
```

`graph rebuild` walks your project and derives every **structural** edge from it: the paragraph links you drew in the editor, the timeline event involvements, the provenance of each fact, the `/factcheck` verdicts, and the `@key[locus]` primary-source citations. It is idempotent — it clears the structural projection and rebuilds it whole — so the discipline is simply to run it whenever the manuscript has changed enough that you want the graph current. It never touches your authorial, promoted, judged, or imported edges.

The lexical bridge — graph lexical

● ● ● *Importing the WordNet lexical net*

```
inkhaven wordnet fetch en      # once, per language
inkhaven graph lexical
```

`graph lexical` imports the **WordNet lexical bridge** for the project language, if you have installed one. It links the words your manuscript actually uses to their senses (the `mentions` edges), lays in the local semantic net between those senses — `hypernym`, `hyponym`, `antonym` — and rides the interlingual index with `translates` edges so a concept in one language meets its counterpart in another. Run `inkhaven wordnet fetch <lang>` first to install the dictionary; `graph lexical` is idempotent and imported, so it survives a structural rebuild and is refreshed only by re-running it.

Judged edges, live — confront and graph link

The third source fills the graph **as you work**. The editor's `confront` (`Ctrl+V ?`) and `/relate` persist their judged stance as `judged` edges — a `contradicts`, an `in_tension`, a `qualifies`, an `agrees` — accumulating rather than repeating, so a second `confront` of the same material does not re-assert the first. And `graph link`, below, asks an LLM to propose stance edges from a fact to its related facts. All of these land as **advisory** `judged` edges: the graph records them, but they wait for your verdict. You **promote** the ones that stick and **dismiss** the rest, and the place they wait is the edge inbox.

TERM The edge inbox

The set of advisory `judged` edges awaiting triage — the stance edges proposed by `confront`, by `graph link`, and by deep research, none of them yet accepted. You read it with `graph pending` or the `Ctrl+B z` hub's `i` view, **promote** what you agree with (it becomes `promoted` and survives rebuilds), and **dismiss** the rest (the edge is deleted).

The graph command

The read surface is one command with a verb, mirroring the shape of the rest of the CLI. Every verb but `link` and `ask` is deterministic and free; those two need a language model.

• • • The graph verbs

```
inkhaven graph <verb>
```

<code>stats</code>	node + edge counts, per-kind breakdown
<code>rebuild</code>	(re)derive the structural edges
<code>lexical</code>	(re)build the WordNet lexical bridge
<code>neighbors <node></code>	one-hop neighbourhood as a tree
<code>contradicting <node></code>	stance clashes touching a node
<code>loci <node></code>	the primary-source loci it cites
<code>paths <from> <to></code>	a bounded path between two nodes (≤ 8)
<code>pending</code>	the judged edge inbox (advisory)
<code>promote <edge></code>	judged $\rightarrow$ promoted (kept across rebuild)
<code>dismiss <edge></code>	delete a stance edge
<code>link <node></code>	propose stance edges (needs an LLM)
<code>ask <question></code>	answer by WALKING the graph (LLM)

stats — the shape of the graph at a glance

`graph stats` is the first thing to run after a rebuild: it reports the node and edge counts and a per-kind breakdown, so you can see at once how much of the graph is structure, how much is lexical, and how many judged edges are waiting. It is the fastest way to answer “did the rebuild actually find anything?”

neighbors — the one-hop neighbourhood

`graph neighbors <node>` is the workhorse read. It renders a node’s immediate neighbourhood as a tree, grouped by kind, with an arrow on each line telling you its direction — $\rightarrow$ outgoing, $\leftarrow$ incoming, $\rightleftharpoons$ symmetric. Large groups truncate, and the neighbourhood is hard-capped so a hub node with five hundred `mentions` edges cannot flood the view.

● ● ● *graph neighbors — a node's one hop, grouped by kind*

```

◆ 003. Quiet hour
├─ contradicts (1)
│   └─ evidence fact: The lantern was lit at dusk — §3
├─ links_to (2)
│   └─ 002. The tide returns
│      └─ 004. A tally of names
├─ mentions (500)
│   └─ sense en:enoewn-01929162-a
│   ...
│   ... +492 more

```

contradicting, loci, paths — the targeted reads

Three verbs answer sharper questions. `graph contradicting <node>` pulls only the recorded stance clashes touching a node — its `contradicts` and `in_tension` edges, in either direction — which is the query you want when you are hunting for a fact that fights another. `graph loci <node>` lists the canonical primary-source loci a node cites, the scholarly companion to `neighbors`. And `graph paths <from> <to>` finds a bounded citation-or-link path between two nodes — up to eight hops — and prints it, or tells you plainly when no path exists within the bound. A path is how you answer “is this claim connected to that source at all, and by what chain?”

promote, dismiss, pending — triaging judgements

`graph pending` prints the edge inbox — the advisory `judged` stance edges awaiting your verdict. For each, `graph promote <edge>` accepts it: the edge’s origin becomes `promoted`, and it is thereafter kept across every rebuild. `graph dismiss <edge>` rejects it, deleting the stance edge outright. These three are the CLI face of the same triage you can do from the editor’s hub, and they are the only way a machine judgement ever becomes a durable part of your graph — nothing is promoted without your say-so.

link — proposing stance edges from a fact

`graph link <node>` asks a language model to look at a fact and propose stance edges from it to its related facts — the contradictions, tensions, and agreements a human might notice reading the two side by side. It writes them as `judged` edges into the inbox; you then triage them with `pending` / `promote` / `dismiss` exactly

as with any other judgement. It is the one write verb that reaches for the LLM, and it never asserts anything directly — its whole output is advisory by construction.

ask — answering by walking the graph

graph ask is the verb that made the graph worth building, and it gets its own section below, because it is not a read of the graph so much as a **conversation** with it.

Chatting with your graph

The graph is not only queried by verb. You can **converse** with it — ground a language model in the graph’s **relations**, not just the prose — so that you can ask how your book connects: what contradicts what, what grounds a claim, how a scene is sourced. There are two surfaces for this, and they differ in how far they let the model travel: one reads a single hop, the other walks.

The Graph AI scope — one hop, in the editor

You met the AI pane’s **scopes** in the assistant chapters — the setting, cycled with **F9**, that decides what the model is grounded in before it answers. The last scope on the cycle, after Editor, is **Graph**.

TERM The Graph scope

An AI-pane scope (F9) that grounds the model in the graph. It retrieves the passages relevant to your question — the same semantic retrieval Book scope uses — and, beneath each passage, folds in the graph edges touching it (**contradicts**, **sourced_from**, **links_to**, **cites**, ...). The answer stands on both the prose **and** those relations, under the same citation contract as Book scope. A **sticky** scope: it retrieves once per chat and re-grounds when you clear history.

A prompt in Graph scope answers from one hop out — the passages it retrieved and the edges directly on them. It carries the same discipline as Book scope: every claim cites a passage’s **[location/path]**, and an invented label is flagged rather than trusted. Press **p** in the AI pane to expand the “Retrieved passages

1. graph relations“ transparency section and read the exact subgraph the answer was built on — the graph never asks you to take its grounding on faith.

graph ask — the traversal loop on the CLI

● ● ● *Asking a question the flat text cannot answer*

```
inkhaven graph ask \
  "which of my claims about the harbour contradict?"
```

Where the Graph scope reads one hop, `graph ask` lets the model **walk**. It searches for seed nodes matching your question, then issues read-only graph queries — neighbours, contradictions, loci, paths — turn by turn, following the lines wherever they lead, until it has seen enough to answer. The exploration transcript prints to **stderr** so you can watch the path it took; the grounded answer goes to **stdout** so you can pipe it. It is honest by construction: when the relations simply do not record what you asked, it says so rather than inventing a connection. The graph is only ever as complete as `graph rebuild`, `confront`, and `graph link` have made it, and `ask` will not paper over a gap.

The cost of a question is bounded and tunable — this is the permissive principle, where the caps inform and never block. Two settings govern the walk.

● ● ● *Bounding the walk (hjson config)*

```
graph: {
  ask_max_steps: 8      // max LLM turns before answering
  ask_search_width: 6   // seed nodes per search
}
```

Walking the graph in the editor — Ctrl+B z

The same traversal runs **inside** the editor, streamed, through the graph hub. `Ctrl+B z` opens a small menu onto the graph with three entries, and it is the one place all of the graph's in-editor surfaces gather.

TERM The graph hub

The **Ctrl+B z** overlay — a three-way menu onto the knowledge graph. Press **n** for the **neighbourhood** of the paragraph you are editing (the same tree as **graph neighbors**, scrollable, **Esc** to close); **i** for the **edge inbox** (the advisory **judged** edges, where **P** promotes and **d** rejects the selected one); or **w** to **walk the graph** and answer the question typed in the AI prompt. Populate the graph first with **graph rebuild** / **graph lexical**.

The **w** walk is the streamed twin of **graph ask**. Type a question into the AI prompt, then **Ctrl+B z** and **w**. The AI pane shows the walk unfold live — each step as the model takes it,  **search**,  **neighbours**,  **contradicting** — and then streams the grounded prose answer, which lands as a normal chat turn. The status bar reads **graph walk · turn k/N** as it goes, and **Esc** stops the whole walk at any moment. The bounds are the same **ask_max_steps** and **ask_search_width** as the CLI. It is deliberately an **explicit** action — a walk is several model calls, not the single hop of the Graph scope — so the depth, and its cost, is something you opt into per question.

Triaging findings without opening the hub

One more pair of keys lives outside the hub. When a **confront finding** is sitting in the Output pane — a stance the fact-checker or continuity watch has proposed — you can act on its edge in place: **P** promotes the finding's stance edge to a kept decision that survives a rebuild, and **d** dismisses the finding and deletes its edge. It is the same promote/dismiss verdict as the inbox, on the finding where you first meet it.

F9	Cycle the AI scope; Graph sits last, after Editor.
Ctrl+B z	Open the graph hub (then n / i / w).
z then n	Neighbourhood of the open paragraph (tree, ↑↓ , Esc).
z then i	Edge inbox — P promotes, d rejects the selection.
z then w	Walk the graph to answer the AI-prompt question.
P / d	On an Output-pane confront finding: promote / dismiss its edge.
p	In the AI pane (Graph scope): expand the grounding subgraph.

From a script — the `ink.graph` words

The graph is scriptable from the embedded Bund language, which you will meet in full in the scripting part. The read and triage verbs are exposed as deterministic words; the LLM walk (`graph ask`) is deliberately **not** a sync word, because a word that fans out to a model is not something a script should call as if it were pure.

• • • *The `ink.graph` Bund surface*

```
ink.graph.stats      ( -- dict )  counts + per-kind
ink.graph.neighbors  ( node -- list ) one-hop edges
ink.graph.contradicting ( node -- list ) stance clashes
ink.graph.loci       ( node -- list ) cited loci
ink.graph.paths      ( from to -- list|nil ) a path
ink.graph.pending    ( -- list )   the judged inbox
ink.graph.rebuild     ( -- dict )   re-derive structure
ink.graph.promote     ( edge -- bool ) judged→promoted
ink.graph.dismiss    ( edge -- )   delete a stance edge
```

The read words are classified `STORE_READ` and the three that change the graph — `rebuild`, `promote`, `dismiss` — are `STORE_WRITE`, so the sandbox policy knows exactly what a script may touch. This is the surface you reach for when you want to bake a graph check into a headless pipeline: `rebuild`, read `pending`, and act on what it returns without opening the editor at all.

Multilingual by construction

The graph is multilingual because its **kinds** are language-neutral. A `contradicts` edge between a Russian fact and a French source is the very same kind of edge as any other — the relation does not know or care what language its endpoints are written in. Locus canonicalisation collapses `John 3:16`, `Иоанна 3:16`, and `Joh 3.16` to a single endpoint, so a citation in any language lands on the same node. And the lexical bridge is cross-lingual by design: because `translates` edges ride the interlingual index, a Russian sense and a German sense of one concept meet at the same ILI, and “what is the German word for this Russian concept?” becomes a reverse-index lookup rather than a translation call.

FICTION

For a novel with an invented world, the graph's value is the stance edges — the `contradicts` and `in_tension` lines confront lays down — and the neighbourhood view that shows a scene's links and mentions at a glance.

NON-FICTION

For non-fiction, the spine is `sourced_from`, `graded_as`, `cites`, and `cites_locus`: `graph paths` and `graph ask` let you trace a claim back to the source that grounds it, across the whole corpus.

The graph also feeds the **Inner-family readers** you met in the intelligences part. Their shared grounding now reports what the graph has established that no single declared-data store computes on its own — recurring character pairings (who keeps appearing together across your scenes) and the running count of unresolved factual contradictions. Relational context, in other words, surfaced where the readers can use it.

Storage and crash-safety

The graph lives in its own DuckDB store, `edges.db`, sitting beside `metadata.db`, `blobs.db`, and the `vectors/` directory under your project. Keeping it separate is deliberate: the graph is a distinct data layer with its own durability rules, and it can be rebuilt without touching anything else.

IT SURVIVES A KILL -9

Writes to `edges.db` are atomic — a batch of edges either lands whole or rolls back — so the store survives a `kill -9` mid-write and reopens consistent, to the 1.2.15 stability bar. Deleting or moving a node cascades its edges away, so no edge is ever left pointing at a node that is gone. And the `structural`, `derived`, and `imported` edges are provably rebuildable — so even a corrupted graph is never **lost data**. Run `graph rebuild` (and `graph lexical`) and it comes back.

That disposability is the quiet theme of the whole chapter. The graph is powerful — it holds relations the flat manuscript cannot express, and it lets a model walk them to answer questions the prose alone cannot — and yet the only part of it you can

truly **lose** is the part you authored: your assertions and the judgements you chose to promote. Everything else is a function of your book, and a function can always be recomputed. Build the graph freely; it will never hold your book hostage.

WHAT YOU LEARNED

- The knowledge graph is a **typed-edge** layer over the UUID nodes you already have — `(src) --kind--> (dst)` with a weight, a reason, an **origin**, and `attrs` — stored in its own `edges.db` and indexed on **both** endpoints so a neighbourhood is a query, not a scan.
- An edge's **origin** decides its durability: `authorial`, `promoted`, `judged`, and `imported` survive a rebuild; only the `structural` and `derived` projections are recomputed — so `graph rebuild` is always safe.
- `graph`
Three actions fill the graph: `graph rebuild` (structural edges), `lexical` (the WordNet bridge), and `judged` edges accumulated live from `confront / relate / graph link` into the **edge inbox**.
- The `graph` command reads it — `stats`, `neighbors`, `contradicting`, `loci`, `paths`, `pending` — and triages it — `promote`, `dismiss` — while `link` and `ask` reach for an LLM.
- You can **chat** with the graph: the **Graph** scope (F9) grounds an answer in one hop of relations, and `graph ask` — on the CLI, or streamed in-editor via `Ctrl+B z → w` — lets the model **walk** the graph, honestly saying so when the relations do not record what you asked.
- `ink.graph.*` scripts the read and triage verbs; the graph is multilingual by construction; and `edges.db` is atomic, cascade-safe, and — for everything but your own assertions — rebuildable, so it is never lost data.

CHAPTER 16

16

The Timeline

A long book keeps two clocks. One is the order in which the reader meets the scenes — the order of the pages. The other is the order in which the events actually happened in the world of the story, which a book with flashbacks, parallel storylines, and a prologue set three ages earlier will scramble on purpose. Most of the mistakes a novel makes about time are mistakes about the gap between those two clocks: a messenger who arrives before he could have ridden the distance, a character standing in two cities on the same afternoon, a midsummer feast three chapters after the first snow. Inkhaven's **timeline** is the machinery for keeping the second clock — a record of what happened **when**, in a calendar you invent, that the reading intelligences can hold up against the prose.

This is an opt-in layer. A project has no timeline until you ask for one, and nothing you have read so far changes when you do — the timeline sits over the paragraph hierarchy you already have, adding a thin skin of **event** metadata to selected paragraphs rather than a new place to store anything. This chapter covers the whole of it: what an event is and how you record one, how to define a calendar of your own devising and how Inkhaven reads dates written in it, the critique that catches the timeline’s internal slips, and — the reason the feature earns its keep — how the events you record feed the continuity watch and the knowledge tracker so that **when** becomes a fact those systems can check.

Turning the timeline on

The feature is off by default so that existing projects upgrade without surprise, and so that a book that has no need of a calendar is never nagged about one. You turn it on with a `timeline` block in the project’s `inkhaven.hjson`, and the smallest possible form is three lines and a preset.

● ● ● *The minimal timeline block in inkhaven.hjson*

```
timeline: {
  enabled: true
  default_track: "main"
  calendar: { preset: "gregorian" }
}
```

Two settings do the everyday work. `enabled` is the gate: with it `false` (or absent) every timeline command refuses politely rather than seeding events into a project that never opted in — `inkhaven event add` errors with a one-line reminder to set the flag. `default_track` names the storyline an event belongs to when you do not say otherwise; think of a track as a swim lane — a POV character, a parallel plot, a “flashback” line — that lets two events at the same moment sit on different rows instead of colliding. The `calendar` block is the substance, and the rest of this chapter’s first half is about filling it in.

WHY OPT-IN

A timeline only pays for itself once you have events in it, and events are work to record. Inkhaven would rather ask you to turn the feature on deliberately than have every project carry an empty calendar it never uses. A non-fiction manuscript, a book of poems, a reference manual — none of these wants a story clock, and none of them is asked to.

What an event is

An event is not a new kind of object with its own file and its own database table. It is a **paragraph** — an ordinary node in your book’s tree — that carries an extra parcel of metadata marking it as something that happened at a moment in story-time. That parcel is small and entirely legible.

TERM Event

An **event** is a paragraph tagged with a start on the calendar and, optionally, an end, a precision, a track, and links to the characters and places it involves. Under the hood it is a block of `EventData` hung on a normal node: a start tick, an optional end tick, a **precision** (how exact the date is), a track label, a list of character ids, and a list of place ids. Everything else about the paragraph — its title, its prose, its position in the tree — is unchanged.

The heart of the parcel is a single number. Every moment on the timeline is stored as one signed 64-bit integer — a count of **ticks** since the calendar’s epoch, negative for anything before it, so a prologue set in a prior age is just a negative tick. All the arithmetic the timeline ever does is integer arithmetic on that number; every scrap of calendar complexity — months of uneven length, named moons, seasons that wrap the year — lives in the **display** layer that converts a tick to a human string and back. An event that is an instant carries a start tick and no end; an event with duration carries both, and the span it occupies is the closed interval between them.

Where events live — the Timeline chapter

The first time you add an event under a given book, Inkhaven creates a chapter called **Timeline** inside that book to hold it, and every later event under the same book goes into the same chapter. It is created lazily — a book you never add an

event to never grows one — and it is marked internally with a system tag rather than by its title, so you can rename it with **F2** and the timeline machinery still finds it. Different books each get their own.

● ● ● *The Timeline chapter, created on the first event add*

```
Aerin Saga/
├─ Chapter 1/
├─ Chapter 2/
└─ Timeline/           ← lazily created on first event add
    ├─ Birth of Aerin   ← an event paragraph (carries
EventData)
    ├─ Storm of Year 1
    └─ Marketplace scene
```

This is worth a moment's thought, because it explains a distinction you will lean on. The event paragraph in the Timeline chapter is the **record** that the event happened; it is not usually the same paragraph as the **scene** that depicts it, which lives out in Chapter 4 where the reader meets it. The two are joined by a **link** — the event's list of linked paragraphs — and that link is what lets a scene inherit its date from the event, and what lets the swim-lane view show a Storm event under Chapter 4 even though the event record sits in the Timeline chapter. An event with no such link, and no characters or places either, is an **orphan**: a moment recorded but attached to nothing, which the tooling marks so you can decide whether it wants a scene or was a stray.

Recording an event

There are three ways to record an event, and they differ only in where you are standing when you do it. The most explicit is the command line, which is also the clearest way to learn the shape of the thing.

● ● ● Adding events from the command line

```
# An instant event – precision inferred from the date shape.
inkhaven event add "Marketplace scene" \
  --start "1A.2.8" --book-name "Aerin Saga"

# A duration event – a start and an end.
inkhaven event add "The Storm" \
  --start "1A.2.3" --end "1A.2.5" \
  --track main --book-name "Aerin Saga"

# --precision overrides what the parser would infer.
inkhaven event add "Spring of Rain" \
  --start "1A.spring" --precision season \
  --book-name "Aerin Saga"
```

The `--start` string is a date written in your calendar, and its **shape** decides the event's precision unless `--precision` overrides it — a point we return to below. `--end` makes the event a duration; leave it off and the event is an instant. `--track` places the event on a storyline, defaulting to your `default_track`. `--book-name` names the book (by slug or title, case-insensitively) and is required only when the project holds more than one. Inkhaven refuses an end that falls before its start — events do not run backwards — and prints back what it recorded, the date rendered in your calendar's own display form.

Inside the editor the everyday way is a chord. `Ctrl+V Shift+E` records an event from wherever you are: with text selected in the Editor, the selection becomes the event's title; on a paragraph row in the Tree, that paragraph's title pre-fills it; inside the timeline view it drops the event at the cursor's tick. This is the flow that fits actual writing — you are drafting a scene, you select the sentence that anchors it in time, you strike the chord, you type the date, and the event is on the timeline without your hands leaving the keyboard. Inside the swim-lane view the bare `n` key does the same at the cursor's position.

FICTION

A novelist records events as the plot demands them: the coronation, the siege, the two-week ride. The track label is where parallel POV lines and flashbacks earn their keep — give each its own track and the swim lanes stop colliding.

NON-FICTION

A non-fiction writer rarely needs a story clock at all. Where a work is genuinely chronological — a history, a biography, a project retrospective — the same events + calendar apparatus will hold real dates on the **gregorian** preset, and the critique still catches the orphan and the impossible overlap.

Linking, characters, and places

A freshly added event is bare — it has a date and nothing else — which is why it starts life as an orphan. You give it substance by linking it to the things it touches. From the swim-lane view, **Enter** on an event opens the picker that attaches a manuscript paragraph — the scene that depicts the event — and saving that link drops the orphan mark. Characters and places attach the same way, pointing the event at entries under the book's Characters and Places system books; those two lists are exactly what the co-location check and the knowledge tracker read later, so an event that names its participants is an event the intelligences can reason about. The links are also scriptable, through the `ink.event.link_paragraph` word covered at the end of this chapter.

ORPHANS ARE A NUDGE, NOT AN ERROR

An orphan event — no linked paragraph, no characters, no places — is drawn with a hollow ○ glyph everywhere it appears, and opening one lands a hint on the status bar telling you the one chord that will link it. It is a soft signal that a recorded moment might want a scene, not a mistake to be corrected. Deliberate backstory that will never get a scene is free to stay orphaned forever.

A calendar of your own devising

The calendar is the part of the timeline you actually design. Its job is to convert between the single tick number the timeline stores and the dates a person writes

and reads, and Inkhaven gives you three ways to specify one: two presets for the common shapes and a **custom** form for a world of your own.

● ● ● *The three calendar flavours*

```

preset: "gregorian"    real-world dates    →  1917.11.7 ·
2026.5.20
preset: "sols"         days since day zero →  Sol 1 · Sol 142
preset: "custom"       anything you invent →  1A.Highsun.15

```

The **gregorian** preset is the real world: years of twelve named months, months of thirty days (a deliberate simplification — the timeline measures spans, not leap-years), and the four seasons already wired up, so a scene dated to month seven reads as summer without any further declaration. The **sols** preset is the opposite extreme — a single unit, “days since day zero”, displayed as **Sol N** — which suits a survival log, a voyage, or any story that counts days from a fixed beginning and needs nothing finer.

Building a custom calendar

A custom calendar is a **stack of units**, declared base-first. The first unit is the base — one tick is one of these — and each unit above it declares how many of the level below make one of itself. A day, then a month of thirty days, then a year of twelve months, is the familiar shape; but the counts are yours, and so are the names.

```
calendar: {
  preset: "custom"
  base_unit: "day"
  units: [
    { name: "day",   names: [] }
    { name: "month", per_parent: 30,
      names: ["Frostmoon", "Snowfall", "Greenstart",
              "Bloomtide", "Highsun", "Goldfall",
              "Mistwane", "Stormrise", "Coldgate",
              "Longnight", "Hearthlit", "Yearfall"] }
    { name: "year",  per_parent: 12, names: [] }
  ]
  seasons: [
    { name: "winter", start_month: 1, span_months: 3 }
    { name: "spring", start_month: 4, span_months: 3 }
    { name: "summer", start_month: 7, span_months: 3 }
    { name: "autumn", start_month: 10, span_months: 3 }
  ]
  epoch_label:      "A"      # 1A.3.15 = First Age, year 1
  epoch_before_label: "BA"    # negative years, e.g. -1BA
  display_format:    "{year}{epoch_label}.{month}.{day}"
}
```

Four pieces here repay attention. A unit's `names` list gives its values display names — the twelve moons above turn month 5 into **Highsun** — and an empty list falls back to a bare number, so a numeric month renders `1.5.15` while a named one renders `1A.Highsun.15`. The `seasons` list names spans of months and is what gives a date its season; each season declares the month it starts on and how many months it covers, and a season may wrap the year boundary (winter over the turn) without confusing the arithmetic. The two `epoch_label` fields are the suffixes for years at or after the epoch and before it — leaving `epoch_before_label` empty makes the calendar reject negative years outright, which is the right choice for a world with no prehistory. And `display_format` is the template every date is rendered through, with `{year}`, `{month}`, `{month-name}`, `{day}`, and the epoch tokens as its placeholders. A `parse_aliases` list, not shown, lets you name landmark dates — **Founding**, **Day Zero** — that resolve to a fixed tick.

How a date is read — precision from shape

Here is the idea that makes the calendar more than decoration. When you write a date, Inkhaven infers not only **which** moment you meant but **how exact** you were being, and it reads that exactness from the **shape** of what you typed. A date that stops at the year is a year-precise date; one that names a month is month-precise; one that runs down to the day is day-precise. That inferred precision — **Tick**, **Hour**, **Day**, **Week**, **Month**, **Season**, or **Year** — is stored with the event and decides how wide a **window** the event occupies when the critique asks whether two events could overlap.

TERM Precision

A date's **precision** is how exact you were when you wrote it, inferred from its shape. A day-precise date is a point — a one-day window. A season-precise date is a whole season — a window ninety days wide, in a thirty-day-month calendar. Precision is why “sometime that spring” and “the fifteenth of Highsun” behave differently: the first can collide with other vague dates; the second is a pin. **--precision** overrides the inference when you want to be explicit.

The parser walks the dotted segments of a date under the year — month, then day, then hour — and the last segment you supply sets the precision. A season **name** in the month slot is read as season-precision; a landmark alias resolves to its fixed tick at day-precision. The table below reads the Aerin calendar above.

● ● ● Parsing the Aerin calendar — shape sets precision

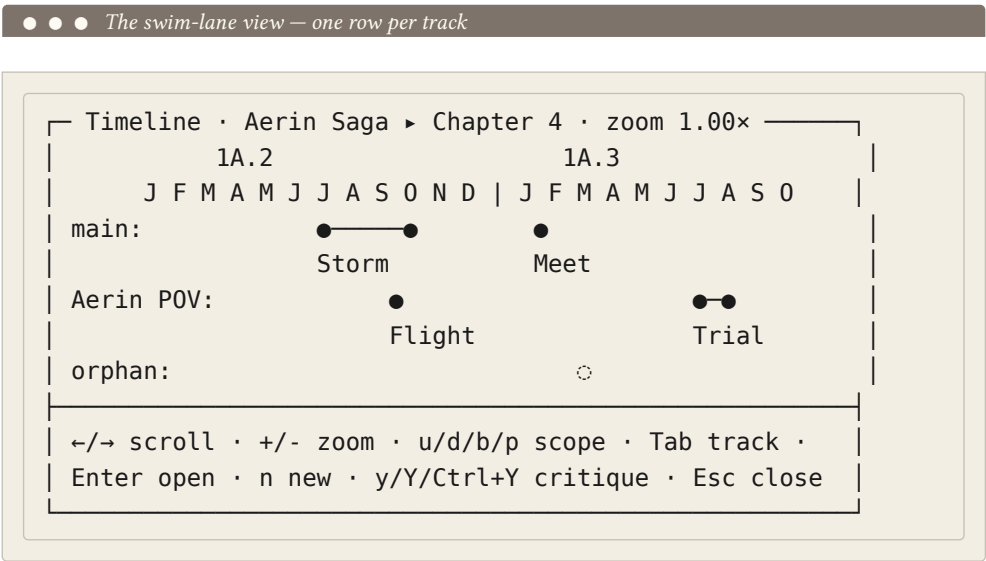
you type	parsed as	precision	ticks	
-----	-----	-----	-----	
1A	year 1	Year	0	
1A.3	year 1 mo 3	Month	60	
1A.Frost	Frostmoon	Month	0	(prefix
match)				
1A.spring	spring	Season	90	
1A.3.15	day 15	Day	74	
-1BA	year -1	Year	-360	
Founding	alias	Day	0	

Two conveniences fall out of this. Month names match on a unique prefix, so `1A.Frost` finds **Frostmoon** without your typing it out; and a date is symmetric through parse and format, so a tick rendered to a string and read back lands on the same tick. The one value the calendar forbids is a year of zero — there is no year 0 between `1A` and `-1BA`, the epoch being year 1 — and Inkhaven says so plainly rather than guessing.

Seeing the timeline

Two views show you the events you have recorded, and both are reached with the view prefix. This manual keeps them brief — the companion book **Building the World** walks every chord — but you should know they exist and what each is for.

`Ctrl+V e` opens a **chronological picker**: a vertical, time-ordered list of every event, filterable by track, from which `Enter` loads an event’s paragraph into the Editor. It is the fast way to jump to a known event. `Ctrl+V Shift+T` opens the **swim-lane view**, the headline UI: a horizontal timeline with one row per track, opened at your current paragraph’s nearest scope, which you scroll, zoom, and drill into by chapter. In both, the same three glyphs read the same way — a filled dot for an instant, a bar for a duration, a hollow ring for an orphan.



The one rule worth carrying out of these views is how **scope** filters events. At book scope every event shows. At a narrower scope — a chapter, a subchapter — an event

appears if it **or any paragraph it links to** lives inside that scope. So a Storm event whose record sits in the Timeline chapter still shows when you scope into Chapter 4, because the scene it links to is there. That is the link doing its second job: not only dating the scene, but placing the event where the scene is read.

The timeline critique

The critique is the timeline watching **itself** — the checks that depend on the timeline’s own semantics and belong to no other system. It once did more; over several releases, four of its original concerns moved to systems that do them better (we come to those next), leaving two genuinely timeline-internal checks. Both are pattern-based, deterministic, and free — no model is called — and both emit their findings to the Output pane alongside the fact-checker’s and the continuity watch’s, with the same severity glyphs and the same **Enter**-to-jump.

Orphan events

The first check finds **orphans** — events linked to nothing, no paragraph, no character, no place — and grades how much the orphan matters. A trivial stub recorded five minutes ago is barely a note; a four-word, day-precise event that has sat unconnected for three months is a likely authorial slip. The grade combines **significance** — read from how concrete the date is, how rich the title, and how active the event’s track — with **staleness**, how long the orphan has gone unlinked past a grace window you can set. High significance is a contradiction-level finding at any age; a middling orphan warns once it goes stale; a low one stays a passing note.

Fuzzy-precision overlap

The second check is the one the precision system was built for. Two events with coarse dates — a season here, a month there — each occupy a **window** rather than a point, and when two such windows collide **suspiciously** the prose probably cannot place both events consistently. Suspicion is not mere overlap; it is overlap plus a reason to worry — the two events share a track, or share a character or place. A same-track, shared-character collision is a warning; a loose cross-track brush is filtered out below the threshold. When three or more fuzzy events mutually overlap on a common window, Inkhaven reports the **cluster** once instead of drowning you in pairwise noise.

● ● ● Critique findings in the Output pane

```

└─ Output · 3/3 · timeline-critique ───
| ⊗ orphan_event
|   "The Lost Coronation Rite" – day-precise, 4 words,
|   orphaned 92 days. Link a scene or retire it.
| △ fuzzy_overlap
|   "Training" and "Journey" (season-precise, main,
|   both with Mara) place in the same window.
| ● overlap_cluster
|   3 season events share a window at Velmaril.
|
| ───────────────────────────────────
| ↑↓ select · Enter jump · a ask-AI · d dismiss

```

You run the critique over a **scope**, and four chords set how wide. Inside the swim-lane view, **y** runs it over the current view for the highlighted track only, **Y** over the current view for all tracks, **Ctrl+Y** widens to the whole book, and **F12** runs it over every event in the project. There is also a unified pass: **Ctrl+B Shift+C** runs every fast, deterministic checker at once — the fact-checker and the Socratic reader over the open paragraph, and the timeline critique over the project — and drops all their findings into the Output pane together. On the command line, **inkhaven event critique** prints the same two checks, scoped with **--track** or **--book-name**, and its findings are localized to the project's working language.

WHAT MOVED OUT OF THE CRITIQUE

Four checks that once lived here now live where they belong, and knowing where saves you looking in the wrong place. **Travel-time** (a journey the calendar says was too fast) and **co-location** (a character in two places at once)

inkhaven realworld fact-check

are the world fact-checker's, run by **--timeline-aware** and **inkhaven realworld co-location**. **Date coherence** (a midsummer feast in a winter scene) is the fact-checker's too. **Pacing** — how densely events pack the page — is the Inner Socratic reader's. The critique keeps only what is irreducibly about the timeline's own structure.

How the timeline feeds the other intelligences

The timeline earns its place not by what it shows you but by what it lets the rest of the tool **know**. Once events carry dates, participants, and places, three of Inkhaven's reading intelligences can ask questions of the prose that no generic reader could — because the answers depend on a story clock that only you have declared.

The fact-checker reads when, not just what

The world fact-checker knows your world's geography and climate; with a timeline it also knows **when** each scene happens. A paragraph linked to an event, or near one in world-time, gains an **effective date** — the date of the event it links to — and from that date, a **season**.

TERM Effective date

A paragraph's **effective date** is the world-time the checker reads it at: the start of the earliest event linked to it. From the effective date the checker derives the **season** per your calendar, and against that it can judge whether the weather, the light, and the festivals a scene describes belong to the time the timeline places it. A paragraph with no linked event has no effective date, and the timeline simply stays quiet about it.

So snow in a scene the timeline dates to summer stops being ambiguous and becomes a contradiction the checker can name; a three-day ride the calendar says took thirty-five is a warning it can raise. These timeline-aware findings carry a small calendar mark in the Output pane to tell you the clock was consulted, and they run in every project language.

SENTINEL continuity uses event placement

The continuity watch — the system that watches a book for the things a long manuscript forgets — folds the timeline's placement into its deterministic findings. **Co-location** is the clearest case: from nothing but the events' participant and place lists, the watch finds a character whose overlapping events put them in two different places at once, and reports it without reading a word of prose. The same event placement backs the date-coherence and travel-time findings the continuity ledger surfaces. When your world genuinely allows the impossible — a teleporting

mage, an astral projection — a rule in the **magic ledger** excuses the conflict with a note instead of nagging.

KEN uses event participation for who was present

The knowledge tracker — Inkhaven’s watch over **who knows what, and when** — draws one of its two grant sources straight from the timeline. Its principle is that a character **present at an event knows what happened there**, from the moment of the scene that depicts it onward. So every character in an event’s participant list is granted knowledge of that event’s subject, dated to the event’s first linked paragraph. That grant is what lets the tracker catch a character who refers to the coronation before any scene could have told them of it — **premature knowledge** — the referenced-before-introduced invariant extended from things to **knowledge**. The event’s title is the topic by default; a **reveals:** tag on the linked scene can name the topic more precisely when the title is terse. This is the timeline’s deepest contribution: presence at an event, which only your timeline records, becomes the ground truth for what a character could plausibly know.

The CLI and the Bund word

Everything the timeline does interactively is also scriptable and headless, which is what lets a timeline live under version control and take part in a batch run.

The **inkhaven event** subcommand is the command-line surface: **add** records an event (with **--start**, **--end**, **--precision**, **--track**, **--book-name**), **list** prints every event in chronological order (filterable by **--book-name** and **--track**), **show** prints one event’s full detail by its slug-path, and **critique** runs the two retained checks. Every one of them refuses unless **timeline.enabled** is **true**, so you cannot seed events into a project that has not opted in.

● ● ● *inkhaven event list — the chronological view*

```
$ inkhaven event list --book-name "Aerin Saga"
  1A.1.1 ○ Birth of Aerin          track=main .../birth
  1A.2.3–2.5 – The Storm          track=main .../storm
  1A.2.8 ● Marketplace scene     track=main .../market
```

From Bund, the `ink.event` family gives scripts the same operations. Seven words cover the lifecycle — listing, adding, and refining an event — and each mutation fires a hook so other scripts can react.

● ● ● *The `ink.event.*` Bund words*

```

                                ink.event.list          ( -- list )
                                ink.event.list_orphans   ( -- list )
    "Saga" "Storm" "1A.2.3"
                                ink.event.add            ( book title spec --
    uuid )
    id "1A.2.5"      ink.event.set_end      ( uuid spec -- )
    id "season"      ink.event.set_precision ( uuid prec -- )
    id "main"        ink.event.set_track    ( uuid track -- )
    id "saga/ch4/storm"
                                ink.event.link_paragraph ( uuid path -- )

```

`ink.event.add` takes a book name, a title, and a start spec, creates the event under that book's Timeline chapter (materialising the chapter if need be), infers precision from the spec's shape exactly as the CLI does, and returns the new event's id — which the `set_*` and `link_paragraph` words then refine. The `list` words return dictionaries carrying every field of an event: its id, title, slug, path, start and end ticks, precision, track, orphan flag, and its linked paragraphs, characters, and places. Reads are default-allowed; the mutating words sit behind Bund's `store_write` category, opt-in like every other write. A companion set of `ink.event.critique.*` words exposes the two checks to scripts, and two hooks — `hook.on_event_added` and `hook.on_event_orphaned` — fire on the lifecycle so a script can, say, tag every new event or flag one that has just lost its last link.

WHAT YOU LEARNED

- The **timeline** is an opt-in layer (`timeline.enabled: true`) that records what happened **when**, as **events** — paragraphs carrying a start on a calendar, an optional end, a precision, a track, and links to characters and places.
- Every moment is one signed integer **tick**; a **calendar** — a `gregorian` or `sols` preset, or a **custom** stack of named units with seasons and epoch labels — converts ticks to dates and back, and a date's **shape** sets its **precision**.
- Events live in a lazily-created **Timeline chapter** per book; an event linked to nothing is an **orphan** (○), a nudge rather than an error. Record events with `inkhaven event add`, `Ctrl+V Shift+E`, or `n` in the swim-lane view.
- The **critique** keeps two timeline-internal checks — **orphan events** graded by significance and staleness, and **fuzzy-precision overlaps** that collide suspiciously — run by `y/Y/Ctrl+Y/F12` or `inkhaven event critique`; travel-time, co-location, date coherence, and pacing moved to other systems.
- The timeline **feeds the intelligences**: the fact-checker reads each scene's **effective date** and season, SENTINEL uses event placement for co-location and date coherence, and KEN turns **presence at an event** into who could know what.
- Everything is scriptable — the `inkhaven event` subcommands and the `ink.event.*` Bund words (with `hook.on_event_added` / `hook.on_event_orphaned`) — so a timeline works headless and under version control.

PART V

The Intelligences

CHAPTER 17

17

Continuity and Knowledge

The two continuity intelligences that watch a manuscript: SENTINEL (the book watches itself) and KEN (who knows what, when). How to run each and read its ledger.

CHAPTER 18

18

The Read-Through and the Voices

LECTOR reads the book forward as a first reader; CHORUS profiles the cast's voices; the dialogue and narrator-voice readers round out how it sounds.

CHAPTER 19

19

Revision and History

Turning findings into confirmed changes with REDLINE's Editorial Pass, and measuring whether a revision helped with CHRONICLE's draft history.

CHAPTER 20

20

The Inner Family

The readers who question rather than answer — Inner Socrates, Editor, Theologian, Poet, and the reasoning-rigor reader — and how to engage each.

PART VI

Language, Verse & Scholarship

CHAPTER 21

21

Constructed Languages

Building an invented language inside the editor — phonology, lexicon, morphology — and the WordNet thesaurus for real-world prose.

CHAPTER 22

22

Poetry

Measuring verse against a declared form — metre, rhyme, syllables — with a reader that observes and never generates.

CHAPTER 23

23

Research and Scholarship

Grounding a book in fact: the Research Assistant, the Sources bibliography, terminology governance, and the scholarly apparatus (loci, verborum, argument outline).

PART VII

Producing the Book

CHAPTER 24

24

Assembly and PDF

Turning the tree into a Typst-compilable directory and a finished PDF — the two-chord path from buffer to book.

CHAPTER 25

25

EPUB, Web and More

The other destinations: EPUB, an HTML website, the arXiv scientific bundle, DOCX, and the audiobook.

CHAPTER 26

26

Technical Documentation

The features for technical and reference books: the `docs` verification checks and the back-of-book index.

PART VIII

Scripting with Bund

CHAPTER 27

27

Scripting with Bund

The embedded scripting language: hook lambdas, the `ink.*` standard library, Bund Script nodes, and the sandbox policy that keeps them safe.

PART IX

Keeping It Healthy

CHAPTER 28

28

Keeping It Healthy

Backups and restore, the `reindex` command, the project doctor, and the advisory project lock — keeping the store sound over years of use.

CHAPTER 29

29

Configuration

A tour of `inkhaven.hjson` — how the layered config works, the blocks you are most likely to touch, and how to edit it safely from inside the editor.

APPENDIX A



The Keybinding Map

Every chord the TUI honours, by pane and overlay — the printable reference.
Mirrors the canonical `KEYBINDING.md`.

APPENDIX B

B

The Command Line

Every `inkhaven` subcommand at a glance, grouped by what it does.

APPENDIX C

C

The Configuration Reference

Every `inkhaven.hjson` block and field, with its default. Mirrors the canonical `CONFIGURATION.md`.

APPENDIX D

D

The Feature Index

The canonical map — every feature to its command, chord, doc, and scripting namespace.

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is the art of **holding a system too large for one head** — of watching it without losing the thread, and never reporting a thing it cannot back up. It is not an accident that a tool he built for writers carries the same instinct to the desk: a book, past a certain size, is exactly such a system, and it too

deserves an instrument that holds it for you.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn’t fit the shape of the work.

Why Inkhaven exists

Inkhaven is Vladimir’s personal reflection on how a single tool can hold the **whole** of writing a book. The frustration it answers is one every long-form writer knows: that the work is scattered across a dozen apps that don’t talk to each other — a browser for research, one program for the prose, another for the characters, a spreadsheet for the word count, a design tool for the final typesetting — and every seam between them is a place the book leaks.

Inkhaven’s wager is that all of it belongs in one place, next to the keyboard: the prose and the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF. Not because integration is elegant, but because a book past a certain size outgrows the mind writing it, and a tool that holds your words should also be able to **hold the book** — its facts, its continuity, its shape — so the writer is free to do the one thing no tool can, which is write.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don’t.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an “Enterprise” tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.

